

索引 (Index)

原书：《Learning Python》（6th Edition），作者 Mark Lutz。本页为原书书末索引（第 1806–1976 页）的逐条整理与中英对照：每条索引按首字母分组，粗体为索引词条（附中文），其后为讨论位置（附中文）。使用说明：按字母顺序检索。词条与位置均中英对照；英文保留以对齐原书/代码，中文便于快速理解。中文说明：每条为「英文词条（中文）」；位置名为「英文（中文）」，与原书索引一一对应。

Symbols (符号)

- **# comments (# 注释)** : # Comments (# 注释)
- **\$ character in interactive coding (交互式编程中的 \$ 字符)** : What Not to Type: Prompts and Comments (不要键入的内容：提示符与注释)
- **(asterisk) use in code (代码中 (星号) 的用法)** : Application to for loops (应用于 for 循环)

A (字母 A)

- **absolute imports (绝对导入)** : Relative and Absolute Imports (相对导入与绝对导入)
- **abstract superclasses (抽象超类)** : Abstract Super

- classes (抽象超类) ; Preview: Abstract superclasses with library tools (预览: 用库工具实现抽象超类) ; Stream Processors Revisited (再访流处理器)
- library tools (库工具) : Preview: Abstract superclasses with library tools (预览: 用库工具实现抽象超类)
 - access-by-key files (按键访问文件) : Other File Tools (其它文件工具)
 - aggregation (聚合)
 - composition (组合) : OOP and Composition: "Has-a" Relationships (OOP 与组合: "有一个" 关系)
 - object (对象) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
 - AI (artificial intelligence) (人工智能) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
 - Ajax (Asynchronous JavaScript and XML) (异步 JavaScript 与 XML) : Internet and Web Scripting (互联网与 Web 脚本)
 - aliasing (别名 (共享引用)) : Arguments and Shared References (参数与共享引用)
 - all variable (all 变量) : Minimizing from Damage: X and all (最小化 from 的破坏: X 与 all)
 - Android (Android 系统)
 - interactive coding and (与交互式编程) : Starting an Interactive REPL (启动交互式 REPL)

- Python installation (Python 安装) : Installing Python on (安装 Python)
- Python on (上的 Python) : Using Python on Android (A.5 在 Android 上使用 Python)
- annotations, functions (函数注解) : General Function Concepts (通用函数概念) ; Function Annotations and Decorations (函数注解与装饰) ; Function decorators alternative: Preview (函数装饰器替代方案: 预览)
- anonymous functions (匿名函数) : lambda Makes Anonymous Functions (lambda 创建匿名函数)
- AOT (ahead-of-time) compilers (AOT (提前) 编译器) : Ahead-of-Time Compilers for Speed (为速度而生的提前编译器)
- PyThran (PyThran 科学计算加速) : Python Implementation Alternatives (Python 实现替代方案)
- Shed Skin (Shed Skin Python 转 C++) : Python Implementation Alternatives (Python 实现替代方案)
- arbitrary arguments, argument matching (任意参数, 参数匹配) : Arbitrary Arguments Examples (任意参数示例) ; Why arbitrary arguments? (为什么需要任意参数?)
- arbitrary expressions (任意表达式) : Sequence Operations (序列操作)
- arbitrary scope nesting (任意深度的作用域嵌套) : A

- arbitrary Scope Nesting (任意深度的作用域嵌套)
- architecture (架构) : Python Program Architecture (Python 程序架构)
- argument matching (参数匹配) : Special Argument-Matching Modes (特殊的参数匹配模式) ; Argument Matching Overview (参数匹配概览)
- arbitrary arguments (任意参数) : Arbitrary Arguments Examples (任意参数示例) ; Why arbitrary arguments? (为什么需要任意参数?)
- defaults (默认值) : Argument Matching Syntax (参数匹配语法)
- function calls (函数调用) : Argument Matching Syntax (参数匹配语法)
- unpacking arguments (解包参数) : Calls: Unpacking arguments (调用: 解包参数) ; Calls: (调用:)
- function definitions (函数定义) : Argument Matching Syntax (参数匹配语法)
- collecting arguments (收集参数) : Definitions: Collecting arguments (定义: 收集参数)
- generalized set functions example (通用集合函数示例) : Example: Generalized Set Functions (示例: 通用集合函数) ; Testing the Code (测试代码)
- keyword-only arguments (仅关键字参数) : Keyword-Only Arguments (仅关键字参数) ; Why keyword-only arguments? (为什么需要仅关键字参数?)

- keywords (关键字) : Argument Matching Syntax (参数匹配语法) ; Keyword and Default Examples (关键字与默认值示例) ; Combining keywords and defaults (组合关键字与默认值) ; Test Your Knowledge: Answers (测验答案)
- minimum value calculation (最小值计算) : Example: The min Wakeup Call (示例: min 唤醒测试) ; The Punch Line (点睛之笔)
- mins.py example (mins.py 示例) : Example: The min Wakeup Call (示例: min 唤醒测试) ; The Punch Line (点睛之笔)
- passing arguments (传递参数) : Argument Passing Details (参数传递细节)
- positional-only arguments (仅位置参数) : Positional-Only Arguments (仅位置参数)
- print function example (print 函数示例) : Example : Rolling Your Own Print (示例: 自己动手写 print) ; Using Keyword- Only Arguments (使用仅限关键字参数)
- syntax (语法) : Argument Matching Syntax (参数匹配语法) ; lambda Basics (lambda 基础)
- argument ordering (参数顺序)
- function calls (函数调用) : Calls Ordering (调用时的参数顺序)
- boundary cases (边界情况) : Boundary cases (边

界情况)

- formal definitions (形式定义) : Formal definition (形式定义)
- function definitions (函数定义) : Definition Ordering (定义时的参数顺序)
- boundary cases (边界情况) : Boundary cases (边界情况)
- formal definitions (形式定义) : Formal definition (形式定义)
- argument passing (参数传递) : Argument-Passing Basics (参数传递基础)
- defaults (默认值) : Keyword and Default Examples (关键字与默认值示例) ; Combining keywords and defaults (组合关键字与默认值)
- function calls (函数调用) : Argument Passing Details (参数传递细节)
- function definitions (函数定义) : Argument Passing Details (参数传递细节)
- immutable arguments (不可变参数) : Argument-Passing Basics (参数传递基础)
- keywords (关键字) : Keyword and Default Examples (关键字与默认值示例) ; Combining keywords and defaults (组合关键字与默认值)
- matching arguments (匹配参数) : Argument Passing Details (参数传递细节)

- multiple results simulation (多返回值模拟) : Simulating Output Parameters and Multiple Results (模拟输出参数与多结果返回)
- mutable arguments (可变参数) : Argument-Passing Basics (参数传递基础)
- changes (修改) : Avoiding Mutable Argument Changes (避免可变参数被修改)
- output parameter simulation (输出参数模拟) : Simulating Output Parameters and Multiple Results (模拟输出参数与多结果返回)
- references, shared (共享引用) : Arguments and Shared References (参数与共享引用)
- arguments (参数) : def Statements (def 语句) ; Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- arbitrary (任意) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- decorators (装饰器) : Decorator Arguments (装饰器参数) ; Adding Decorator Arguments (添加装饰器参数)
- callables (可调用对象) : Decorator Arguments (装饰器参数)
- class decorators (类装饰器) : Decorator arguments (装饰器参数)

- descriptors (描述符) : Descriptor method arguments (描述符方法参数)
- immutable (不可变) : Argument-Passing Basics (参数传递基础)
- keyword-only (仅限关键字) : Using Keyword-Only Arguments (使用仅限关键字参数)
- mutable (可变) : Argument-Passing Basics (参数传递基础)
- coupling (耦合) : Function Design Concepts (函数设计概念)
- passing (传递) : General Function Concepts (通用函数概念)
- passing functions as (把函数当作参数传递) : The First-Class Object Model (一等对象模型)
- positional (位置式) : A Basic Range-Testing Decorator for Positional Arguments (基础版: 仅支持位置参数)
- scopes (作用域) : Scopes and Argument Defaults (作用域与参数默认值) ; Loops Require Defaults, Not Scopes (循环需要默认值, 而不是作用域)
- super function (super 函数) : Same argument lists (相同的参数列表)
- validation (校验) : Example: Validating Function Arguments (案例二: 验证函数参数) ; The Goal (目标)

- ArithmeticError exception class (ArithmeticError 异常类) : Built-in Exception Classes (内置异常类)
- arrays, associative (关联数组) : Dictionaries (字典)
- artificial intelligence (AI) (人工智能 (AI)) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- as clause (as 子句) : The as Extension for import and from (import 与 from 的 as 扩展)
- ASCII (ASCII 编码) : Character Representations (字符表示)
- Latin-1 and (与 Latin-1) : Character Encodings (字符编码)
- UTF-8 and (与 UTF-8) : Character Encodings (字符编码)
- aspect-oriented programming (面向切面编程) : Why Decorators? (为什么使用装饰器?)
- assert statement (assert 语句)
- as conditional raise statement (作为有条件的 raise) : The assert Statement (assert 语句)
- constraint trapping (约束捕获) : Example: Trapping Constraints (but Not Errors!) (示例: 捕获约束条件——但不是错误!)
- assignments (赋值总论)

- argument passing (参数传递) : Argument-Passing Basics (参数传递基础) ; Simulating Output Parameters and Multiple Results (模拟输出参数与多结果返回)
- augmented (增强赋值) : Assignment Syntax Forms (赋值语法形式) ; Augmented Assignments (增强赋值) ; Augmented assignment and shared references (增强赋值与共享引用)
- basic assignment (基本赋值) : Basic Assignments (基本赋值)
- class attributes (类属性) : General Syntax and Usage (一般语法与用法)
- extended-unpacking (扩展解包) : Assignment Syntax Forms (赋值语法形式) ; Extended-Unpacking Assignments (扩展解包赋值) ; Extended unpacking in action (扩展解包实战)
- boundary cases (边界情况) : Boundary cases (边界情况)
- for loops (for 循环) : Application to for loops (应用于 for 循环)
- function calls (函数调用) : Application to for loops (应用于 for 循环)
- function headers (函数头部) : Application to for loops (应用于 for 循环)
- implicit (隐式) : Assignments (赋值总论)

- from (from (导入自)) : Changing mutables in modules (在模块中修改可变对象)
- import (import (导入)) : Changing mutables in modules (在模块中修改可变对象)
- lists (列表) : Assignment Syntax Forms (赋值语法形式)
- module attribute creation (模块属性的创建) : How Files Generate Namespaces (文件如何生成命名空间)
- multiple-targets (多目标) : Assignment Syntax Forms (赋值语法形式) ; Multiple-Target Assignments (多重目标赋值)
- shared references (共享引用) : Multiple-target assignment and shared references (Multiple-target assignment 与共享引用)
- mutables (可变对象) : Common Coding Gotchas (常见编码陷阱)
- name references, nested scopes (名字引用与嵌套作用域) : Nested Scopes Overview (嵌套作用域概述)
- named assignment expression (命名赋值 (海象) 表达式) : Assignment Syntax Forms (赋值语法形式) ; Named Assignment Expressions (命名赋值表达式) ; When to use named assignment (When 到用法命名赋值 (海象))
- names (名字) : Assignments (赋值总论) ; Scopes Overview (作用域概述) ; Name Resolution: The LEG

B Rule (名字解析: LEGB 规则)

- namespaces (命名空间) : The “Zen” of Namespaces: Assignments Classify Names (命名空间的“禅意” : 赋值给名字分类)
- object references (对象引用) : Assignments (赋值总论)
- scopes (作用域) : Preview: Other Python scopes (预览: Python 的其他作用域)
- sequence assignments (序列赋值) : Sequence Assignments (序列赋值) ; Advanced sequence- assignment patterns (高级序列赋值模式)
- sequences (序列) : Assignment Syntax Forms (赋值语法形式)
- setitem method (setitem 方法) : Intercepting Item Assignments (拦截项赋值)
- statements (语句) : Python’ s Statements (Python 的语句) ; Assignment Syntax Forms (赋值语法形式)
- syntax (语法) : Application to for loops (应用于 for 循环)
- tuples (元组) : Assignment Syntax Forms (赋值语法形式)
- associative arrays (关联数组) : Dictionaries (字典)

- `async function` (`async 函数`) : `Asynchronous Functions: The Short Story` (异步函数: 短篇故事)
- `concurrent tasks` (并发任务)
- `async def` (`async def (异步函数定义)`) : `Running concurrent tasks with "await" and "async def"` (用 `await` 与 `async def` 运行并发任务)
- `async for` (`async 用于`) : `Running concurrent tasks with "async with" and "async for"` (运行并发任务带 `"async 带"` 与 `"async 用于"`)
- `async with` (`async 带`) : `Running concurrent tasks with "async with" and "async for"` (运行并发任务带 `"async 带"` 与 `"async 用于"`)
- `ascompleted` (`ascompleted (完成即取)`) : `Running concurrent tasks with "ascompleted" and "gather"` (用 `ascompleted` 与 `gather` 运行并发任务)
- `await` (`await (等待)`) : `Running concurrent tasks with "await" and "async def"` (用 `await` 与 `async def` 运行并发任务)
- `gather` (`gather (汇合)`) : `Running concurrent tasks with "ascompleted" and "gather"` (用 `ascompleted` 与 `gather` 运行并发任务)
- `I/O (input/output) operations` (`I/O (输入/输出) 操作`) : `Asynchronous Functions: The Short Story` (异步函数: 短篇故事)
- `multitasking` (多任务) : `Async Basics` (异步基础)

- serial tasks (串行任务) : Running serial tasks with normal blocking calls (用普通阻塞调用运行串行任务)
- tasks to avoid (应避免的任务) : How not to use async functions (如何不要使用异步函数)
- async statement (async 语句) : Advanced Function Tools (高级函数工具)
- attribute accessors (属性访问器)
- get method (get 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getattr method (getattr 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getattribute method (getattribute 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- set method (set 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- setattr method (setattr 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- attribute fetches (属性 fetches) : Method Call Syntax (方法 Call 语法) ; Attribute-fetch algorithm (属性获取算法)
- computing value (computing 值) : Computed Attributes (getattr 计算属性)
- getattr method, getattr (getattr 方法, getattr)

- `getattr` method (`getattr` 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- `getattr` and `getattr`-Using `getattr` (`getattr` 与 `getattr` -改用 `getattr`)
- inheritance (继承) : Classes: Under the Hood (类 : 揭开引擎盖)
- loop avoidance (避免循环) : Avoiding loops in attribute interception methods (避免属性拦截方法中的循环)
- properties (properties (属性)) : The Basics (基础)
- attribute interception (属性 interception) : Managed Attributes (管理属性)
- descriptors, `getattr` (描述符, `getattr`)
- access (访问) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- built-in attributes, intercepting (内置- 属性, intercepting) : Intercepting Built-in Operation Attributes (拦截内建运算属性) ; Revisiting Chapter 28' s delegation example (重访第 28 章委托示例)
- computed attributes (`getattr` 计算属性) : Computed Attributes (`getattr` 计算属性)
- descriptors (描述符) : Descriptors (描述符) ; Management Techniques Compared (管理技术对比)

- computed attributes (getattr 计算属性) : Comput ed Attributes (getattr 计算属性)
- method arguments (方法参数) : Descriptor metho d arguments (描述符方法参数)
- properties and (properties (属性) 与) : How Pro perties and Descriptors Relate (property 与描述符如何关联) ; Descriptors (描述符)
- read-only (只读) : Read-only descriptors (只读描述符)
- state information (状态 information) : Using Stat e Information in Descriptors (在描述符中使用状态信息)
- validation and (校验与) : Using Descriptors to Val idate (用描述符做校验)
- getattr method, getattr (getattr 方法, getattr)
- validation (校验) : Using getattr to Validate (用 g etattr 做校验)
- getattribute method, getattr (getattribute 方法, ge tattr)
- validation (校验) : Using getattribute to Validate (用 getattribute 做校验) ; Using getattribute (改用 getattribute)
- to Validate (到 Validate)
- properties (properties (属性)) : Properties (pro perties (属性)) ; Management Techniques Compa

red (管理技术对比)

- decorators (装饰器) : Coding Properties with Decorators (用装饰器编写 property) ; Setter and deleter decorators (setter 与 deleter 装饰器)
- validation and (校验与) : Using Properties to Validate (用 property 做校验) ; Testing code (测试代码)
- validation (校验)
- descriptors and (描述符与) : Using Descriptors to Validate (用描述符做校验)
- properties and (properties (属性) 与) : Using Properties to Validate (用 property 做校验) ; Testing code (测试代码)
- attribute trees (属性 trees) : Attribute Tree Construction (属性树构建)
- attributes (属性) : Imports and Attributes (导入与属性)
- assignment (赋值) : Attribute Assignment and Deletion (属性赋值与删除) ; Python Inheritance Algorithm: The Less Simple Version (Python 继承算法: 不那么简单版)
- assignments (赋值总论) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- changing (更改) : Changing Class Attributes Can Have Side Effects (Changing 类属性 Can Have Side

Effects)

- class statement (类语句) : Example: Class Attributes (示例: 类属性)
- assignment-rule exception (assignment-规则异常) : Example: Class Attributes (示例: 类属性)
- defs (def 定义) : Example: Class Attributes (示例: 类属性)
- visibility (可见性) : Example: Class Attributes (示例: 类属性)
- classes (类) : Instance Versus Class Attributes (实例对比类属性)
- computing (计算) : Computed Attributes (getattr 计算属性) ; Using getattr (改用 getattr)
- conflict resolution (冲突解决) : Attribute Conflict Resolution (属性冲突的解决)
- deleting (删除) : Attribute Assignment and Deletion (属性赋值与删除)
- descriptors, getattr (描述符, getattr)
- functions (函数) : General Function Concepts (通用函数概念)
- getattr method (getattr 方法) : Attribute Reference (属性引用)
- getattr method, getattr (getattr 方法, getattr)

- instance objects (实例对象) : Coding Constructors (编码构造器)
- instances (实例) : Instance Versus Class Attributes (实例对比类属性)
- introspection tools (introspection 工具) : Special Class Attributes (Special 类属性)
- listing (列出) : The dir Function (dir 函数)
- name strings (名字字符串) : The dir Function (dir 函数)
- type names (类型名字) : The dir Function (dir 函数)
- listing, mix-in classes (listing, mix-中的类) : Example: "Mix-in" Attribute Listers (示例: "混入"式属性列出器)
- class trees (类 trees) : Listing attributes per object in class trees (在类树中按对象列出属性)
- dict (dict (属性字典)) : Listing instance attributes with dict (用 dict 列出实例属性)
- dir (dir (属性列表)) : Listing inherited attributes with dir (用 dir 列出继承属性)
- main (main (主模块)) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- metaclasses (元类) : Inheritance: The Finale (继承 : 终章)

- modules (模块) : Creating Modules (创建模块)
- classes (类) : Classes Are Attributes in Modules (类是模块中的属性)
- creating (创建) : How Files Generate Namespaces (文件如何生成命名空间)
- names, qualification (名字, qualification) : Attribute Name Qualification (属性名限定)
- mutable (可变) : Changing Mutable Class Attributes Can Have Side Effects, Too (Changing 可变类属性 Can Have Side Effects, Too)
- name (name (名字)) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- names, namespaces (名字, 命名空间) : Attribute Names: Object Namespaces (属性名: 对象命名空间)
- nested modules (嵌套模块) : Using the basic package (使用基础包)
- private (私有) : class decorators (类装饰器) ; Implementing Private Attributes (实现私有属性) ; Using dict and slots (and other virtuals) (Using dict 与 slots (与 other virtuals))
- properties (properties (属性)) : Properties: Attribute Accessors (属性: 访问器)
- assignment support (赋值支持) : Property basics (property (属性) 基础)

- pseudoprivate (伪私有) : Pseudoprivate Class Attributes (伪私有类属性) ; Why Use Pseudoprivate Attributes? (为何使用伪私有属性?)
- public (公有) : class decorators (类装饰器) ; Generalizing for Public Declarations (泛化出 Public 声明) ; Workaround: Generating operator-overloading descriptors (Workaround: Generating 运算符重载-描述符)
- reference interception (引用 interception) : Attribute Reference (属性引用)
- self (self (绑定实例)) : Why Use Pseudoprivate Attributes? (为何使用伪私有属性?)
- setattr method (setattr 方法) : Attribute Assignment and Deletion (属性赋值与删除)
- slots (slots (插槽)) : Classes: Under the Hood (类: 揭开引擎盖) ; Slots: Attribute Declarations (slots (插槽) : 属性 Declarations)
- slots (slots (槽位)) : Multiple slot lists in superclasses (Multiple slot 列表中的超类)
- user-defined (用户定义) : Function Attributes (函数属性)
- augmented assignments (增强赋值) : Assignment Syntax Forms (赋值语法形式) ; Augmented Assignments (增强赋值) ; Augmented assignment and shared references (增强赋值与共享引用)

- await statement (await 语句) : Advanced Function Tools (高级函数工具)

B (字母 B)

- backslash characters (反斜杠字符) : Test Your Knowledge: Part III Exercises (第三部分练习) ; Part III, Statements and Syntax (第 III 部分, 语句与语法)
- strings (字符串) : Escape Sequences Are Special Characters (转义序列是特殊字符) ; Escape Sequences (Escape 序列)
- f-string literal (f-字符串 literal) : F-string formatting basics (F-字符串格式化基础)
- BaseException exception class (BaseException 异常类) : Built-in Exception Classes (内置异常类)
- bases attribute (bases 属性) : Classes: Under the Hood (类: 揭开引擎盖) ; Special Class Attributes (Special 类属性)
- Namespace Dictionaries: Review (命名空间字典: 回顾) : Multiple Inheritance and the MRO (多重继承与 MRO) ; The Inheritance Bifurcation (继承 Bifurcation) ; Metaclass Inheritance (元类继承)
- basic assignment (基本赋值) : Basic Assignments (基本赋值)
- Beautiful Soup (Beautiful Soup 库) : Internet and Web Scripting (互联网与 Web 脚本)

- BeeWare Toga (BeeWare Toga 跨平台 UI) : GUIs and UIs (GUI 与 UI)
- benchmarking (基准测试) : Benchmarking with Homegrown Tools (使用自制工具进行基准测试)
- iteration (迭代) : Iteration Results (迭代测试结果) ; For more good times: Function calls and map (更多好戏: 函数调用与 map)
- function calls (函数调用) : For more good times: Function calls and map (更多好戏: 函数调用与 map) ; For more (用于 more)
- good times: Function calls and map (good times: 函数调用与 map)
- map function (map 函数) : For more good times: Function calls and map (更多好戏: 函数调用与 map) ; For more (用于 more)
- good times: Function calls and map (good times: 函数调用与 map)
- timeit module (timeit 模块) : Benchmarking with Python' s timeit (用 Python 的 timeit 做基准测试) ; Conclusion: Comparing tools (结论: 比较工具)
- timing module (timing 模块)
- example (示例) : Timer Module: Take 1 (计时模块: 第一版) ; Timer Module: Take 2 (计时模块: 第二版)

- test functions (test 函数) : Timing Runner and Script (计时运行器与脚本)
- binary data (二进制数据) : Unicode and Byte (Unicode 与字节) ; Strings (字符串) ; Introducing Python String Tools (介绍 Python 字符串工具)
- struct module (struct 模块) : The struct Binary-Data Module (struct 二进制数据- 模块)
- binary files (binary 文件) : Text and Binary Files: The Short Story (文本与二进制文件: 速览) ; Text and Binary Files: (文本文件与二进制文件:)
- bytes object (字节串对象) : Text and Binary Modes (Text 与 Binary Modes)
- blank lines (空行) : Common Coding Gotchas (常见编码陷阱)
- block strings (block 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- blocks, statements (blocks, 语句) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- BOM (byte order marker) (BOM (字节顺序标记)) : Character Encodings (字符编码)
- adding (添加) : Making BOMs in Python (在 Python 中制作 BOM)
- text editors (文本编辑器) : Making BOMs in Text Editors (Making BOMs 中的文本编辑器)

- text-mode files (文本模式- 文件) : Using Text and Binary Files (使用文本与二进制文件)
- bool method (bool 方法) : Boolean Tests: bool and len (布尔测试: bool 和 len: bool 与 len)
- Boolean type (布尔类型) : The bool type (bool 类型)
- Boolean values (布尔值) : Boolean Tests: bool and len (布尔测试: bool 和 len: bool 与 len)
- Booleans (布尔) : Booleans and None (布尔与 None) ; Test Your Knowledge: Answers (测验答案)
- Boost.Python (Boost.Python 绑定库) : Component Integration (组件集成)
- bound methods (bound 方法) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Why the Special Methods? (Why Special 方法?)
- callback functions (回调函数) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- bounds (边界) : Bounds Checking (边界检查)
- break statements (break 语句) : break, continue, pass, and the Loop else (break、continue、pass 与循环 else)
- interactive loops (交互式循环) : A Simple Interactive Loop (简单交互循环)

- breaks (中断) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识) ; Part I, Getting Started (第 I 部分: 入门)
- buffering (缓冲)
- files (文件) : Using Files (使用文件)
- output (输出) : Opening Files (打开文件)
- built-in docstrings (内置- 文档字符串) : Built-in docstrings (内置- 文档字符串)
- built-in exception classes (内置异常类) : Built-in Exception Classes (内置异常类)
- ArithmeticError (ArithmeticError 异常) : Built-in Exception Classes (内置异常类)
- BaseException (BaseException 异常) : Built-in Exception Classes (内置异常类)
- Exception (异常) : Built-in Exception Classes (内置异常类)
- LookupError (LookupError 异常) : Built-in Exception Classes (内置异常类)
- OSError (OSError 异常) : Built-in Exception Classes (内置异常类)
- built-in functions (内置- 函数)
- handling by assignment (通过赋值处理) : Hiding built-ins by assignment (Hiding 内置- by assignment)

- super (super (超类代理)) : Augmenting Method
s: The Good Way (增强方法: 正确做法)
- built-in modules, module search path (内置- 模块,
模块 search path) : and, The (与)
- collections (集合容器) : Why Use Built-in Objects?
(为什么使用内置对象?)
- dictionaries (字典) : Why Use Built-in Objects? (为什么使用内置对象?)
- lists (列表) : Why Use Built-in Objects? (为什么使用内置对象?)
- search tables (查找表) : Why Use Built-in Objects?
(为什么使用内置对象?)
- built-in operations (内置- 操作)
- delegating (委托) : Other Ways to Combine Class
es: Composites (组合类的其他方式: 复合) ; Delegati
ng Built- In Operations (委托内置运算)
- operator overloading methods (运算符重载方法)
: Workaround: Coding operator- overloading meth
ods inline (Workaround: 编码运算符重载- 方法 inline
) ; Workaround: Generating operator- overloading
descriptors (Workaround: Generating 运算符重载-
描述符)
- built-in tools (内置- 工具) : It' s Powerful (它很强大)
; The Python Toolset (Python 工具组)

- built-in types (内置- 类型) : Extending Built-in Object Types (扩展内置对象类型)
- class creation (类 creation) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- classes (类) : The Python Object Model (Python 对象模型)
- cyclic data structures (循环数据 structures) : Beware of Cyclic Data Structures (当心循环数据结构)
- extending (扩展 (重复/连接))
- embedding (嵌入) : Extending Types by Embedding (扩展 (重复/连接) 类型 by 嵌入)
- subclassing (子类化) : Extending Types by Subclassing (扩展 (重复/连接) 类型 by Subclassing)
- immutable types (不可变类型) : Immutable Types Can't Be Changed in Place (不可变类型不能就地修改)
- references (引用) : Assignment Creates References, Not Copies (赋值制造引用, 而非拷贝)
- repetition (重复) : Repetition Adds One Level Deep (重复只加深一层)
- built-ins (内置) : The Built-in Scope (内置作用域)
- generators (生成器) : Generation in built-ins and classes (Generation 中的内置- 与类)

- LEGB rule (LEGB 规则) : Redefining built-in names: For better or worse (Redefining 内置- 名字: 用于 better 或 worse)
- byte files (字节文件) : Unicode and Byte Files (Unicode 与字节文件)
- byte strings (字节字符串) : Using Byte Strings (使用字节字符串)
- bytearray object (bytearray 对象) : The bytearray Object (bytearray 对象)
- bytes constructor (字节串 constructor) : Other Ways to Make Bytes (构造 bytes 的其他方式)
- formatting (格式化) : Formatting (格式化)
- methods (方法) : Methods (方法)
- mixing types (mixing 类型) : Mixing String Types (类型混用)
- sequence operations (序列操作) : Sequence Operations (序列操作) ; Formatting (格式化)
- bytearray object (bytearray 对象) : The bytearray Object (bytearray 对象)
- bytecode (字节码) : OK, but What' s the Downside? (好, 但缺点是什么?) ; Bytecode compilation (字节码编译) ; Development implications (对开发的影响) ; Test Your Knowledge: Answers (测验答案) ; Step 2: Compile It (Maybe) (第二步: 编译 (或许))

- executing (执行) : Step 3: Run It (第三步: 运行)
- files (文件) : Bytecode compilation (字节码编译)
- hash-based files (hash-based 文件) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- import optimization (import 优化) : Bytecode compilation (字节码编译)
- interactive prompt code (交互式提示符 code) : Bytecode compilation (字节码编译)
- magic number (魔数) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- PyPy (PyPy 实现) : OK, but What' s the Downside? (好, 但缺点是什么?)
- Python versions (Python 版本) : Bytecode compilation (字节码编译)
- startup-speed optimization (startup-速度优化) : Bytecode compilation (字节码编译)
- timestamp-based files (timestamp-based 文件) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- bytes object (字节串对象) : Unicode and Byte (Unicode 与字节) ; Strings (字符串) ; The bytes Object (bytes 对象)
- binary-mode files (二进制模式- 文件) : Text and Binary Modes (Text 与 Binary Modes)

C (字母 C)

- call expressions, methods (调用表达式: call, 方法)
: Method Call Syntax (方法 Call 语法)
- operator overloading (运算符重载) : Call Expressions: call (调用表达式: call: call) ; Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- call method (call 方法) : Call Expressions: call (调用表达式: call: call) ; Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- call proxies (调用代理) : Managing Calls and Instances (管理调用与实例)
- callback functions (回调函数)
- bound methods (bound 方法) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- lambda functions (lambda 函数) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- callbacks (回调) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- bound methods (bound 方法) : Bound Methods in Action (绑定方法实战)

- calls (调用)
- def statements (def 语句) : Calls (调用)
- functions (函数) : Calls (调用)
- CFFI (CFFI 外部函数接口) : Component Integration (组件集成)
- chaining exceptions (chaining 异常) : Exception Chaining: raise from (异常链: raise from)
- Chaquopy (Chaquopy (Android 上的 Python)) : Component Integration (组件集成)
- character code conversion (字符码转换) : Character-code conversions (字符码转换)
- Cinder (Cinder (Meta 的 CPython 分支)) : Python Implementation Alternatives (Python 实现替代方案)
- circular imports (see recursion) (circular 导入 (see recursion))
- circular references, garbage collection (circular 引用, 垃圾回收) : Objects Are Garbage-Collected (对象会被垃圾回收)
- class attribute (class 属性) : Classes: Under the Hood (类: 揭开引擎盖) ; Special Class Attributes (Special 类属性) ; How the MRO Works (MRO 如何工作) ; Classes Are Types Are Classes (类 Are 类型 Are 类) ; A "magic" proxy ("魔法" 代理) ; Metaclass Versus Superclass (元类 vs 超类)

- class decorators (类装饰器) : Decorators and Metaclasses (装饰器与元类) ; A First Look at Class Decorators and Metaclasses (类装饰器与元类初探) ; What's a Decorator? (什么是装饰器?) ; Class Decorator Basics (类装饰器基础) ; Adding methods to classes (向类添加方法)
- arguments (参数) : Decorator arguments (装饰器参数)
- built-in types (内置- 类型) : Applying class decorators to built-in types (把类装饰器用于内置类型)
- class managers (类 managers) : Managing Functions and Classes (管理函数与类)
- implementation (实现)
- instances, multiple (实例, multiple) : Supporting multiple instances (Supporting multiple 实例) ; Class Pitfall: Retaining Multiple Instances (类陷阱: 保留多个实例)
- interface proxies (接口 proxies) : Managing Calls and Instances (管理调用与实例)
- metaclasses comparison (元类 comparison) : Metaclasses Versus Class Decorators: Round 1 (元类 vs 类装饰器: 第一回合)
- object interfaces (对象 interfaces) : Tracing Object Interfaces (跟踪对象接口)
- private attributes (private 属性) : Implementing P

private Attributes (实现私有属性) ; Using dict and slots (and other virtuals) (Using dict 与 slots (与 other virtuals))

- public attributes (public 属性) : Generalizing for Public Declarations (泛化出 Public 声明) ; Workaround: Generating operator-overloading descriptors (Workaround: Generating 运算符重载- 描述符)
- singleton classes (单例类) : Singleton Classes (单例类) ; Applying class decorators to built-in types (把类装饰器用于内置类型)
- state, enclosing scopes (状态, enclosing 作用域)
- attributes (属性) : Example: Class Attributes (示例: 类属性)
- assignment-rule exception (assignment-规则异常) : Example: Class Attributes (示例: 类属性)
- defs (def 定义) : Example: Class Attributes (示例: 类属性)
- visibility (可见性) : Example: Class Attributes (示例: 类属性)
- syntax (语法) : General Syntax and Usage (一般语法与用法)
- class trees (类 trees)
- attribute listings (属性 listings) : Listing attributes per object in class trees (在类树中按对象列出属性)

- inheritance (继承) : Python Inheritance Algorithm : The Simple Version (Python 继承算法: 简单版)
- OOP (OOP (面向对象编程)) : Coding Class Trees (编写类树) ; Part VI, Classes and OOP (第 VI 部分, 类与 OOP (面向对象编程))
- class-based exceptions (类-based 异常) : Catching Too Little: Use Class-Based Categories (捕获太少 : 使用基于类的分类)
- classes (类) : OOP: The Big Picture (OOP: 大图景)
- attributes (属性) : Instance Versus Class Attributes (实例对比类属性)
- changing (更改) : Changing Class Attributes Can Have Side Effects (Changing 类属性 Can Have Side Effects)
- descriptors (描述符) : A First Example (getattr 第一个示例)
- introspection tools (introspection 工具) : Special Class Attributes (Special 类属性)
- mutable (可变) : Changing Mutable Class Attributes Can Have Side Effects, Too (Changing 可变类属性 Can Have Side Effects, Too)
- pseudoprivate (伪私有) : Pseudoprivate Class Attributes (伪私有类属性) ; Why Use Pseudoprivate Attributes? (为何使用伪私有属性?)

- slots (slots (插槽)) : Classes: Under the Hood (类: 揭开引擎盖)
- bases attribute (bases 属性) : Classes: Under the Hood (类: 揭开引擎盖) ; Special Class Attributes (Special 类属性)
- Namespace Dictionaries: Review (命名空间字典: 回顾) : Multiple Inheritance and the MRO (多重继承与 MRO) ; The Inheritance Bifurcation (继承 Bifurcation) ; Metaclass Inheritance (元类继承)
- built-in operation override (内置- 操作 override) : Classes Can Intercept Python Operators (类可以拦截 Python 运算符)
- built-in types (内置- 类型) : The Python Object Model (Python 对象模型)
- class attribute (class 属性) : Classes: Under the Hood (类: 揭开引擎盖) ; Special Class Attributes (Special 类属性)
- How the MRO Works (MRO 如何工作) : Classes Are Types Are Classes (类 Are 类型 Are 类) ; A “magic” proxy (“魔法” 代理)
- Metaclass Versus Superclass (元类 vs 超类)
- composites (复合) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- composition (组合) : Why Use Classes? (为什么使用类?)

- conflation (混同) : The Roles of init.py Files (init.py 文件的作用)
- delegation-based (委托-based) : Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
- descriptors (描述符) : Descriptors (描述符)
- versus dictionaries (对比字典) : Records Revisited: Classes Versus Dictionaries (回顾记录: 类对比字典)
- docstrings (文档字符串) : User-defined docstrings (用户定义的文档字符串) ; Documentation Strings Revisited (重温文档字符串)
- exception classes (异常类) : Exception Objects (异常对象)
- extending (扩展 (重复/连接)) : Why Use Classes? (为什么使用类?)
- generators (生成器) : Generation in built-ins and classes (Generation 中的内置- 与类)
- inheritance (继承) : OOP: The Big Picture (OOP: 大图景) ; Why Use Classes? (为什么使用类?) ; Inheritance (继承) ; Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- attribute trees (属性 trees) : Attribute Tree Construction (属性树构建)
- inherited methods (inherited 方法) : Specializing Inherited Methods (特化继承方法)

- metaclass subclasses and (元类子类与) : The Inheritance Bifurcation (继承 Bifurcation)
- MRO (method resolution order) (MRO (方法 resolution order)) : Universal deployment (通用部署)
- multiple inheritance (多重继承) : Multiple Inheritance and the MRO (多重继承与 MRO) ; Example: Mapping Attributes to Inheritance Sources (示例: 将属性映射到继承来源)
- privacy (隐私) : Inheritance Fine Print (继承细则)
- subverting (颠覆/绕过) : Augmenting Methods: The Good Way (增强方法: 正确做法)
- instances (实例) : Why Use Classes? (为什么使用类?)
- attributes (属性) : State with class-instance attributes (状态带类-实例属性)
- creating (创建) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- metaclasses (see metaclasses) (元类 (see 元类))
- method objects (方法对象) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Bound Methods in Action (绑定方法实战)
- methods (方法) : Step 2: Adding Behavior Methods (第二步: 添加行为方法) ; Static and Class Methods (静态方法与类方法)

- Using Static and Class Methods (Using 静态方法与类方法)
- adding (添加) : Adding methods to classes (向类添加方法)
- double underscores (双下划线) : Classes Can Intercept Python Operators (类可以拦截 Python 运算符)
- encapsulation (封装) : Coding Methods (编码方法)
- instance counting (实例 counting) : Counting Instances with Class Methods (Counting 实例带类方法) ; Counting instances per class with class methods (Counting 实例 per 类带类方法)
- metaclass comparison (元类 comparison) : Metaclass Methods Versus Class Methods (元类方法 vs 类方法)
- method calls (方法调用) : Methods (方法) ; Other Method-Call Possibilities (其他方法调用可能)
- module attributes (模块属性) : Classes Are Attributes in Modules (类是模块中的属性)
- mro attribute (mro 属性) : How the MRO Works (MRO 如何工作) ; Example: Mapping Attributes to Inheritance Sources (示例: 将属性映射到继承来源) ; The Inheritance Bifurcation (继承 Bifurcation) ; Attribute-fetch algorithm (属性获取算法)
- Metaclass Versus Superclass (元类 vs 超类)

- namespaces (命名空间)
- hierarchies (层次结构) : Why Use Classes? (为什么使用类?)
- inheritance (继承) : Example: Class Attributes (示例: 类属性)
- nested (嵌套)
- descriptors (描述符) : A First Example (getattr 第一个示例)
- scope (作用域) : Nested Classes: The LEGB Scope s Rule Revisited (嵌套类: 重温 LEGB 作用域规则)
- object factories (对象 factories) : Classes Are Objects: Generic Object Factories (类也是对象: 通用对象工厂) ; Why Factories? (为什么需要工厂?)
- objects (对象) : Classes Generate Multiple Instance Objects (类生成多个实例对象)
- default behavior (默认行为) : Class Objects Provide Default Behavior (类对象提供默认行为)
- OOP (OOP (面向对象编程)) : Attribute Inheritance Search (属性继承搜索) ; Step 1: Making Instances (第一步: 创建实例)
- constructors (构造器) : Coding Constructors (编码构造器)
- inheritance (继承) : Classes Are Customized by Inheritance (类通过继承进行定制) ; A Second Example (第二个示例)

- intercepting operators (intercepting 运算符) : Classes Can Intercept Python Operators (类可以拦截 Python 运算符) ; Other operator-overloading methods (Other 运算符重载- 方法)
- methods (方法) : Step 2: Adding Behavior Methods (第二步: 添加行为方法) ; Coding Methods (编码方法)
- methods, augmenting (方法, augmenting) : Augmenting Methods: The Bad Way (增强方法: 糟糕做法) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- namespaces (命名空间) : Classes and Instances (类与实例)
- operator overloading (运算符重载) : Step 3: Operator Overloading (第三步: 运算符重载) ; Providing Print Displays (提供打印显示)
- polymorphism (多态) : Polymorphism in Action (多态中的 Action)
- subclassing (子类化) : Coding Subclasses (编码子类) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- superclasses (超类) : Attribute Inheritance Search (属性继承搜索) ; Classes Are Customized by Inheritance (类通过继承进行定制)
- testing (测试) : Testing as You Go (测试 as You G

o)

- persistence (持久化) : Stream Processors Revisited (再访流处理器)
- polymorphism (多态) : Polymorphism and classes (多态与类)
- as program units (as 程序 units) : Why Use Classes? (为什么使用类?)
- redundancy (冗余) : OOP: The Big Picture (OOP : 大图景) ; Coding Methods (编码方法)
- scopes (作用域) : Scopes in Methods and Classes (作用域中的方法与类)
- state retention (状态 retention) : Classes: Changeable, Per-Call, OOP (类: 可修改、每调用独立、OOP)
- storage (存储) : Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
- subclasses (子类) : Classes Are Customized by Inheritance (类通过继承进行定制)
- superclass constructors (超类构造器) : Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
- types and (类型与) : The Python Object Model (Python 对象模型)
- user-defined (用户定义) : Classes Are Types Are Classes (类 Are 类型 Are 类)

- versus modules (对比模块) : Classes Versus Modules (类与模块)
- classmethod function (classmethod 函数) : Using Static and Class Methods (Using 静态方法与类方法)
- closure functions (闭包函数) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- closures (闭包) : Closures and Factory Functions (闭包与工厂函数)
- code folders (代码目录)
- dedicated (专用) : Where to Run: Code Folders (Where 到 Run: 代码目录)
- directory structure (目录结构) : Where to Run: Code Folders (Where 到 Run: 代码目录)
- per-chapter subfolders (按章分子文件夹) : Where to Run: Code Folders (Where 到 Run: 代码目录)
- code obfuscation (代码混淆) : list comprehensions (列表推导式)
- Unicode (Unicode 统一码) : Character Representations (字符表示)
- escapes (转义) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- value (值) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)

- code reuse (code 复用) : Test Your Knowledge: Part VI Exercises (第六部分练习)
- exceptions (异常) : Exception Objects (异常对象)
- functions (函数) : Function Basics (函数基础)
- modules (模块) : Why Use Modules? (为什么使用模块?)
- OOP (OOP (面向对象编程)) : OOP Is About Code Reuse (OOP 的本质是代码复用) ; Programming by customization (通过定制来编程)
- code strings, module imports (code 字符串, 模块导入) : Running Code Strings (运行 Code 字符串)
- code wrapping, "Overwrapping-itis" (代码包装, "过度包装症")
- coding gotchas (编码 gotchas) : Common Coding Gotchas (常见编码陷阱)
- collections (集合容器) : Why Use Built-in Objects? (为什么使用内置对象?)
- argument matching (参数匹配) : Argument Matching Overview (参数匹配概览)
- collector modules (collector 模块) : Part V, Modules and Packages (第 V 部分, 模块与包)
- colons (冒号)
- compound statement headers (compound 语句 headers) : Common Coding Gotchas (常见编码陷阱)

- statements (语句) : What Python Adds (Python 增加了什么)
- columns in code (columns 中的 code) : Why Indentation Syntax? (为什么采用缩进语法?)
- command line (命令行) : Test Your Knowledge: Answers (测验答案)
- launchers (启动器) : Command-line launchers (命令行启动器)
- program file running (程序文件运行) : Running Files with Command Lines (用命令行运行文件)
- program files (程序文件) : Program Files (程序文件)
- shell programs (shell 程序) : Command-Line Usage Variations (命令行用法变体)
- tools (工具) : Systems Programming (系统编程)
- commas (逗号)
- expressions (表达式) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- tuples (元组) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- comments (注释) : Running Code Interactively (交互式运行代码) ; A First Script (第一个脚本)
- interactive coding (交互式编程) : What Not to Type: Prompts and Comments (不要键入的内容: 提示符)

与注释)

- comparison operators (比较运算符) : Comparisons: lt, gt, and Others (比较运算: lt、gt 等: lt, gt, 与 Others)
- comparisons (比较运算: lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值) ; Dictionary comparisons (字典比较)
- core object types (core 对象类型) : Comparisons, Equality, and Truth (比较、相等与真值)
- dictionaries (字典) : Dictionary comparisons (字典比较)
- lists (列表) : Basic List Operations (列表基本操作)
- mixed-type (mixed-类型) : Comparisons, Equality, and Truth (比较、相等与真值)
- recursive (递归的) : Comparisons, Equality, and Truth (比较、相等与真值)
- relative magnitude (相对大小) : Comparisons, Equality, and Truth (比较、相等与真值)
- compiled extensions (compiled 扩展) : OK, but What's the Downside? (好, 但缺点是什么?)
- compilers (编译器)
- AOT (ahead-of-time) (AOT (提前编译)) : Ahead-of-Time Compilers for Speed (为速度而生的提前编译器)

- Python-to-Wasm (Python-到-Wasm) : WebAssembly for Browsers (浏览器中的 WebAssembly)
- completion certificate (结业证书) : Encore: Print Your Own Completion Certificate! (加餐: 打印自己的完成证书)
- complex numbers (复数) : Numbers (数字) ; Test Your Knowledge: Answers (测验答案) ; Core Types Review and Summary (核心类型回顾与总结)
- component integration (组件集成) : Why Do People Use Python? (为什么人们使用 Python?) ; Component Integration (组件集成)
- composites (复合)
- delegation (委托) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- nesting (嵌套) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- composition (组合)
- abstract superclasses (抽象超类) : Stream Processors Revisited (再访流处理器)
- aggregation (聚合) : OOP and Composition: "Has-a" Relationships (OOP 与组合: "有一个" 关系)
- has-a relationships ("有一个" 关系) : OOP and Composition: "Has-a" Relationships (OOP 与组合: "有一个" 关系) ; Stream Processors Revisited (再访流处理器)

- OOP (OOP (面向对象编程)) : OOP and Composition: "Has-a" Relationships (OOP 与组合: "有一个" 关系) ; Stream Processors Revisited (再访流处理器)
- stream processors (流处理器) : Stream Processors Revisited (再访流处理器)
- compound objects (compound 对象) : Comparisons, Equality, and Truth (比较、相等与真值)
- compound statements (compound 语句) : Missing Keys: if Tests (Missing 键: if Tests) ; What Python Adds (Python 增加了什么)
- if statements (if 语句) : if Statements (if 语句)
- multiple-choice selection (多路选择) : Multiple-Choice Selections (多选逻辑) ; Handling larger actions (处理更大型动作)
- match statements (match 语句) : match Statements (match 语句)
- attribute patterns (属性 patterns) : Advanced match Usage (match 高级用法)
- if statements (if 语句) : Match versus if live (Match 对比 if live)
- instance patterns (实例 patterns) : Advanced match Usage (match 高级用法)
- literal patterns (字面量模式) : Advanced match Usage (match 高级用法)

- mapping patterns (映射 patterns) : Advanced match Usage (match 高级用法)
- nested patterns (嵌套 patterns) : Advanced match Usage (match 高级用法)
- nesting (嵌套) : Basic match Usage (match 基本用法)
- parentheses (圆括号) : Advanced match Usage (match 高级用法)
- sequence patterns (序列 patterns) : Advanced match Usage (match 高级用法)
- syntax (语法) : Python Syntax Revisited (Python 语法回顾)
- blocks (代码块) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- delimiters (分隔符) : Statement Delimiters: Lines and Continuations (语句 Delimiters: Lines 与 Continuations)
- indentation (缩进) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- special syntax (special 语法) : Special Syntax Cases in Action (Special 语法 Cases 中的 Action)
- comprehensions (推导式) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)

- converting tuples (converting 元组) : Conversions , methods, and immutability (转换、方法与不可变性)
- dictionaries (字典) : Dictionary Comprehensions (字典推导式)
- dictionary (字典) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器) ; Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- for loops (for 循环) : Changing Lists: range and Comprehensions (Changing 列表: range 与推导式)
- generators and (生成器与) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- list comprehensions (列表推导式) : Comprehensions (推导式) ; List Comprehension Basics (列表推导式基础)
- Comprehensions: The Final Act (推导式: 最后一幕)
- files (文件) : List Comprehensions and Files (列表推导式与文件)
- if clause (if 子句) : Filter clauses: if (过滤子句: if)
- nested for loops (嵌套 for 循环) : Nested loops: for (嵌套循环: 用于)

- sequence operations (序列操作) : Conversions, methods, and immutability (转换、方法与不可变性)
- syntax (语法) : Comprehensions (推导式)
- lists (列表) : List comprehensions and maps (列表推导式与 maps)
- loop variables (loop 变量) : Preview: Other Python scopes (预览: Python 的其他作用域)
- set (集合) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- syntax (语法) : Comprehensions and (推导式与)
- type calls (类型调用) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- variables, scopes (变量, 作用域)
- list literals (列表 literals) : List-literal unpacking (列表-字面量解包)
- strings (字符串) : Single and Double Quotes Are the Same (Single 与 Double Quotes Are Same) ; Basic Operations (基本操作)
- concept hierarchy (A.7.1 概念: 把 Python 打包成普通程序 hierarchy) : The Python Conceptual Hierarchy Revisited (重访 Python 概念层级)
- expressions (表达式) : The Python Conceptual Hierarchy Revisited (重访 Python 概念层级)

- statements (语句) : The Python Conceptual Hierarchy Revisited (重访 Python 概念层级)
- constraints (约束)
- assert statement (assert 语句) : Example: Trapping Constraints (but Not Errors!) (示例: 捕获约束条件——但不是错误!)
- MicroPython (MicroPython 微控制器版) : Python Implementation Alternatives (Python 实现替代方案)
- constructor convolution (构造器的纠结) : Constructors and Expressions: init and sub (构造器与表达式 : init 和 sub: init 与 sub)
- constructors (构造器)
- classes (类) : Coding Constructors (编码构造器)
- customizing (定制) : Step 5: Customizing Constructors, Too (第五步: 同样定制构造函数)
- init method (init 方法) : Operator Overloading (运算符重载)
- operator overload (运算符 overload) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)
- superclass methods (超类方法) : Specializing Inherited Methods (特化继承方法)
- superclasses (超类) : Step 5: Customizing Constructors, Too (第五步: 同样定制构造函数) ; Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)

- container object length (container 对象 length) : Test Your Knowledge: Answers (测验答案)
- contains method (contains 方法) : Membership: contains, iter, and getitem (成员测试: contains、iter 与 getitem: contains, iter, 与 getitem)
- context managers (上下文管理器) : File Context Managers (文件上下文管理器)
- with statement (带语句)
- context-management protocol (上下文管理协议) : The Context-Management Protocol (上下文管理协议)
- multiple context managers (多个上下文管理器) : Multiple Context Managers (多个上下文管理器)
- continuation lines in interactive coding (continuation lines 中的交互式编程) : What Not to Type: Prompts and Comments (不要键入的内容: 提示符与注释)
- continue statements (continue 语句) : break, continue, pass, and the Loop else (break、continue、pass 与循环 else)
- control language (控制语言) : Is Python a "Scripting Language" ? (Python 是"脚本语言"吗?)
- control-nesting, exception handlers (control-嵌套异常处理器) : Example: Control-Flow Nesting (示例: 控制流嵌套)

- conversions (转换)
- text files (text 文件) : Storing Objects with Conversions (用转换来存储对象)
- tuples (元组) : Conversions, methods, and immutability (转换、方法与不可变性)
- cooperative method dispatch (cooperative 方法 dispatch) : Call-chain anchors (调用链锚点)
- copies (拷贝)
- build-in types (build-中的类型) : Assignment Creates References, Not Copies (赋值制造引用, 而非拷贝)
- of objects (的对象) : References Versus Copies (引用与拷贝)
- copy method (拷贝方法) : Missing Keys: if Tests (Missing 键: if Tests)
- core object types (core 对象类型) : Python' s Core Object Types (Python 的核心对象类型)
- coroutines (协程) : Advanced Function Tools (高级函数工具)
- counter loops (counter 循环)
- variables (变量) : Function Design Concepts (函数设计概念)
- coupling modules (耦合模块) : Module Design Concepts (模块设计概念)

- CPython (CPython 实现) : Python Implementation Alternatives (Python 实现替代方案) ; Python (Python 语言)
- WebAssembly/Emscripten platform (WebAssembly/Emscripten 平台) : WebAssembly for Browsers (浏览器中的 WebAssembly)
- cross-file changes (cross-文件修改) : Program Design: Minimize Cross-File Changes (程序设计: 尽量减少跨文件修改)
- CSV module, object storage (CSV 模块与对象存储) : Storing Objects with Other Tools (用其它工具存储对象)
- current working directory (CWD) (当前工作目录 (CWD)) : Opening Files (打开文件)
- customization by inheritance (通过...定制继承) : Step 4: Customizing Behavior by Subclassing (第四步 : 通过子类化定制行为)
- CWD (current working directory) (CWD (当前工作目录)) : Opening Files (打开文件)
- cycles (循环) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识) ; Part I, Getting Started (第 I 部分: 入门)
- recursion (递归) : Cycles, paths, and stack limits (Cycles, paths, 与栈限制)
- cyclic data (循环数据) : Part I, Getting Started (第

I 部分：入门)

- structures (结构) : Beware of Cyclic Data Structures (当心循环数据结构)
- Cython (Cython 静态编译扩展) : OK, but What' s the Downside? (好, 但缺点是什么?) ; Component Integration (组件集成) ; Numeric and Scientific Programming (Numeric 与科学计算编程) ; Python Implementation Alternatives (Python 实现替代方案) ; Test Your Knowledge: Answers (测验答案)

D (字母 D)

- data descriptors (data 描述符) : The Basics (基础)
- inheritance (继承) : The assignment addendum (赋值附录)
- data hiding (数据隐藏) : The Roles of init.py Files (init.py 文件的作用) ; Data Hiding in Modules (模块中的数据隐藏)
- Pseudoprivate Class Attributes (伪私有类属性)
- all variable (all 变量) : Minimizing from Damage: X and all (最小化 from 的破坏: X 与 all)
- dir function (dir 函数) : Managing Attribute Access: getattr and dir (管理属性访问: getattr 与 dir)
- getattr function (getattr 函数) : Managing Attribute Access: getattr and dir (管理属性访问: getattr 与

dir)

- X prefix (X 前缀) : Minimizing from Damage: X and all (最小化 from 的破坏: X 与 all)
- databases (数据库) : Database Access (数据库访问)
- API (API (应用程序接口)) : Database Access (数据库访问)
- books database example (books 数据库示例) : Intermission: Books Database (Intermission: Books 数据库) ; Mapping values to keys (映射值到键)
- object storage (对象 storage) : Step 7 (Final): Storing Objects in a Database (第七步 (最终) : 把对象存入数据库)
- pickle module (pickle 模块) : Pickles and Shelves (Pickles 与 Shelves)
- shelf object updates (shelf 对象 updates) : Updating Objects on a Shelf (Updating 对象 on Shelf)
- shelve database (shelve 数据库) : Storing Objects on a shelve Database (Storing 对象 on shelve 数据库) ; Exploring Shelves Interactively (交互式探索 shelf)
- shelve module (shelve 模块) : Pickles and Shelves (Pickles 与 Shelves) ; The shelve module (shelve 模块)

- debugging (调试) : Development Tools for Larger Projects (更大型项目的开发工具) ; Part I, Getting Started (第 I 部分: 入门)
- exceptions and (异常与) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- try statement (try 语句) : Debugging with Outer try Statements (调试带 Outer try 语句)
- decimals (十进制数) : Numbers (数字) ; Test Your Knowledge: Answers (测验答案) ; Types (类型)
- functions (函数) : Function Annotations and Decorations (函数注解与装饰) ; Function decorators (函数装饰器)
- alternative: Preview (替代方案: 预览)
- decorator timers (装饰器计时器) : More Module Mods (更多的模块改进)
- decorators (装饰器)
- arguments (参数) : Decorator Arguments (装饰器参数) ; Adding Decorator Arguments (添加装饰器参数)
- callables (可调用对象) : Decorator Arguments (装饰器参数)
- function annotations (函数注解) : Decorator Arguments (装饰器参数)

- call tracers (调用追踪器) : Tracing Function Calls (跟踪函数调用)
- callables returning callables (可调用对象 returning 可调用对象)
- class decorators (类装饰器) : Decorators and Metaclasses (装饰器与元类) ; A First Look at Class Decorators and Metaclasses (类装饰器与元类初探) ; What's a Decorator? (什么是装饰器?) ; Class Decorator Basics (类装饰器基础) ; Adding methods to classes (向类添加方法)
- arguments (参数) : Decorator arguments (装饰器参数)
- built-in types (内置- 类型) : Applying class decorators to built-in types (把类装饰器用于内置类型)
- class managers (类 managers) : Managing Functions and Classes (管理函数与类)
- implementation (实现)
- instances, multiple (实例, multiple) : Supporting multiple instances (Supporting multiple 实例) ; Class Pitfall: Retaining Multiple Instances (类陷阱: 保留多个实例)
- interface proxies (接口 proxies) : Managing Calls and Instances (管理调用与实例)
- metaclasses comparison (元类 comparison) : Metaclasses Versus Class Decorators: Round 1 (元类 vs

类装饰器：第一回合)

- object interfaces (对象 interfaces) : Tracing Object Interfaces (跟踪对象接口)
- private attributes (private 属性) : Implementing Private Attributes (实现私有属性) ; Using dict and slots (and other virtuals) (Using dict 与 slots (与 other virtuals))
- public attributes (public 属性) : Generalizing for Public Declarations (泛化出 Public 声明) ; Workaround: Generating operator-overloading descriptors (Workaround: Generating 运算符重载- 描述符)
- singleton classes (单例类) : Singleton Classes (单例类) ; Applying class decorators to built-in types (把类装饰器用于内置类型)
- state, enclosing scopes (状态, enclosing 作用域)
- class management (类 management) : Decorators Manage Functions and Classes, Too (装饰器也能管理函数和类)
- class methods (类方法) : Automatically decorating class methods (自动装饰类方法)
- function decorators (函数装饰器) : Function Decorator Basics (函数装饰器基础)
- What's a Decorator? (什么是装饰器?)
- argument validation (argument 校验) : Example: Validating Function Arguments (案例二: 验证函数参

- 数) ; Decorator Arguments Versus Function Annotations (装饰器参数对比函数注解)
- call proxies (调用代理) : Managing Calls and Instances (管理调用与实例)
 - function call timing (函数 call timing) : Timing Function Calls (为函数调用计时)
 - function managers (函数 managers) : Managing Functions and Classes (管理函数与类)
 - metafunctions (元函数)
 - runtime declarations (运行时声明)
 - syntax (语法)
 - usage (用法) : Function Decorator Basics (函数装饰器基础)
 - user-defined (用户定义) : A First Look at User-Defined Function Decorators (用户定义函数装饰器初探)
 - function management (函数 management) : Decorators Manage Functions and Classes, Too (装饰器也能管理函数和类)
 - function validation (函数校验)
 - keywords (关键字) : Generalizing for Keywords and Defaults (泛化: 支持关键字与默认值)
 - positional arguments (位置式参数) : A Basic Range-Testing Decorator for Positional Arguments (基础版: 仅支持位置参数)

- functions (函数) : General Function Concepts (通用函数概念)
- implementation (实现)
- versus macros (对比 macros) : Why Decorators? (为什么使用装饰器?)
- methods (方法) : Supporting method decoration (支持方法装饰) ; Class Pitfall: Decorating Methods (类陷阱: 装饰方法)
- descriptors (描述符) : Using descriptors to decorate methods (Using 描述符到 decorate 方法)
- nested functions (嵌套函数) : Using nested functions to decorate methods (Using 嵌套函数到 decorate 方法)
- nesting (嵌套) : Decorator Nesting (装饰器嵌套) ; Decorator nesting (装饰器嵌套)
- properties (properties (属性)) : Coding Properties with Decorators (用装饰器编写 property)
- deleters (删除器) : Setter and deleter decorators (setter 与 deleter 装饰器)
- setters (设置器) : Setter and deleter decorators (setter 与 deleter 装饰器)
- range-testing (range-测试) : A Basic Range-Testing Decorator for Positional Arguments (基础版: 仅支持位置参数) ; Decorator Arguments Versus Function Annotations (装饰器参数对比函数注解)

- state retention (状态 retention)
- class-instance attributes (类-实例属性) : State with class-instance attributes (状态带类-实例属性)
- enclosing-scope nonlocals (外层作用域的 nonlocal) : State with enclosing-scope nonlocals (状态带 enclosing-scope nonlocals)
- function attributes (函数属性) : State with function attributes (状态带函数属性)
- global variables and (全局变量与) : State with global variables (以全局变量保存状态)
- syntax (语法) : Why Decorators? (为什么使用装饰器?)
- timer decorator (计时装饰器) : Adding Decorator Arguments (添加装饰器参数)
- wrapper function objects (wrapper 函数对象) : Decorator Nesting (装饰器嵌套)
- def statement (def 语句) : Basic Function Tools (基本函数工具) ; def Statements (def 语句)
- function calls (函数调用) : Calls (调用)
- name assignments (名字赋值总论) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- nonlocal statement and (nonlocal 语句与) : The nonlocal Statement (nonlocal 语句) ; nonlocal Boundary Cases (nonlocal 边界情形)

- runtime execution (运行时执行) : def Executes at Runtime (def 在运行时执行)
- default argument values, scopes (default argument 值, 作用域) : Scopes and Argument Defaults (作用域与参数默认值) ; Loops Require Defaults, Not Scopes (循环需要默认值, 而不是作用域)
- definition, functions (定义, 函数) : Definition (定义)
- del method (del 方法) : Object Destruction: del (对象销毁: del: del) ; Destructor Usage Notes (Destructor 用法 Notes)
- delegation (委托) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- versus inheritance (对比继承) : Inheritance versus delegation (继承对比委托)
- like-a relationships (“像一个” 关系) : OOP and Delegation: “Like-a” Relationships (OOP 与委托: “像一个” 关系)
- OOP (OOP (面向对象编程)) : OOP and Delegation: “Like-a” Relationships (OOP 与委托: “像一个” 关系)
- delimiter string, “Changing” (delimiter 字符串, “Changing”) : Strings (字符串)
- arguments (参数) : Descriptor method arguments (描述符方法参数)

- attributes, computed (属性, computed) : Computed Attributes (getattr 计算属性)
- class attributes (类属性) : A First Example (getattr 第一个示例)
- classes (类) : Descriptors (描述符)
- data descriptors (data 描述符) : The Basics (基础)
- inheritance (继承) : The descriptors deviation (描述符偏误)
- decorating methods (decorating 方法) : Using descriptors to decorate methods (Using 描述符到 decorate 方法)
- files, os module (文件, os 模块) : Other File Tools (其它文件工具)
- hiding (隐藏) : The Basics (基础)
- inheritance (继承) : A First Example (getattr 第一个示例)
- inheritance and (继承与) : The Basics (基础)
- nested classes (嵌套类) : A First Example (getattr 第一个示例)
- operator overloading (运算符重载) : Workaround: Generating operator-overloading descriptors (Workaround: Generating 运算符重载- 描述符)
- properties and (properties (属性) 与) : How Pro

- properties and Descriptors Relate (property 与描述符如何关联) ; Descriptors (描述符)
- read-only (只读) : Read-only descriptors (只读描述符)
 - set method (set 方法) : The Basics (基础)
 - slots (slots (插槽)) : Descriptors and slots and more (描述符、slots 及更多)
 - state (状态) : Using State Information in Descriptors (在描述符中使用状态信息)
 - design (设计)
 - cross-file changes (cross-文件修改) : Program Design: Minimize Cross-File Changes (程序设计: 尽量减少跨文件修改)
 - functions (函数) : Function Basics (函数基础)
 - global variable minimization (全局 variable minimization) : Program Design: Minimize Global Variables (程序设计: 尽量减少全局变量)
 - modules (模块) : Module Design Concepts (模块设计概念)
 - patterns, factory functions (patterns, 工厂函数) : Closures and Factory Functions (闭包与工厂函数)
 - destructors (析构器) : Object Destruction: del (对象销毁: del: del)
 - developer productivity (开发者生产力) : Why Do P

- people Use Python? (为什么人们使用 Python?) ; Developer Productivity (开发者生产力)
- development implications (对开发的影响) : Development implications (对开发的影响)
 - development tools (development 工具) : Development Tools for Larger Projects (更大型项目的开发工具)
 - debuggers (调试器) : Development Tools for Larger Projects (更大型项目的开发工具)
 - documentation tools (文档工具) : Development Tools for Larger Projects (更大型项目的开发工具)
 - error-checking tools (error-checking 工具) : Development Tools for Larger Projects (更大型项目的开发工具)
 - IDEs (集成开发环境) (IDE (集成开发环境) (IDE)) : Development Tools for Larger Projects (更大型项目的开发工具)
 - installation management (A.3.1 安装: 预装迷局与 python.org 安装器 management) : Development Tools for Larger Projects (更大型项目的开发工具)
 - optimization (优化) : Development Tools for Larger Projects (更大型项目的开发工具)
 - profilers (性能分析器) : Development Tools for Larger Projects (更大型项目的开发工具)

- shipping (发布交付) : Development Tools for Larger Projects (更大型项目的开发工具)
- testing tools (测试工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- DFLR (depth first, then left to right) (DFLR (深度优先、然后从左到右)) : Multiple Inheritance and the MRO (多重继承与 MRO)
- diagonals (对角线) : Example: List Comprehensions and Matrixes (示例: 列表推导式与矩阵)
- diamonds, user-defined (diamonds, 用户定义-) : How the MRO Works (MRO 如何工作)
- dict attribute (dict 属性) : How Files Generate Namespaces (文件如何生成命名空间) ; Namespace Dictionaries: dict (命名空间字典: dict) ; Module Introspection (模块内省) ; Example: Listing Modules with dict (示例: 用 dict 列出模块)
- dictionaries (字典) : Why Use Built-in Objects? (为什么使用内置对象?) ; Python' s Core Object Types (Python 的核心对象类型)
- Dictionaries (字典) : Test Your Knowledge: Answers (测验答案) ; Types (类型)
- backward indexing (反向索引) : Mapping values to keys (映射值到键)
- books database example (books 数据库示例) : Intermission: Books Database (Intermission: Books 数

- 数据库) ; Mapping values to keys (映射值到键)
- building (构建) : Other Dictionary Makers (Other 字典 Makers) ; Dictionary-literal unpacking (字典-字面量解包)
- characteristics (特征) : Dictionaries (字典)
- classes versus dictionaries (类对比字典) : Records Revisited: Classes Versus Dictionaries (回顾记录: 类对比字典)
- collections (集合容器) : Dictionaries (字典)
- comparisons (比较运算: lt、gt 等) : Dictionary comparisons (字典比较)
- comprehensions (推导式) : Dictionary Comprehensions (字典推导式)
- Comprehensions versus type calls and generators (推导式对比类型调用与生成器) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- syntax (语法) : Formal Comprehension Syntax (推导式的正式语法)
- converting to tuples (converting 到元组) : Records Revisited: Named Tuples (再看记录: 命名元组)
- copying (拷贝) : Missing Keys: if Tests (Missing 键: if Tests)
- empty (空) : Dictionaries (字典)
- function annotations (函数注解) : Function Annot

ations and Decorations (函数注解与装饰)

- in operator (中的运算符) : Basic Dictionary Operations (字典基本操作)

- indexing (索引) : Mapping Operations (映射操作) ; Test Your Knowledge: Part II Exercises (自测: 第二部分练习)

- Part II (第 II 部分)

- initializing (正在初始化) : Dictionary Comprehensions (字典推导式)

- iteration (迭代) : Item Iteration: for Loops (按项迭代: for 循环) ; Reprise: Dictionaries, range, enumerate, and zip (重温: 字典、range、enumerate 与 zip)

- key insertion order (键插入顺序) : Key Insertion Ordering (键 Insertion Ordering)

- key order (键 order) : Dictionaries (字典)

- keys (键) : Dictionaries (字典)

- immutable objects (不可变对象) : Dictionary Usage Tips (字典用法 Tips)

- integer keys (整数键) : Using dictionaries to simulate flexible lists: Integer keys (Using 字典到 simulate flexible 列表: 整数键)

- mapping to (映射到) : Mapping values to keys (映射值到键)

- missing (缺失) : Avoiding missing-key errors (避免键缺失错误)
- sorting (排序) : Sorting dictionary keys (Sorting 字典键)
- storage (存储) : Basic Dictionary Operations (字典基本操作)
- tuple keys (元组键) : Using dictionaries for sparse data structures: Tuple keys (Using 字典用于 sparse data structures: 元组键)
- keys method (键方法) : Basic Dictionary Operations (字典基本操作)
- len function (len 函数) : Basic Dictionary Operations (字典基本操作)
- lists, sorting (列表, sorting) : Sorting lists (Sorting 列表)
- literal expressions (literal 表达式) : Dictionaries (字典)
- literal unpacking (字面量解包) : Dictionary-literal unpacking (字典-字面量解包)
- literals (字面量) : Dictionaries (字典)
- magnitude comparisons (magnitude 比较运算: lt、gt 等) : Dictionary magnitude comparisons (字典 magnitude 比较运算: lt、gt 等)
- methods (方法) : Missing Keys: if Tests (Missing 键: if Tests) ; More Dictionary Methods (More 字典

方法)

- module namespaces (模块命名空间) : How Files Generate Namespaces (文件如何生成命名空间) ; Namespace Dictionaries: dict (命名空间字典: dict)
- mutability (可变性) : Dictionaries (字典) ; Changing Dictionaries in Place (Changing 字典中的 Place)
- namespaces (命名空间) : Classes: Under the Hood (类: 揭开引擎盖) ; Namespace Dictionaries: Review (命名空间字典: 回顾)
- slots (slots (插槽)) : Slot basics (slot 基础) ; Slots and namespace dictionaries (slots (插槽) 与命名空间字典)
- nesting (嵌套) : Nesting Revisited (再谈嵌套) ; Nesting in dictionaries (嵌套中的字典)
- new indexes (新索引) : Dictionary Usage Tips (字典用法 Tips)
- object references (对象引用) : Dictionaries (字典)
- operations (操作) : Dictionaries (字典)
- mapping (映射) : Mapping Operations (映射操作)
- sequence operations (序列操作) : Dictionary Usage Tips (字典用法 Tips)

- sets (集合) : Dictionary views and sets (字典 views 与集合)
- sorting (排序) : Test Your Knowledge: Part III Exercises (第三部分练习) ; Part III, Statements and Syntax (第 III 部分, 语句与语法)
- union operator (union 运算符) : Dictionary "Union" Operator (字典 "Union" 运算符)
- updating (正在更新) : Missing Keys: if Tests (Missing 键: if Tests)
- views (视图) : Dictionary views and sets (字典 views 与集合)
- views objects (views 对象) : Dictionary key/value/item view objects (字典 key/value/item 视图对象)
- zip object (zip 对象) : More zip roles: dictionaries (zip 的更多角色: 字典)
- dictionary keys (字典键) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; Part II, Objects and Operations (第 II 部分, 对象与操作)
- if statements (if 语句) : Missing Keys: if Tests (Missing 键: if Tests)
- indexing and (索引与) : Mapping Operations (映射操作)
- insertion order (插入顺序) : Mapping Operations (映射操作)

- dictionary tools (字典工具) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- dictionary-based formatting expressions (字典-based 格式化表达式) : Dictionary-based formatting expressions (字典-based 格式化表达式)
- dir function (dir 函数) : Getting Help (获取帮助)
- object attributes (对象属性) : The dir Function (dir 函数)
- name strings (名字字符串) : The dir Function (dir 函数)
- type names (类型名字) : The dir Function (dir 函数)
- direct recursion (直接递归) : Coding Alternatives (编码替代方案)
- directories (目录)
- commands (命令) : Running Files with Command Lines (用命令行运行文件)
- module search path (模块 search path) : Search-Path Components (搜索路径的组成部分)
- .pth path-file (.pth 路径文件) : Search-Path Components (搜索路径的组成部分)
- PYTHONPATH (PYTHONPATH 环境变量) : Search-Path Components (搜索路径的组成部分) ; Configuring the Search Path (配置搜索路径)

- site-packages (site-packages 目录) : Search-Path Components (搜索路径的组成部分)
- standard-library directories (标准库- 目录) : Search-Path Components (搜索路径的组成部分)
- directory paths (目录路径) : Files in Action (文件实战)
- filename (文件名) : Opening Files (打开文件)
- disabling code, triple-quoted strings (disabling code, triple-quoted 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- Django (Django Web 框架) : GUIs and UIs (GUI 与 UI)
- do until loop (do-until 风格 loop) : Examples (示例)
- docstrings (文档字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串) ; Python Documentation Sources (Python 文档资源)
- Docstrings and doc (文档字符串与 doc) : Documentation Strings Revisited (重温文档字符串)
- built-in (内置) : Built-in docstrings (内置- 文档字符串)
- content (内容) : User-defined docstrings (用户定义的文档字符串)
- indentation (缩进) : User-defined docstrings (用户定义的文档字符串)

- literals (字面量) : User-defined docstrings (用户定义的文档字符串)
- object classes (对象类) : User-defined docstrings (用户定义的文档字符串)
- print (print (打印)) : User-defined docstrings (用户定义的文档字符串)
- raw strings (raw 字符串) : User-defined docstrings (用户定义的文档字符串)
- standards (标准) : Docstring standards (文档字符串标准)
- triple-quoted block (三引号代码块) : User-defined docstrings (用户定义的文档字符串)
- user-defined (用户定义) : User-defined docstrings (用户定义的文档字符串)
- documentation (文档) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识) ; Python (Python 语言)
- # comments (# 注释) : # Comments (# 注释)
- dir function (dir 函数) : The dir Function (dir 函数)
- priorities (优先级) : Docstring standards (文档字符串标准)
- Sphinx (Sphinx 文档) : Beyond Docstrings: Sphinx (超越 Docstring: Sphinx)

- documentation tools (文档工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- dotted-path syntax, packages (点分路径- 语法, 包) : Package Imports (包导入)
- module import (模块 import) : Using the basic package (使用基础包)
- relative imports (相对导入) : Relative-Import Rationales and Trade-Offs (相对导入的理由与权衡)
- Durus (Durus 持久化库) : Database Access (数据库访问)
- dynamic typing (动态类型) : It' s Powerful (它很强大)
- declaration statements (declaration 语句) : The Case of the Missing Declaration Statements (缺失的声明语句)
- objects (对象) : Variables, Objects, and References (变量、对象与引用)
- polymorphism and (多态与) : Dynamic Typing Is Everywhere (动态类型无处不在)
- variables (变量) : Variables, Objects, and References (变量、对象与引用)

E (字母 E)

- ease of use (ease 的用法) : It' s Relatively Easy to Use (It' s Relatively Easy 到用法)

- else clause (else 子句) : Loop else (循环 else) ;
Why the loop else? (为什么要有循环 else?)
- else statements (else 语句) : break, continue, pass, and the Loop else (break、continue、pass 与循环 else)
- embedding (嵌入)
- built-in object types (内置- 对象类型) : Extending Types by Embedding (扩展 (重复/连接) 类型 by 嵌入)
- embedded objects (embedded 对象) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- empty data structures (空数据结构) : The Meaning of True and False in Python (Python 中 True 与 False 的含义)
- empty dictionaries (empty 字典) : Dictionaries (字典)
- empty strings (empty 字符串) : Files in Action (文件实战)
- encapsulation (封装) : Coding Methods (编码方法) ; Python and OOP (Python 与 OOP) ; Test Your Knowledge: Part VI Exercises (第六部分练习)
- packaging (打包) : Python and OOP (Python 与 OOP)

- enclosing scopes (enclosing 作用域)
- functions (函数) : Enclosing scopes and loop variables (外层作用域与循环变量)
- state retention and (状态 retention 与) : State retention and enclosing scopes (状态 retention 与 enclosing 作用域)
- encoding (编码)
- source files (source 文件) : Source-File Encoding Declarations (源文件编码声明)
- Unicode (Unicode 统一码) : Character Encodings (字符编码)
- UTF-8 (UTF-8 编码) : Character Encodings (字符编码)
- end-of-line character (end-of-line character) : The print function in action (print 函数中的 action)
- enumerate function, loops (enumerate 函数, 循环) : Offsets and Items: enumerate (Offsets 与项: enumerate)
- equality, testing (equality, 测试)
- exceptions (异常) : Exception Roles (异常的角色)
- testing user input (测试 user input) : Handling Errors by Testing Inputs (Handling Errors by 测试 Inputs)
- try statement (try 语句) : Handling Errors with try

Statements (Handling Errors 带 try 语句)

- error messages (错误消息) : Displaying Errors and Tracebacks (显示错误与回溯信息)

- error-checking tools (error-checking 工具) : Development Tools for Larger Projects (更大型项目的开发工具)

- errors (错误) : Part I, Getting Started (第 I 部分: 入门)

- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)

- escape characters (转义字符) : Escape Sequences Are Special Characters (转义序列是特殊字符) ; Escape Sequences (Escape 序列)

- Are Special Characters (是特殊字符)

- raw strings (raw 字符串) : Raw Strings Suppress Escapes (Raw 字符串抑制转义)

- eval function (eval 函数) : String Conversion Tools (字符串 Conversion 工具)

- event notification (事件通知) : Exception Roles (异常的角色)

- Excel (Excel 表格) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...) ; Other Launch Options (其它启动选项)

- exception classes (异常类) : Exception Objects (异常对象)
- built-in (内置) : Built-in Exception Classes (内置异常类)
- ArithmeticError (ArithmeticError 异常) : Built-in Exception Classes (内置异常类)
- BaseException (BaseException 异常) : Built-in Exception Classes (内置异常类)
- Exception (异常) : Built-in Exception Classes (内置异常类)
- LookupError (LookupError 异常) : Built-in Exception Classes (内置异常类)
- OSError (OSError 异常) : Built-in Exception Classes (内置异常类)
- categories, catching (按类别捕获) : Coding Exceptions Classes (编写异常类)
- Exception (异常) : Built-in Exception Classes (内置异常类)
- exception details (异常细节) : Coding Exceptions Classes (编写异常类)
- hierarchies (层次结构) : Exception Classes (异常类) ; Why Exception Hierarchies? (为什么需要异常层次结构?)
- raising instances (raising 实例) : Coding Exceptions Classes (编写异常类)

- state and (状态与) : Exception Objects (异常对象)
- superclasses (超类) : Coding Exceptions Classes (编写异常类)
- exception groups (异常 groups) : Exception Groups: Yet Another Star! (异常组: 又一颗新星!)
- nesting (嵌套) : Example: Control-Flow Nesting (示例: 控制流嵌套)
- exception handlers (异常 handlers) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- nesting (嵌套) : Nesting Exception Handlers (嵌套异常处理器)
- control-flow (控制流) : Example: Control-Flow Nesting (示例: 控制流嵌套)
- syntactic nesting (语法嵌套) : Example: Syntactic Nesting (示例: 语法嵌套)
- sys.excinfo (sys.excinfo 异常信息) : More on sys.excinfo (深入 sys.excinfo) ; The sys.exception alternative—and diss (sys.exception 替代方案——以及吐槽)
- try/except/else (try/except/else 异常结构) : Functions Can Signal Conditions with raise (函数可用 raise 发出条件信号)
- exception superclass (异常超类) : Coding Exceptions Classes (编写异常类)

- exceptions (异常) : Why Use Exceptions? (为什么要使用异常?)
- as string objects (as 字符串对象) : The raise Statement (raise 语句)
- assert statement (assert 语句) : The assert Statement (assert 语句) ; Example: Trapping Constraints (but Not Errors!) (示例: 捕获约束条件——但不是错误!)
- built-in (内置)
- catching (捕获) : Example: Catching built-in exceptions (示例: 捕捉内置异常)
- custom printing (自定义打印) : Custom Print Displays (自定义打印显示)
- default printing (默认打印) : Default Printing and State (默认打印与状态)
- state (状态) : Default Printing and State (默认打印与状态) ; Custom State and Behavior (自定义状态与行为) ; Providing Exception Methods (提供异常方法)
- catching (捕获) : Catching Exceptions (Catching 异常)
- categories (类别) : Built-in Exception Categories (内置异常类别)
- chaining (链式) : Exception Chaining: raise from (异常链: raise from)

- class instance objects (类实例对象) : Exception Objects (异常对象)
- class-based (类-based) : Catching Too Little: Use Class-Based Categories (捕获太少: 使用基于类的分类)
- code reuse (code 复用) : Exception Objects (异常对象)
- connection closure (connection 闭包) : Closing Files and Server Connections (Closing 文件与 Server Connections)
- control flow (控制流) : Exception Roles (异常的角色)
- debugging and (调试与) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- default behaviors (默认行为) : Example: Default behavior (示例: 默认行为)
- details (细节) : Providing Exception Details (提供异常细节)
- error checks (错误检查) : Test Your Knowledge: Answers (测验答案)
- error handling (错误处理) : Exception Roles (异常的角色)
- error messages (错误消息) : Displaying Errors and Tracebacks (显示错误与回溯信息)

- event notification (事件通知) : Exception Roles (异常的角色)
- except clause (except 子句) : Catching Too Much: Avoid Empty except and Exception (捕获得过多: 避免空 except 与 Exception)
- exception handler (异常 handler) : Default Exception Handler (Default 异常 Handler)
- file closing (文件 closing) : Closing Files and Server Connections (Closing 文件与 Server Connections)
- hierarchies (层次结构) : Exception Classes (异常类) ; Why Exception Hierarchies? (为什么需要异常层次结构?)
- methods (方法) : Providing Exception Methods (提供异常方法)
- no exception case (no 异常 case) : Catching the no-exception case with else (用 else 捕捉"无异常"的情况)
- propagating (传播) : Propagating Exceptions with raise (用 raise 传播异常)
- raise statement (raise 语句) : The raise Statement (raise 语句) ; Exception Chaining: raise from (异常链: raise from)
- Breaking Out of Multiple Nested Loops: "go to" (跳出多层嵌套循环: "goto")

- raising (引发) : Raising Exceptions (引发异常)
- scopes (作用域) : Scopes and except as (作用域与 except as)
- as signals (作为信号) : Exceptions Aren' t Always Errors (异常 Aren' t Always Errors)
- special-case handling (特殊情况处理) : Exception Roles (异常的角色)
- superclasses (超类) : Exception Classes (异常类)
- termination (终止) : Exception Roles (异常的角色) ; Termination Actions (终止动作)
- termination handlers (终止处理器) : The Termination-Handlers Shoot-Out (终止处理器巅峰对决)
- tests, in-process (tests, 中的-process) : Running In-Process Tests (运行中的-Process Tests)
- traceback object (traceback 对象) : Displaying Errors and Tracebacks (显示错误与回溯信息)
- try statement (try 语句) : The try Statement (try 语句) ; Combined-clauses example (组合子句示例)
- debugging and (调试与) : Debugging with Outer try Statements (调试带 Outer try 语句)
- tuples (元组) : Catching many exceptions with a tuple (用元组捕捉多个异常)
- unexpected (意外) : Catching all exceptions with empties and Exception (用空 except 与 Exception 捕

捉所有异常)

- user-defined (用户定义) : User-Defined Exceptions (用户定义- 异常)
- nonerror conditions (非错误条件) : Functions Can Signal Conditions with raise (函数可用 raise 发出条件信号)
- variables, scopes (变量, 作用域) : Preview: Other Python scopes (预览: Python 的其他作用域)
- with statement (带语句) : The with Statement and Context Managers (with 语句与上下文管理器) ; The Termination-Handlers Shoot-Out (终止处理器巅峰对决)
- wrappers (包装器) : What Should Be Wrapped (应该包裹什么)
- exec function (exec 函数) : The exec built-in (exec 内置-)
- executable statements (executable 语句)
- from (from (导入自)) : Imports Are Runtime Assignments (导入是运行时赋值)
- import (import (导入)) : Imports Are Runtime Assignments (导入是运行时赋值)
- executables, standalone (独立可执行文件) : Standalone Executables (独立可执行文件) ; Standalone Apps and Executables (A.7 独立应用与可执行程序) ; Etcetera (A.8 其他)

- execution (执行)
- deferring (推迟) : Anonymous Functions: lambda (匿名函数: lambda)
- speed (速度) : OK, but What' s the Downside? (好, 但缺点是什么?)
- exponentiation (乘方) : Numbers (数字)
- expression statements (表达式语句) : Expression S tatements (表达式语句)
- in-place changes (中的-place 修改) : Expression S tatements and In-Place Changes (表达式语句与中的-Place 修改)
- expressions (表达式) : Running Code Interactively (交互式运行代码) ; Test Your Knowledge: Answers (测验答案)
- arbitrary (任意) : Sequence Operations (序列操作) ; List Comprehensions Review (列表推导式回顾)
- commas (逗号) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- concept hierarchy (A.7.1 概念: 把 Python 打包成普通程序 hierarchy) : The Python Conceptual Hierarc hy Revisited (重访 Python 概念层级)
- curly braces (花括号) : Python' s Core Object Typ es (Python 的核心对象类型)
- dictionaries (字典) : Python' s Core Object Types (Python 的核心对象类型)

- function-related (函数-related) : Function Basics (函数基础)
- generator expressions (生成器表达式) : Comprehensions and (推导式与)
- Functions and Expressions (函数与表达式) : Generator Functions (生成器函数)
- Expressions (表达式) : Listing instance attributes with dict (用 dict 列出实例属性)
- as iterables (as 可迭代对象) : Generator Expressions: Iterables Meet Comprehensions (生成器表达式: 可迭代对象遇见推导式)
- best uses (最佳用途) : Why generator expressions ? (Why 生成器表达式?)
- versus filter function (对比 filter 函数) : Generator expressions versus filter (生成器表达式与 filter 的对比)
- generator objects (生成器对象) : Generator Expressions: (生成器表达式:)
- versus map function (对比 map 函数) : Generator expressions versus map (生成器表达式与 map 的对比)
- slicing sequences (slicing 序列) : Generator expressions (生成器表达式)
- if/then/else (if/then/else 分支) : The if/else Ternary Expression (if/else 三元表达式)

- indexing expressions (索引表达式) : Sequence Operations (序列操作)
- slicing (切片) : Sequence Operations (序列操作)
- lambda (lambda (匿名函数)) : Basic Function Tools (基本函数工具)
- list comprehensions (列表推导式) : Comprehensions: The Final Act (推导式: 最后一幕)
- mapping (映射) : List Comprehensions Review (列表推导式回顾)
- lists (列表) : Python's Core Object Types (Python 的核心对象类型)
- literal expressions (literal 表达式) : Python's Core Object Types (Python 的核心对象类型)
- method call expressions (方法调用表达式: call) : Method Call Syntax (方法 Call 语法)
- named assignment expression (命名赋值 (海象) 表达式) : Assignment Syntax Forms (赋值语法形式) ; Named Assignment Expressions (命名赋值表达式) ; When to use named assignment (When 到用法 命名赋值 (海象))
- objects (对象) : The Python Conceptual Hierarchy (Python 概念层级)
- operator overload (运算符 overload) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)

- slicing expression (slicing 表达式) : Extended slicing: The third limit and slice objects (扩展切片: third limit 与 slice 对象)
- square brackets (方括号) : Python' s Core Object Types (Python 的核心对象类型)
- string formatting (字符串格式化) : String-Formatting Options (字符串-格式化 Options) ; Formatting expression basics (格式化表达式基础)
- advanced (高级) : Advanced formatting expression examples (高级格式化表达式示例)
- dictionary-based (字典-based) : Dictionary-based formatting expressions (字典-based 格式化表达式) ; Dictionary based formatting expressions (字典 based 格式化表达式)
- type codes (类型 codes) : Formatting expression custom formats (格式化表达式自定义格式)
- strings (字符串) : String Object Basics (字符串对象基础)
- extended slicing (扩展切片) : Extended slicing: The third limit and slice objects (扩展切片: third limit 与 slice 对象)
- extended-unpacking assignments (扩展解包赋值) : Extended-Unpacking Assignments (扩展解包赋值) ; Extended unpacking in action (扩展解包实战)
- boundary cases (边界情况) : Boundary cases (边

界情况)

- for loops (for 循环) : Application to for loops (应用于 for 循环) ; Extended-unpacking assignment (扩展解包- assignment)

F (字母 F)

- f-strings (f-字符串) : The print function in action (print 函数中的 action)
- literals (字面量) : The F-String Formatting Literal (F-String 格式化字面量)
- backslash (反斜杠) : F-string formatting basics (F-字符串格式化基础)
- custom formats (自定义格式) : F-string custom formats (F-字符串自定义格式)
- quotes (引号) : F-string formatting basics (F-字符串格式化基础)
- REPL (REPL (交互运行环境)) : F-string formatting basics (F-字符串格式化基础)
- substitutions (替换) : F-string formatting basics (F-字符串格式化基础)
- factorials (阶乘) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- factory functions (工厂函数) : Closures and Factory Functions (闭包与工厂函数) ; Classes Are Objects: Generic Object Factories (类也是对象: 通用对象工厂)

-) ; Why Factories? (为什么需要工厂?)
- metaclasses (元类) : Using simple factory functions (使用简单工厂函数)
- false values (False 值) : The Meaning of True and False in Python (Python 中 True 与 False 的含义) ; The bool type (bool 类型)
- FIFO (first in, first out), queues (FIFO (先进先出) 队列) : Other File Tools (其它文件工具)
- file extensions, imports (文件扩展, 导入) : Common Coding Gotchas (常见编码陷阱)
- file icons, program running (文件 icons, 程序运行) : Clicking and Tapping File Icons (Clicking 与 Tapping 文件 Icons)
- file objects (文件对象)
- creating (创建) : Files (文件)
- print operations (print 操作) : Print Operations (print 操作)
- file operations (文件操作) : Files (文件)
- file-processing (文件-processing) : Files in Action (文件实战)
- filenames (文件名) : Filenames in open and Other Filename Tools (文件名: open 与其它文件名工具)
- directory path (目录路径) : Opening Files (打开文件)

- modules (模块) : Module Filenames (模块文件名)
- program files (程序文件) : The Programmer' s View (程序员视角)
- files (文件) : Files (文件) ; Test Your Knowledge: Answers (测验答案)
- access-by-key (access-by-键) : Other File Tools (其它文件工具)
- binary (二进制) : Text and Binary Files: The Short Story (文本与二进制文件: 速览) ; Text and Binary Files: (文本文件与二进制文件:)
- binary-mode (二进制模式) : Using Text and Binary Files (使用文本与二进制文件)
- buffered (缓冲的) : Using Files (使用文件)
- byte files (字节文件) : Unicode and Byte Files (Unicode 与字节文件)
- bytecode (字节码) : Bytecode compilation (字节码编译)
- close method (close 方法) : Using Files (使用文件)
- content (内容) : Using Files (使用文件)
- context managers (上下文管理器) : File Context Managers (文件上下文管理器)
- cross-file changes, minimizing (cross-文件修改, mi

- nimizing) : Program Design: Minimize Cross-File Changes (程序设计: 尽量减少跨文件修改)
- descriptor files, os module (描述符文件, os 模块) : Other File Tools (其它文件工具)
- directory paths (目录路径) : Files in Action (文件实战)
- FIFOs (FIFO 队列) : Other File Tools (其它文件工具)
- iteration (迭代) : Files (文件)
- iterators (迭代器) : Using Files (使用文件) ; Files in Action (文件实战)
- list comprehensions (列表推导式) : List Comprehensions and Files (列表推导式与文件) ; List Comprehensions (列表推导式)
- and Files (与文件)
- methods export (方法 export) : Core Types Review and Summary (核心类型回顾与总结)
- newline characters (换行符) : Files in Action (文件实战)
- number types (数字类型) : Core Types Review and Summary (核心类型回顾与总结)
- object storage (对象 storage) : Storing Objects with Conversions (用转换来存储对象)
- open function (open 函数) : Other File-Like Tools

(Other 文件-Like 工具)

- opening (开篇引言) : Opening Files (打开文件)
- pipes (管道) : Other File Tools (其它文件工具)
- program architecture (程序架构) : How to Structure a Program (如何组织一个程序)
- running as standalone script (运行 as standalone 脚本) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- sets (集合) : Core Types Review and Summary (核心类型回顾与总结)
- shell-command streams (shell 命令流) : Other File Tools (其它文件工具)
- shelves (shelve 存储) : Other File Tools (其它文件工具)
- sockets (套接字) : Other File Tools (其它文件工具)
- standard streams (标准流) : Other File Tools (其它文件工具)
- string types (字符串类型) : Core Types Review and Summary (核心类型回顾与总结)
- strings (字符串) : Using Files (使用文件)
- empty (空) : Files in Action (文件实战)
- text files (text 文件) : Using Files (使用文件)

- conversions (转换) : Storing Objects with Conversions (用转换来存储对象)
- pathname (路径名) : Text-File Basics (Text-文件基础)
- text output (文本输出) : Files (文件)
- text-mode (文本模式) : Using Text and Binary Files (使用文本与二进制文件)
- Unicode and (Unicode 与) : Unicode and Byte Files (Unicode 与字节文件)
- filter function (filter 函数) : Selecting Items in Iterables: filter (Selecting 项中的可迭代对象: filter) ; Generator expressions versus filter (生成器表达式与 filter 的对比)
- finalization (see termination actions) (finalization (see 终止动作))
- find method, "Changing" (find 方法, "Changing") : Strings (字符串)
- conversion (转换) : String Conversion Tools (字符串 Conversion 工具)
- files (文件) : Core Types Review and Summary (核心类型回顾与总结)
- interactive loops (交互式循环) : Supporting Floating-Point Numbers (Supporting 浮点- 数字) ; Supporting Floating- Point Numbers (Supporting 浮点- 数字)

- folder hierarchies, package imports (文件夹层次, 包导入) : Package Imports (包导入)
- folders (文件夹)
- packages (包) : Module Packages (模块包)
- names (名字) : Basic Package Structure (基本包结构)
- nesting (嵌套) : Basic Package Structure (基本包结构)
- for loops (for 循环) : Item Iteration: for Loops (按项迭代: for 循环) ; for Loops (for 循环)
- comprehensions (推导式) : Changing Lists: range and Comprehensions (Changing 列表: range 与推导式)
- data types (data 类型) : Basic usage (基本用法)
- dictionary comprehensions (字典推导式) : Dictionary Comprehensions (字典推导式)
- extended-unpacking assignments (扩展解包赋值) : Application to for loops (应用于 for 循环) ; Extended- unpacking assignment in for loops (扩展解包-assignment 中的 for 循环) ; Extended-unpacking assignment (扩展解包- assignment)
- in functions (中的函数) : Definition (定义)
- getitem method (getitem 方法) : Index Iteration: getitem (索引迭代: getitem)

- iteration protocol (迭代协议) : Iteration protocol i
ntegration (与迭代协议的集成)
- lists (列表) : Changing Lists: range and Comprehe
nsions (Changing 列表: range 与推导式)
- nested (嵌套) : Nested for loops (嵌套 for 循环)
- list comprehensions (列表推导式) : Nested loops:
for (嵌套循环: 用于) ; Formal Comprehension Synta
x (推导式的正式语法)
- sequence assignment (序列 assignment) : Tuple (
sequence) assignment in for loops (元组 (序列) assig
nment 中的 for 循环)
- argument matching syntax (参数匹配语法) : Argu
ment Matching Syntax (参数匹配语法)
- arguments (参数) : def Statements (def 语句)
- coupling (耦合) : Function Design Concepts (函
数设计概念)
- passing (传递) : General Function Concepts (通用
函数概念)
- validation (校验) : Example: Validating Function A
rguments (案例二: 验证函数参数) ; Decorator Argu
ments Versus Function Annotations (装饰器参数对比
函数注解)
- async (异步) : Asynchronous Functions: The Short
Story (异步函数: 短篇故事) ; The Async Wrap-Up (
异步总结)

- async statement (async 语句) : Advanced Function Tools (高级函数工具)
- attributes (属性) : General Function Concepts (通用函数概念)
- state (状态) : State with function attributes (状态带函数属性)
- state retention (状态 retention) : Function Attributes: Changeable, Per-Call, Explicit (函数属性: 可修改、每调用独立、显式)
- user-defined (用户定义) : Function Attributes (函数属性)
- await statement (await 语句) : Advanced Function Tools (高级函数工具)
- built-in, handling by assignment (内置-, 通过赋值处理) : Hiding built-ins by assignment (Hiding 内置-by assignment)
- calls (调用) : Calls (调用)
- def statement (def 语句) : Calls (调用)
- classmethod (classmethod (类方法)) : Using Static and Class Methods (Using 静态方法与类方法)
- closure functions (闭包函数) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- cohesion (内聚) : Function Design Concepts (函数设计概念)

- coroutines (协程) : Advanced Function Tools (高级函数工具)
- coupling (耦合) : Function Design Concepts (函数设计概念)
- decorations (装饰) : Function Annotations and Decorations (函数注解与装饰) ; Function decorators (函数装饰器)
- alternative: Preview (替代方案: 预览)
- decorators (装饰器) : General Function Concepts (通用函数概念)
- def statements (def 语句) : Basic Function Tools (基本函数工具) ; def Statements (def 语句) ; def Executes at Runtime (def 在运行时执行)
- defaults (默认值) : Defaults and Mutable Objects (默认参数与可变对象)
- deferring execution (推迟执行) : Anonymous Functions: lambda (匿名函数: lambda)
- definitions (定义) : General Function Concepts (通用函数概念) ; Definition (定义)
- dir (dir (目录/内省)) : Getting Help (获取帮助)
- eval (eval (求值)) : String Conversion Tools (字符串 Conversion 工具)
- exec (exec (执行)) : The exec built-in (exec 内置-)

- execution environment (执行环境) : Function Design Concepts (函数设计概念)
- factory functions (工厂函数) : Using simple factory functions (使用简单工厂函数)
- filter (filter (过滤)) : Selecting Items in Iterables: filter (Selecting 项中的可迭代对象: filter)
- first-class object model (first-类对象模型) : The First-Class Object Model (一等对象模型)
- for loops in (for 循环中的) : Definition (定义)
- generator functions (生成器函数) : Comprehensions and (推导式与)
- and Expressions (与表达式) : Generator functions in action (生成器函数中的 action) ; Generator Functions (生成器函数)
- best uses (最佳用途) : Why generator functions? (Why 生成器函数?)
- iteration protocol (迭代协议) : Generator functions in action (生成器函数中的 action)
- send method (send 方法) : Extended generator function protocol: send versus next (扩展的生成器函数协议: send 与 next)
- slicing sequences (slicing 序列) : Generator functions (生成器函数)
- state suspension (状态 suspension) : State suspension (状态 suspension)

- yield statement (yield 语句) : The yield from extension (yield from 扩展)
- generators (生成器) : Advanced Function Tools (高级函数工具)
- global statement (全局语句) : Advanced Function Tools (高级函数工具)
- global variables (全局变量) : Function Design Concepts (函数设计概念)
- helper functions (helper 函数) : The Downside of “Helper” Functions (「辅助函数」的弊端)
- inlining (内联) : Anonymous Functions: lambda (匿名函数: lambda)
- intersect (相交) : Calls (调用)
- lambda body expression (lambda body 表达式) : Calls (调用)
- polymorphism (多态) : Polymorphism Revisited (再谈多态)
- introspection (内省) : Function Introspection (函数 Introspection) ; Function introspection (函数 introspection)
- lambda expressions (lambda 表达式) : Basic Function Tools (基本函数工具) ; lambda Makes Anonymous Functions (lambda 创建匿名函数)
- len (len (长度)) : Escape Sequences Are Special Characters (转义序列是特殊字符)

- loop variables (loop 变量) : Enclosing scopes and loop variables (外层作用域与循环变量)
- manager functions (manager 函数) : The Downside of “Helper” Functions (「辅助函数」的弊端)
- metafunctions (元函数) : Function Decorator Basics (函数装饰器基础)
- names (名字)
- nested, decorating methods (嵌套, decorating 方法) : Using nested functions to decorate methods (Using 嵌套函数到 decorate 方法)
- nonlocal statement (nonlocal 语句) : Advanced Function Tools (高级函数工具)
- objects, mutable (对象, 可变) : Defaults and Mutable Objects (默认参数与可变对象)
- open (open (打开)) : Other File-Like Tools (Other 文件-Like 工具)
- overview (A.5.1 总览: Python 应用三剑客) : Function Basics (函数基础)
- parameters (参数) : def Statements (def 语句)
- passing as arguments (传递 as 参数) : The First-Class Object Model (一等对象模型)
- plain functions (plain 函数) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Why the Special Methods? (Why Special 方法?)

- recursive (see recursion) (递归 (见 recursion))
- reduce (reduce (归约)) : Combining Items in Iterables: reduce (Combining 项中的可迭代对象: reduce)
- reload (reload (重新加载)) : Reloading Modules (重载模块) ; reload Odds and Ends (reload Odds 与 Ends)
- return statements (return 语句) : Basic Function Tools (基本函数工具) ; return Statements (return 语句)
- returns (返回值) : Functions Without returns (没有 return 的函数)
- scopes, enclosing (作用域, enclosing) : Enclosing scopes and loop variables (外层作用域与循环变量)
- set (集合) : Sets (集合)
- size (大小) : Function Design Concepts (函数设计概念)
- slicing sequences (slicing 序列) : Simple functions (Simple 函数)
- staticmethod (staticmethod (静态方法)) : Using Static and Class Methods (Using 静态方法与类方法)
- str (str (字符串类型)) : Numbers (数字)
- super function (super 函数) : The super Function (super 函数的完整故事)

- argument lists (argument 列表) : Same argument lists (相同的参数列表)
- attribute-fetch algorithm (属性获取算法) : Attribute-fetch algorithm (属性获取算法)
- call-chain anchors (调用链锚点) : Call-chain anchors (调用链锚点)
- deployment (部署) : Universal deployment (通用部署)
- MRO algorithm (MRO 算法) : A “magic” proxy (“魔法” 代理)
- noncalls (非调用) : Noncalls and operator overloading (Noncalls 与运算符重载)
- operator overloading (运算符重载) : Noncalls and operator overloading (Noncalls 与运算符重载)
- testdriver (测试驱动器) : Running In-Process Tests (运行中的-Process Tests)
- timer utility functions (timer utility 函数) : Timer Module: Take 2 (计时模块: 第二版)
- type (类型) : Types (类型)
- type hints (类型提示) : Type Hinting: Optional, Unused, and Why? (类型标注: 可选、不使用、为什么?)
- uses (用途) : Why Use Functions? (为什么要用函数?)

- yield statement (yield 语句) : Advanced Function Tools (高级函数工具)
- future import (future 导入) : Enabling Language Changes: future (启用语言变更: future)

G (字母 G)

- game programming (游戏编程) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- garbage collection (垃圾回收) : Nesting Revisited (再谈嵌套)
- circular references (circular 引用) : Objects Are Garbage-Collected (对象会被垃圾回收)
- objects (对象) : Objects Are Garbage-Collected (对象会被垃圾回收)
- generalized set functions, argument matching (generalized 集合函数, 参数匹配) : Example: Generalized Set Functions (示例: 通用集合函数) ; Testing the Code (测试代码)
- generator expressions (生成器表达式) : Comprehensions and (推导式与)
- as iterables (as 可迭代对象) : Generator Expressions: Iterables Meet Comprehensions (生成器表达式: 可迭代对象遇见推导式)

- best uses (最佳用途) : Why generator expressions ? (Why 生成器表达式?)
- versus filter function (对比 filter 函数) : Generator expressions versus filter (生成器表达式与 filter 的对比)
- generator objects (生成器对象) : Generator Expressions: Iterables Meet Comprehensions (生成器表达式: 可迭代对象遇见推导式)
- versus map function (对比 map 函数) : Generator expressions versus map (生成器表达式与 map 的对比)
- nesting (嵌套) : Generator expressions versus map (生成器表达式与 map 的对比)
- slicing sequences (slicing 序列) : Generator expressions (生成器表达式)
- generator functions (生成器函数) : Comprehensions and (推导式与)
- best uses (最佳用途) : Why generator functions? (Why 生成器函数?)
- iterable objects (可迭代对象: iter 和 next) : Classes versus generators (类对比生成器)
- iteration protocol (迭代协议) : Iteration protocol integration (与迭代协议的集成) ; Generator functions (生成器函数)

- send method (send 方法) : Extended generator function protocol: send versus next (扩展的生成器函数协议: send 与 next)
- slicing sequences (slicing 序列) : Generator functions (生成器函数)
- state suspension (状态 suspension) : State suspension (状态 suspension)
- yield statement (yield 语句) : The yield from extension (yield from 扩展)
- generator objects (生成器对象) : Generator Functions Versus Generator Expressions (生成器函数与生成器表达式对比) ; Coding Alternative: iter Plus yield (编码替代: iter 加 yield) ; Multiple iterators with yield (用 yield 实现多个迭代器)
- generator expressions (生成器表达式) : Generator Expressions: (生成器表达式:)
- built-ins (内置) : Generation in built-ins and classes (Generation 中的内置- 与类)
- classes (类) : Generation in built-ins and classes (Generation 中的内置- 与类)
- comprehensions (推导式) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- memory footprint (内存 footprint) : Why generators here: Space, time, and more (Why 生成器 here: S

pace, time, 与 more)

- permutation (排列) : Why generators here: Space, time, and more (Why 生成器 here: Space, time, 与 more)
- results generation (结果生成) : Generating “infinite” (well, indefinite) results (生成 “无限” (实为不定长) 结果)
- single-pass iterables (single-pass 可迭代对象) : Generators are single-pass iterables (生成器 are single-pass 可迭代对象)
- get method (get 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getattr method (getattr 方法) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合) ; Attribute Reference (属性引用) ; Miscellaneous Class Gotchas (Miscellaneous 类的陷阱) ; Inserting Code to Run on Attribute Access (在属性访问时插入代码) ; getattr and getattribute (getattr 与 getattribute) ; Using getattribute (改用 getattribute)
- versus getattribute method, getattr (对比 getattribute 方法, getattr)
- versus getattr method, getattr (对比 getattr 方法, getattr)
- iteration (迭代) : Index Iteration: getitem (索引迭代: getitem)

- iteration and (迭代与) : Intercepting Item Assignments (拦截项赋值)
- slice expressions (slice 表达式) : Intercepting Slices (拦截切片)
- global namespaces (全局命名空间) : Simple Names: Global Unless Assigned (简单名字: 除非赋值否则全局)
- global scope, state retention (全局 scope, 状态 retention)
- emulation (模拟) : Other Ways to Access Globals (访问全局的其他方式)
- global variables (全局变量)
- coupling (耦合) : Function Design Concepts (函数设计概念)
- cross-file changes (cross-文件修改) : Program Design: Minimize Cross-File Changes (程序设计: 尽量减少跨文件修改)
- multithreading (多线程) : Program Design: Minimize Global Variables (程序设计: 尽量减少全局变量)
- program design (程序设计) : Program Design: Minimize Global Variables (程序设计: 尽量减少全局变量)
- state, decorators (状态与装饰器)
- IDLE (A.2.6 IDLE: Python 自带 IDE) : The IDLE Graphical User Interface (A.2.6 IDLE: Python 自带 IDE)

H (字母 H)

- has-a relationships, composition (has- relationships, 组合) : OOP and Composition: “Has-a” Relationships (OOP 与组合: “有一个” 关系) ; Stream Processors Revisited (再访流处理器)
- hash-based bytecode files (hash-based 字节码文件) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- hashes (哈希) : Dictionaries (字典)
- Haskell (Haskell 语言) : Comprehensions: The Final Act (推导式: 最后一幕)
- helper functions (helper 函数) : The Downside of “Helper” Functions (「辅助函数」的弊端)
- hex escapes (十六进制转义) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- hierarchies, subclassing (层次与子类化) : OOP: The Big Idea (OOP: 大思想)
- HPy (HPy (更好的 C API)) : Component Integration (组件集成)

I (字母 I)

- I/O (input/output) operations (I/O (输入/输出) 操作)

- attribute lists (属性列表) : The dir Function (dir 函数)
- IDLE (A.2.6 IDLE: Python 自带 IDE) : The IDLE Graphical User Interface (A.2.6 IDLE: Python 自带 IDE Graphical User 接口)
- IDLE (A.2.6 IDLE: Python 自带 IDE) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- IDLE GUI (A.2.6 IDLE: Python 自带 IDE GUI) : The IDLE Graphical User Interface (A.2.6 IDLE: Python 自带 IDE Graphical User 接口)
- if else expression (if else 表达式) : Missing Keys: if Tests (Missing 键: if Tests)
- recursion (递归) : Coding Alternatives (编码替代方案)
- if statements (if 语句) : A Tale of Two ifs (两种 if 的故事) ; if Statements (if 语句)
- dictionary keys (字典键) : Missing Keys: if Tests (Missing 键: if Tests)
- list comprehensions (列表推导式) : Filter clauses: if (过滤子句: if)
- match statements (match 语句) : Match versus if live (Match 对比 if live)
- multiple-choice selection (多路选择) : Multiple-Choice Selections (多选逻辑)

- function definition (函数定义) : Handling larger actions (处理更大型动作)
- switch defaults (switch 默认值) : Handling switch defaults (Handling switch 默认值)
- syntax (语法) : Missing Keys: if Tests (Missing 键: if Tests)
- if/then/else expression (if/then/else 表达式) : The if/else Ternary Expression (if/else 三元表达式)
- image processing (图像处理) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- immutability (不可变性) : Test Your Knowledge: Answers (测验答案)
- strings (字符串) : Immutability (不可变性) ; Types (类型)
- tuples (元组) : Tuples (元组) ; Conversions, methods, and immutability (转换、方法与不可变性)
- immutable arguments (不可变参数) : Argument-Passing Basics (参数传递基础)
- immutable sequence of 8-bit integers (不可变序列的 8-bit 整数) : bytes type, The bytes Object immutable sequence of characters, str type, The str Object immutable sequences, strings, String Object Basics immutable types, Mutable Types Can Be Changed in Place, Test Your Knowledge: Part II Exercises, Part II,

Objects and Operations implementation alternatives , Python Implementation Alternatives-Python (字节串类型, bytes 对象不可变序列的 characters, str 类型, str 对象不可变序列, 字符串, 字符串对象基础不可变类型, 可变类型 Can Be Changed 中的 Place, 知识测验: 第 II 部分 Exercises, 第 II 部分, 对象与操作实现替代方案, Python 实现替代方案 -Python)

- implementation alternatives (实现替代方案) : Python Implementation Alternatives (Python 实现替代方案) ; Python (Python 语言)
- Implementation Alternatives (实现替代方案)
- implementation-related objects (实现-related 对象) : Python' s Core Object Types (Python 的核心对象类型)
- import models (import 模型)
- modules (模块) : Python Import Models (Python 的导入模型)
- namespace packages (命名空间包) : Namespace Packages (命名空间包) ; Namespace Packages in Action (命名空间包实战)
- module searches (模块 searches) : The Module Search Algorithm (模块搜索算法)
- package-relative imports (包相对导入) : Package-Relative Imports (包相对导入) ; Python Import Models (Python 的导入模型)

- packages (包) : Module Packages (模块包) ; Package Imports (包导入) ; Python Import Models (Python 的导入模型)
- module searches (模块 searches) : Packages and the Module Search Path (包与模块搜索路径)
- package creation (包 creation) : Creating Packages (创建包) ; Using the updated package (使用更新后的包)
- package initialization (包 initialization) : The Role of init.py Files (init.py 文件的作用)
- import optimization, bytecode (import 优化, 字节码) : Bytecode compilation (字节码编译)
- import statement (import 语句) : The import Statement (import 语句)
- as clause (as 子句) : The as Extension for import and from (import 与 from 的 as 扩展)
- modules (模块) : Step 1: Find It (第一步: 查找文件)
- imports (导入)
- as runtime operations (as runtime 操作) : How Imports Work (import 是如何工作的)
- circular (循环的) : Part V, Modules and Packages (第 V 部分, 模块与包)
- future (future (未来特性)) : Enabling Language Changes: future (启用语言变更: future)

- modules (模块) : Imports and Attributes (导入与属性) ; Step 1: Find It (第一步: 查找文件)
- bytecode (字节码) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- bytecode execution (字节码执行) : Step 3: Run It (第三步: 运行)
- initialization code (初始化代码) : Initialization code (初始化代码)
- namespace nesting (命名空间嵌套) : Namespace Nesting (命名空间嵌套)
- one-time (一次性) : Imports Happen Only Once (导入只发生一次)
- previous imports (previous 导入) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- runtime assignment (运行时赋值) : Imports Are Runtime Assignments (导入是运行时赋值) ; Cross-file name changes (Cross-文件名字修改)
- statement order (语句 order) : Statement Order Matters in Top-Level Code (语句 Order Matters 中的顶层- Code)
- variables (变量) : Imports Versus Scopes (导入与作用域)
- nested (嵌套) : Part V, Modules and Packages (第 V 部分, 模块与包)

- nested modules, attributes (嵌套模块, 属性) : Using the basic package (使用基础包)
- package-relative (包-relative) : Module Packages (模块包)
- absolute (绝对) : Relative and Absolute Imports (相对导入与绝对导入)
- name clashes (名字 clashes) : Module Name Clashes: Package and Package-Relative (模块名字 Clashes: 包与包-Relative)
- Imports (导入)
- relative (相对) : Relative and Absolute Imports (相对导入与绝对导入) ; Python Import Models (Python 的导入模型)
- packages (包) : Module Packages (模块包) ; Package Imports (包导入) ; Python Import Models (Python 的导入模型) ; Part V, Modules and Packages (第 V 部分, 模块与包)
- folder hierarchies (文件夹层次) : Package Imports (包导入)
- from statement (from 语句) : Using the basic package (使用基础包)
- recursive (递归的) : Recursive from Imports May Not Work (Recursive from 导入 May Not Work)
- in-place changes (中的-place 修改) : Shared References and In-Place Changes (共享引用与原地修改)

- Shared References (共享引用)
- and In-Place Changes (与中的-In-Place 修改)
- expression statements (表达式语句) : Expression Statements and In-Place Changes (表达式语句与中的-In-Place 修改)
- names (名字) : Scopes Overview (作用域概述)
- incremental prototyping (增量原型) : Testing as You Go (测试 as You Go)
- indentation (缩进)
- blocks (代码块) : End of indentation is end of block (缩进结束即代码块结束) ; Why Indentation Syntax? (为什么采用缩进语法?)
- consistency (一致性) : Common Coding Gotchas (常见编码陷阱)
- text editors (文本编辑器) : Why Indentation Syntax? (为什么采用缩进语法?)
- index assignments (索引赋值总论) : Index (索引)
- dictionaries (字典) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; Part II, Objects and Operations (第 II 部分, 对象与操作)
- expressions (表达式) : Sequence Operations (序列操作)
- slicing (切片) : Sequence Operations (序列操作)
- instances (实例)

- iteration, getitem (迭代, getitem) : Index Iteration : getitem (索引迭代: getitem) ; Index (索引)
- operator overloading (运算符重载) : Indexing and Slicing: (索引与切片: getitem 和 setitem:)
- lists (列表) : Indexing and Slicing (索引与切片: getitem 和 setitem)
- out of bounds (out 的边界) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- string indexing (字符串索引) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; Part II, Objects and Operations (第 II 部分, 对象与操作)
- strings (字符串) : Indexing and Slicing (索引与切片: getitem 和 setitem) ; Extended slicing: (扩展切片:)
- attribute fetches (属性 fetches) : Classes: Under the Hood (类: 揭开引擎盖)
- built-ins (内置) : The built-ins bifurcation (内置分岔)
- class trees (类 trees) : Python Inheritance Algorithm: The Simple Version (Python 继承算法: 简单版)
- classes (类) : OOP: The Big Picture (OOP: 大图景) ; Inheritance (继承)
- attribute trees (属性 trees) : Attribute Tree Construction (属性树构建)

- customization (定制) : Classes Are Customized by Inheritance (类通过继承进行定制) ; A Second Example (第二个示例)
- inheritance search (继承 search) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- metaclass subclasses and (元类子类与) : The Inheritance Bifurcation (继承 Bifurcation)
- privacy (隐私) : Inheritance Fine Print (继承细则)
- composites (复合) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- customization by (通过...定制) : Step 4: Customizing Behavior by Subclassing (第四步: 通过子类化定制行为)
- data descriptors (data 描述符) : The descriptors deviation (描述符偏误) ; The assignment addendum (赋值附录)
- versus delegation (对比委托) : Inheritance versus delegation (继承对比委托)
- descriptors (描述符) : The Basics (基础) ; A First Example (getattr 第一个示例)
- inherited methods (inherited 方法) : Specializing Inherited Methods (特化继承方法)
- is-a relationships ("是一个" 关系) : OOP and Inheritance: "Is-a" Relationships (OOP (面向对象编程

-) 与继承: “Is-” Relationships)
- metaclasses (元类)
- Inheritance: The Finale (继承: 终章) : Metaclass Inheritance (元类继承)
- inheritance relationships (继承 relationships) : Inheritance: The Finale (继承: 终章)
- names acquisition (名字 acquisition) : Metaclass Versus Superclass (元类 vs 超类)
- MRO (method resolution order) (MRO (方法 resolution order)) : Multiple Inheritance and the MRO (多重继承与 MRO) ; How the MRO Works (MRO 如何工作) ; The Inheritance Bifurcation (继承 Bifurcation)
- multiple (多个) : Multiple Inheritance and the MRO (多重继承与 MRO)
- attribute conflicts (属性 conflicts) : Attribute Conflict Resolution (属性冲突的解决)
- DFLR (DFLR (深度优先从左到右)) : Multiple Inheritance and the MRO (多重继承与 MRO)
- inheritance sources (继承 sources) : Example: Mapping Attributes to Inheritance Sources (示例: 将属性映射到继承来源)
- mix-in classes (mix-中的类) : Example: “Mix-in” Attribute Listers (示例: “混入”式属性列出器) ; Listing attributes per object in class trees (在类树中按对象

列出属性)

- order (顺序) : Multiple Inheritance: Order Matters (多重继承: Order Matters) ; Multiple Inheritance: (多重继承:)
- overview (A.5.1 总览: Python 应用三剑客) : How Multiple Inheritance Works (多重继承如何工作)
- slots (slots (插槽)) : Slot usage rules (slot 使用规则) ; Example impacts of slots: ListTree and map attrs (示例 impacts 的 slots (插槽) : ListTree 与 map attrs)
- namespaces (命名空间) : Example: Class Attributes (示例: 类属性)
- nonclass instances (nonclass 实例) : The Inheritance Bifurcation (继承 Bifurcation)
- OOP (OOP (面向对象编程)) : Why Use Classes? (为什么使用类?) ; OOP and Inheritance: "Is-a" Relationships (OOP (面向对象编程) 与继承: "Is-" Relationships)
- attributes (属性) : Attribute Inheritance Search (属性继承搜索)
- class customization (类 customization) : Classes Are Customized by Inheritance (类通过继承进行定制) ; A Second Example (第二个示例)
- multiple inheritance (多重继承) : Coding Class Trees (编写类树)

- operator-overloading methods (运算符重载- 方法)
: Common Operator-Overloading Methods (常用运算符重载方法)
- properties (properties (属性)) : A First Example (getattr 第一个示例)
- secondary trees (次级树) : The Inheritance Bifurcation (继承 Bifurcation)
- single-inheritance, slots (single-继承, slots (插槽)) : Example impacts of slots: ListTree and mapattrs (示例 impacts 的 slots (插槽) : ListTree 与 mapattrs)
- string representation methods (字符串表示: repr 和 str 方法) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str)
- subclassing (子类化) : Inherit, Customize, and Extend (Inherit, Customize, 与 Extend)
- subverting (颠覆/绕过) : Augmenting Methods: The Good Way (增强方法: 正确做法)
- super function (super 函数) : The super supplement (super 补充)
- inheritance algorithm (继承 algorithm) : Python Inheritance Algorithm: The Simple Version (Python 继承算法: 简单版) ; The Inheritance Wrap-Up (继承收束)
- inheritance hierarchy (继承 hierarchy) : OOP: The

Big Picture (OOP: 大图景)

- init method (init 方法)
- constructors (构造器) : Operator Overloading (运算符重载)
- operator overload and (运算符 overload 与) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)
- init.py files (init.py 文件) : Package init.py Files (包的 init.py 文件) ; Using the updated package (使用更新后的包) ; The Roles of init.py Files (init.py 文件的作用)
- from statement (from 语句) : The Roles of init.py Files (init.py 文件的作用)
- module namespace initialization (模块命名空间 initialization) : The Roles of init.py Files (init.py 文件的作用)
- package initialization (包 initialization) : The Role s of init.py Files (init.py 文件的作用)
- searches (搜索) : The Roles of init.py Files (init.py 文件的作用)
- inlining functions (inlining 函数) : Anonymous Functions: lambda (匿名函数: lambda)
- inputs, user (see user inputs) (用户输入 (见 user inputs))

- installation (A.3.1 安装：预装迷局与 python.org 安装器)
- Android (Android 系统) : Installing Python (安装 Python)
- Linux (Linux 系统) : Installing Python (安装 Python)
- macOS (macOS 系统) : Installing Python (安装 Python)
- Unix (Unix 系统) : Installing Python (安装 Python)
- Windows (Windows 系统) : Installing Python (安装 Python)
- installation management (A.3.1 安装：预装迷局与 python.org 安装器 management) : Development Tools for Larger Projects (更大型项目的开发工具)
- instance methods (实例方法) : Why the Special Methods? (Why Special 方法?) ; Using Static and Class Methods (Using 静态方法与类方法)
- metaclass comparison (元类 comparison) : Metaclass Methods Versus Instance Methods (元类方法 vs 实例方法)
- instance objects (实例对象) : Classes Generate Multiple Instance Objects (类生成多个实例对象) ; Instance Objects Are Concrete Items (实例对象是具体条目)

- attributes (属性) : Coding Constructors (编码构造器)
- instances (实例)
- attribute privacy (属性 privacy) : Emulating Privacy for Instance Attributes: Part 1 (模拟 Privacy 用于实例属性: Part 1)
- attributes (属性) : Instance Versus Class Attributes (实例对比类属性)
- class decorators (类装饰器) : Supporting multiple instances (Supporting multiple 实例) ; Class Pitfall: Retaining Multiple Instances (类陷阱: 保留多个实例)
- classes (类) : Why Use Classes? (为什么使用类?)
- counting (计数) : Counting Instances with Static Methods (Counting 实例带 Static 方法)
- class methods (类方法) : Counting Instances with Class Methods (Counting 实例带类方法) ; Counting instances per class with class methods (Counting 实例 per 类带类方法)
- creation (创建) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- indexing (索引)
- iteration (迭代) : Index Iteration: getitem (索引迭代: getitem)

- operator overloading (运算符重载) : Indexing and Slicing: (索引与切片: getitem 和 setitem:)
- literal syntax (literal 语法) : Classes Are Types Are Classes (类 Are 类型 Are 类)
- metaclasses (元类) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- nonclass instances (nonclass 实例) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- inheritance (继承) : The Inheritance Bifurcation (继承 Bifurcation)
- OOP (OOP (面向对象编程)) : Attribute Inheritance Search (属性继承搜索)
- namespaces (命名空间) : Classes and Instances (类与实例)
- state (状态) : Using State Information in Descriptors (在描述符中使用状态信息)
- state, decorators (状态与装饰器)
- subclasses, namespace dictionaries (子类, 命名空间字典) : Namespace Dictionaries: Review (命名空间字典: 回顾)
- integer keys (整数键) : Using dictionaries to simulate flexible lists: Integer keys (Using 字典到 simulate flexible 列表: 整数键)

- integers (整数) : Numbers (数字) ; Test Your Knowledge: Answers (测验答案) ; Types (类型)
- files (文件) : Core Types Review and Summary (核心类型回顾与总结)
- index method (index 方法) : But index Means As-Integer (But index Means As-整数)
- precision in large numbers (precision 中的 large 数字) : Numbers (数字)
- interaction (交互) : Part I, Getting Started (第 I 部分: 入门)
- interactive coding (交互式编程)
- \$ character (\$ 字符) : What Not to Type: Prompts and Comments (不要键入的内容: 提示符与注释)
- Android (Android 系统) : Starting an Interactive REPL (启动交互式 REPL)
- command line (命令行) : Test Your Knowledge: Answers (测验答案)
- comments (注释) : What Not to Type: Prompts and Comments (不要键入的内容: 提示符与注释)
- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- macOS (macOS 系统) : Starting an Interactive REPL (启动交互式 REPL)
- prompt (提示符) : Why the Interactive Prompt? (

为什么用交互式提示符？)

- prompts (提示符) : What Not to Type: Prompts and Comments (不要键入的内容: 提示符与注释)
- REPLs (real-eval-print loops) (REPLs (real-eval-print 循环))
- code folders (代码目录) : Where to Run: Code Folders (Where 到 Run: 代码目录)
- IPython (IPython 增强交互环境) : Other Python REPLs (其他 Python REPL)
- starting (启动) : Starting an Interactive REPL (启动交互式 REPL)
- Test Your Knowledge: Answers (测验答案)
- running code (运行 code) : Running Code Interactively (交互式运行代码)
- Windows (Windows 系统) : Starting an Interactive REPL (启动交互式 REPL)
- interactive command-line (交互式命令行-)
- REPLs (real-eval-print loops) (REPLs (real-eval-print 循环)) : Starting an Interactive REPL (启动交互式 REPL)
- interactive loops (交互式循环) : A Simple Interactive Loop (简单交互循环)
- error handling (错误处理)
- input testing (input 测试) : Handling Errors by Te

sting Inputs (Handling Errors by 测试 Inputs)

- try statements (try 语句) : Handling Errors with try Statements (Handling Errors 带 try 语句)
- floating-point numbers (浮点- 数字) : Supporting Floating-Point Numbers (Supporting 浮点- 数字)
- math on user inputs (对用户输入做数学运算) : Doing Math on User Inputs (对用户输入做数学运算)
- nesting code (嵌套代码) : Nesting Code Three Levels Deep (三层嵌套代码)
- interactive tests (交互式 tests) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- from statement, reload (from 语句, reload)
- reload statement, reload (reload 语句, reload)

J (字母 J)

- Java (Java 语言) : Component Integration (组件集成)
- JIT compilers (JIT 编译器)
- Numba (Numba JIT 加速) : Python Implementation Alternatives (Python 实现替代方案)
- PyPy (PyPy 实现) : Python Implementation Alternatives (Python 实现替代方案)
- join method, "Changing" (join 方法, "Changing") : Strings (字符串)

- Jython (Jython (Java 上的 Python)) : Python Implementation Alternatives (Python 实现替代方案)
- Jython (Jython (Java 上的 Python)) : Component Integration (组件集成) ; Python Implementation Alternatives (Python 实现替代方案)

K (字母 K)

- keys (键)
- dictionaries (字典) : Dictionaries (字典)
- insertion order (插入顺序) : Key Insertion Ordering (键 Insertion Ordering)
- integer keys (整数键) : Using dictionaries to simulate flexible lists: Integer keys (Using 字典到 simulate flexible 列表: 整数键)
- mapping values to (映射值到) : Mapping values to keys (映射值到键)
- missing (缺失) : Avoiding missing-key errors (避免键缺失错误)
- sorting (排序) : Sorting dictionary keys (Sorting 字典键)
- tuple keys (元组键) : Using dictionaries for sparse data structures: Tuple keys (Using 字典用于 sparse data structures: 元组键)
- keys method (键方法) : Item Iteration: for Loops (按项迭代: for 循环)

- keyword arguments (keyword 参数)
- list order (列表 order) : Sorting lists (Sorting 列表)
- print operations (print 操作) : The print function in action (print 函数中的 action)
- keyword-only arguments (仅关键字参数) : Keyword-Only Arguments (仅关键字参数) ; Why keyword-only arguments? (为什么需要仅关键字参数?)
- keywords (关键字) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- argument matching (参数匹配) : Argument Matching Overview (参数匹配概览) ; Argument Matching Syntax (参数匹配语法) ; Test Your Knowledge: Answers (测验答案)
- argument passing (参数传递) : Keyword and Default Examples (关键字与默认值示例) ; Combining keywords and defaults (组合关键字与默认值)
- function validation (函数校验) : Generalizing for Keywords and Defaults (泛化: 支持关键字与默认值)
- required (必需) : Keyword-Only Arguments (仅关键字参数)
- Kivy (Kivy 跨平台 UI 框架) : GUIs and UIs (GUI 与 UI) ; Standalone Apps and Executables (A.7 独立应用与可执行程序)

L (字母 L)

- lambda expression (lambda 表达式) : Basic Function Tools (基本函数工具) ; lambda Makes Anonymous Functions (lambda 创建匿名函数) ; lambda Basics (lambda 基础)
- argument-matching syntax (参数匹配语法-) : lambda Basics (lambda 基础)
- callbacks (回调) : Test Your Knowledge: Answers (测验答案)
- code proximity and (code proximity 与) : Multiway branches: The finale (多路分支: 终章)
- loops in (循环中的) : How (Not) to Obfuscate Your Python Code (How (Not) 到 Obfuscate Your Python Code)
- name assignments (名字赋值总论) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- names (名字) : Python Scopes Basics (Python 作用域基础)
- nesting (嵌套) : Scopes: lambdas Can Be Nested Too (作用域: lambdas Can Be 嵌套 Too)
- nesting logic (嵌套逻辑) : How (Not) to Obfuscate Your Python Code (How (Not) 到 Obfuscate Your Python Code)

- reasons to use (reasons 到用法) : Why Use lambda? (Why 用法 lambda?) ; Multiway branches: The finale (多路分支: 终章)
- selection logic (选择逻辑) : How (Not) to Obfuscate Your Python Code (How (Not) 到 Obfuscate Your Python Code)
- lambda functions (lambda 函数)
- callbacks (回调) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- wrapper layers (wrapper 层) : Decorator Nesting (装饰器嵌套)
- Latin-1 (Latin-1 编码) : Character Encodings (字符编码)
- learning curve (学习曲线) : It's Relatively Easy to Learn (It's Relatively Easy 到 Learn)
- LEGB rule (LEGB 规则) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则) ; Preview: Other Python scopes (预览: Python 的其他作用域)
- Nested Classes: The LEGB Scopes Rule Revisited-Nested Classes: The LEGB (嵌套类: LEGB 作用域规则 Revisited-嵌套类: LEGB)
- Scopes Rule Revisited (作用域规则 Revisited)
- built-ins (内置) : Redefining built-in names: For better or worse (Redefining 内置- 名字: 用于 better 或 worse)

- len function (len 函数) : Escape Sequences Are Special Characters (转义序列是特殊字符) ; Test Your Knowledge: Answers (测验答案)
- sequence reordering (序列 reordering) : Sequence Shufflers: range and len (序列 Shufflers: range 与 len)
- lexical scoping (词法作用域) : Python Scopes Basics (Python 作用域基础) ; Imports Versus Scopes (导入与作用域)
- libraries (库) : Why Do People Use Python? (为什么人们使用 Python?)
- utilities (工具) : It's Powerful (它很强大)
- like-a relationships (“像一个” 关系) : OOP and Delegation: “Like-a” Relationships (OOP 与委托: “像一个” 关系)
- lines comparison, text files (行比较, text 文件) : The Termination-Handlers Shoot-Out (终止处理器巅峰对决)
- Linux (Linux 系统)
- Python installation (Python 安装) : Installing Python (安装 Python)
- Python on (上的 Python) : Using Python on Linux (A.4 在 Linux 上使用 Python)
- list comprehension pattern (列表 comprehension pattern)

- best use (best 用法)
- to use list comprehensions: Speed, conciseness, etc. (到用法列表推导式: 速度, conciseness, etc.)
- code obfuscation and (代码混淆与) : When not to use list comprehensions: Code obfuscation (何时不要用列表推导式: 代码混淆)
- diagonals (对角线) : Example: List Comprehensions and Matrixes (示例: 列表推导式与矩阵)
- expression mapping (表达式映射) : List Comprehensions Review (列表推导式回顾)
- expressions (表达式) : Comprehensions: The Final Act (推导式: 最后一幕)
- arbitrary (任意) : List Comprehensions Review (列表推导式回顾)
- files (文件) : List Comprehensions and Files (列表推导式与文件)
- filter function (filter 函数) : Generator expressions versus filter (生成器表达式与 filter 的对比)
- for clauses, nested (用于 clauses, 嵌套) : Formal Comprehension Syntax (推导式的正式语法)
- if clause (if 子句) : Filter clauses: if (过滤子句: if) ; List Comprehensions Review (列表推导式回顾)
- iteration (迭代) : Formal Comprehension Syntax (推导式的正式语法)

- map function (map 函数) : List Comprehensions Review (列表推导式回顾)
- matrixes and (matrixes 与) : Example: List Comprehensions and Matrixes (示例: 列表推导式与矩阵)
- list comprehensions: Speed, conciseness, etc. (列表推导式: 速度, conciseness, etc.)
- nested for loops (嵌套 for 循环) : Nested loops: for (嵌套循环: 用于)
- sequence operations (序列操作) : Conversions, methods, and immutability (转换、方法与不可变性)
- syntax, formal (语法, formal) : Formal Comprehension Syntax (推导式的正式语法)
- lists (列表) : Why Use Built-in Objects? (为什么使用内置对象?) ; Test Your Knowledge: Answers (测验答案) ; Types (类型)
- as literal expression (as literal 表达式) : Lists (列表)
- assignments (赋值总论) : Assignment Syntax Forms (赋值语法形式)
- bounds (边界) : Bounds Checking (边界检查)
- breadth-first traversal (广度优先遍历) : Recursion versus queues and stacks (Recursion 对比 queues 与 stacks)
- building (构建) : Test Your Knowledge: Answers (测验答案)

- characteristics (特征) : Lists (列表)
- compared to tuples (compared 到元组) : Why Lists and Tuples? (为什么需要列表和元组?)
- comparisons (比较运算: lt、gt 等) : Basic List Operations (列表基本操作) ; Comparisons, Equality, and Truth (比较、相等与真值)
- comprehensions (推导式) : List comprehensions and maps (列表推导式与 maps) ; List comprehensions (列表推导式)
- deleting one item (deleting one 项) : More List Methods (More 列表方法)
- dict attribute (dict 属性) : Example: Listing Modules with dict (示例: 用 dict 列出模块)
- dictionaries, sorting (字典, sorting) : Sorting lists (Sorting 列表)
- index assignments (索引赋值总论) : Index (索引)
- indexing (索引) : Indexing and Slicing (索引与切片: getitem 和 setitem)
- iteration (迭代) : Iteration, Comprehensions, and Unpacking (迭代, 推导式, 与 Unpacking)
- iteration protocol (迭代协议) : Reprise: Dictionaries, range, enumerate, and zip (重温: 字典、range、enumerate 与 zip)
- literals (字面量) : Lists (列表)

- unpacking (解包) : List-literal unpacking (列表-字面量解包)
- maps (映射) : List comprehensions and maps (列表推导式与 maps)
- method calls (方法调用) : List method calls (列表方法调用)
- mutability (可变性) : Lists (列表)
- nesting (嵌套) : Nesting (嵌套)
- operations (操作) : Lists (列表)
- operators (运算符) : Basic List Operations (列表基本操作)
- ordering (排序/次序) : Sorting lists (Sorting 列表)
- reversing (反转) : More List Methods (More 列表方法)
- searching (搜索中) : Test Your Knowledge: Answers (测验答案)
- sequence operations (序列操作) : Sequence Operations (序列操作)
- slice assignments (slice 赋值总论) : Index (索引)
- slicing (切片) : Indexing and Slicing (索引与切片 : getitem 和 setitem)
- type-specific operations (类型-specific 操作) : Type-Specific Operations (类型-Specific 操作)

- literal expressions (literal 表达式) : Python' s Core Object Types (Python 的核心对象类型)
- dictionaries (字典) : Dictionaries (字典)
- lists as (列表 as) : Lists (列表)
- literal patterns (字面量模式) : Advanced match Usage (match 高级用法)
- literals (字面量) : Python' s Core Object Types (Python 的核心对象类型)
- dictionaries (字典) : Dictionaries (字典)
- docstrings (文档字符串) : User-defined docstrings (用户定义的文档字符串)
- f-string formatting (f-字符串格式化) : The F-String Formatting Literal (F-String 格式化字面量)
- backslash (反斜杠) : F-string formatting basics (F-字符串格式化基础)
- custom formats (自定义格式) : F-string custom formats (F-字符串自定义格式)
- quotes (引号) : F-string formatting basics (F-字符串格式化基础)
- REPL (REPL (交互运行环境)) : F-string formatting basics (F-字符串格式化基础)
- substitutions (替换) : F-string formatting basics (F-字符串格式化基础)
- lists, unpacking (列表, unpacking) : List-literal un

packing (列表-字面量解包)

· string formatting (字符串格式化) : String-Formatting Options (字符串-格式化 Options)

· local scopes (局部作用域) : Scopes Overview (作用域概述)

· local variables (局部变量) : Segue: Local Variables (过渡: 局部变量)

· recursion (递归) : Scopes Overview (作用域概述)

· static locals (静态局部量) : Function Attributes (函数属性)

· logic alternatives (logic 替代方案) : Test Your Knowledge: Part III Exercises (第三部分练习)

· LookupError exception class (LookupError 异常类) : Built-in Exception Classes (内置异常类)

· loops (循环) : Part III, Statements and Syntax (第 III 部分, 语句与语法)

· attribute interception methods (属性 interception 方法) : Avoiding loops in attribute interception methods (避免属性拦截方法中的循环)

· coding (编码) : Test Your Knowledge: Part III Exercises (第三部分练习)

· comprehensions (推导式) : Preview: Other Python scopes (预览: Python 的其他作用域)

· counters (计数器)

- defaults (默认值) : Loops Require Defaults, Not Scopes (循环需要默认值, 而不是作用域)
- dictionary comprehensions (字典推导式) : Dictionary Comprehensions (字典推导式)
- else clause (else 子句) : Loop else (循环 else) ; Why the loop else? (为什么要有循环 else?)
- enumerate function (enumerate 函数) : Offsets and Items: enumerate (Offsets 与项: enumerate)
- for loops (for 循环) : Item Iteration: for Loops (按项迭代: for 循环) ; for Loops (for 循环)
- Common Coding Gotchas (常见编码陷阱)
- data types (data 类型) : Basic usage (基本用法)
- extended-unpacking assignments (扩展解包赋值) : Extended-unpacking assignment (扩展解包- assignment)
- in functions (中的函数) : Definition (定义)
- getitem (getitem (索引获取)) : Index Iteration: getitem (索引迭代: getitem)
- nested (嵌套) : Nested for loops (嵌套 for 循环) ; Nested loops: for (嵌套循环: 用于)
- sequence assignment (序列 assignment) : Tuple (sequence) assignment in for loops (元组 (序列) assignment 中的 for 循环)

M (字母 M)

- machine code (机器码) : Test Your Knowledge: Answers (测验答案)
- macOS (macOS 系统)
- interactive coding and (与交互式编程) : Starting an Interactive REPL (启动交互式 REPL)
- Python installation (Python 安装) : Installing Python (安装 Python)
- Python on (上的 Python) : Using Python on macOS (A.3 在 macOS 上使用 Python)
- magnitude comparisons (magnitude 比较运算: It、gt 等) : Truth Values Revisited (真值回顾)
- main attribute (main 属性) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- main.py files (main.py 文件) : Package main.py Files (包的 main.py 文件) ; Using the updated package (使用更新后的包)
- manager functions (manager 函数) : The Downside of “Helper” Functions (「辅助函数」的弊端)
- mangled names (mangled 名字) : Pseudoprivate Class Attributes (伪私有类属性)

- manual iteration (手动迭代) : Manual iteration (手动迭代)
- manuals, standard (标准手册) : The Standard Manuals (标准手册) ; Web Resources (网络资源)
- map function (map 函数) : Mapping Functions over Iterables: map (映射函数 over 可迭代对象: map)
- benchmarking (基准测试) : For more good times: Function calls and map (更多好戏: 函数调用与 map) ; For more (用于 more)
- good times: Function calls and map (good times: 函数调用与 map)
- emulating (模拟) : Example: Emulating zip and map (示例: 模拟 zip 和 map) ; Coding Your Own zip and 2.X map (自己实现 zip 与 2.X 的 map)
- iterables, multiple (可迭代对象, multiple) : Coding Your Own map (编码 Your Own map)
- list comprehension pattern (列表 comprehension pattern) : Coding Your Own map (编码 Your Own map)
- list comprehensions (列表推导式) : List Comprehensions Review (列表推导式回顾)
- nesting (嵌套) : Generator expressions versus map (生成器表达式与 map 的对比)
- versus generator expressions (对比生成器表达式) : Generator expressions versus map (生成器表达式)

与 map 的对比)

- mapping (映射) : Test Your Knowledge: Answers (测验答案)
- mutability (可变性) : Dictionaries (字典)
- operations (操作) : Mapping Operations (映射操作)
- mapping patterns (映射 patterns) : Advanced match Usage (match 高级用法)
- mappings (映射) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- match statement (match 语句) : Missing Keys: if Tests (Missing 键: if Tests)
- match statements (match 语句) : match Statements (match 语句)
- attribute patterns (属性 patterns) : Advanced match Usage (match 高级用法)
- if statements (if 语句) : Match versus if live (Match 对比 if live)
- instance patterns (实例 patterns) : Advanced match Usage (match 高级用法)
- literal patterns (字面量模式) : Advanced match Usage (match 高级用法)
- mapping patterns (映射 patterns) : Advanced match Usage (match 高级用法)

- nested patterns (嵌套 patterns) : Advanced match Usage (match 高级用法)
- nesting (嵌套) : Basic match Usage (match 基本用法)
- parentheses (圆括号) : Advanced match Usage (match 高级用法)
- sequence patterns (序列 patterns) : Advanced match Usage (match 高级用法)
- syntax (语法) : Python Syntax Revisited (Python 语法回顾)
- blocks (代码块) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- delimiters (分隔符) : Statement Delimiters: Lines and Continuations (语句 Delimiters: Lines 与 Continuations)
- indentation (缩进) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- special syntax (special 语法) : Special Syntax Cases in Action (Special 语法 Cases 中的 Action)
- math module (math 模块) : Numbers (数字)
- mathematical operations (mathematical 操作) : Numbers (数字)

- user inputs (用户输入) : Doing Math on User Inputs (对用户输入做数学运算)
- matplotlib (matplotlib 绘图库) : Numeric and Scientific Programming (Numeric 与科学计算编程)
- matrixes (矩阵) : list comprehensions (列表推导式) ; Example: List Comprehensions and Matrixes (示例: 列表推导式与矩阵)
- generators (生成器) : Why generators here: Space, time, and more (Why 生成器 here: Space, time, 与 more)
- object memory (对象内存) : Nesting Revisited (再谈嵌套)
- metaclass/class dichotomy (元类/类 dichotomy) : The Metaclass/Class Dichotomy (元类/类 Dichotomy)
- metaclasses (元类) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等") ; Coding Metaclasses (编写元类)
- attributes (属性) : Inheritance: The Finale (继承: 终章)
- class creation (类 creation)
- class decorators comparison (类装饰器 comparison) : Metaclasses Versus Class Decorators: Round 1 (元类 vs 类装饰器: 第一回合)

- class statements (类语句) : Adding methods to classes (向类添加方法)
- types (类型) : Class Statements Call a type (class 语句会调用 type) ; Class Statements Can Choose a type (class 语句可选择 type)
- construction (构造) : Customizing Construction and Initialization (定制构造与初始化)
- decorating class methods (decorating 类方法) : Automatically decorating class methods (自动装饰类方法)
- factory functions (工厂函数) : Using simple factory functions (使用简单工厂函数)
- inheritance (继承) : The Inheritance Bifurcation (继承 Bifurcation)
- Customizing Construction and Initialization (定制构造与初始化) : Inheritance: The Finale (继承: 终章)
- Inheritance: The Finale (继承: 终章) : Metaclass Inheritance (元类继承)
- names acquisition (名字 acquisition) : Metaclass Versus Superclass (元类 vs 超类)
- inheritance relationships (继承 relationships) : Inheritance: The Finale (继承: 终章)
- method protocol (方法协议) : Metaclass Method Protocol (元类方法协议)

- methods (方法)
- adding (添加) : Adding methods to classes (向类添加方法)
- class methods comparison (类方法 comparison) : Metaclass Methods Versus Class Methods (元类方法 vs 类方法)
- instance methods comparison (实例方法 comparison) : Metaclass Methods (元类方法)
- operator overloading in (运算符重载中的) : Operator Overloading in Metaclass Methods (元类方法中的运算符重载)
- nonclass instances (nonclass 实例) : Overloading class creation calls with normal classes (用普通类重载类创建调用)
- superclasses comparison (超类 comparison) : Metaclass Versus Superclass (元类 vs 超类)
- types (类型) : Classes Are Instances of type (类是 type 的实例)
- subclasses (子类) : Metaclasses Are Subclasses of type (元类是 type 的子类)
- when to use (when 到用法) : To Metaclass or Not to Metaclass (用不用元类)
- metafunctions (元函数) : Function Decorator Basics (函数装饰器基础)

- function decorators (函数装饰器)
- lists (列表) : List method calls (列表方法调用)
- syntax (语法) : Method Call Syntax (方法 Call 语法)
- method calls (OOP) (方法调用 (OOP (面向对象编程))) : Method Calls (方法调用)
- method objects (方法对象) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Bound Methods in Action (绑定方法实战)
- bound methods (bound 方法) : Method Objects: Bound or Not (方法对象: 绑定与否)
- plain functions (plain 函数) : Method Objects: Bound or Not (方法对象: 绑定与否)
- methods (方法)
- attribute fetches (属性 fetches) : Method Call Syntax (方法 Call 语法)
- bool (bool (真值测试)) : Boolean Tests: bool and len (布尔测试: bool 和 len: bool 与 len)
- bound methods (bound 方法) : Why the Special Methods? (Why Special 方法?)
- byte strings (字节字符串) : Methods (方法)
- call expressions (调用表达式: call) : Method Call Syntax (方法 Call 语法)
- call (call (可调用)) : Call Expressions: call (调用)

表达式: call: call) ; Function Interfaces and Callback- Based Code (函数 Interfaces 与回调- Based Code)

- class methods (类方法) : Why the Special Methods? (Why Special 方法?) ; Using Static and Class Methods (Using 静态方法与类方法)

- Using Static and Class Methods (Using 静态方法与类方法)

- classes (类) : Static and Class Methods (静态方法与类方法)

- adding to (adding 到) : Adding methods to classes (向类添加方法)

- method calls (方法调用) : Methods (方法) ; Other Method-Call Possibilities (其他方法调用可能)

- contains (contains (成员测试)) : Membership: contains, iter, and getitem (成员测试: contains、iter 与 getitem: contains, iter, 与 getitem)

- decorators (装饰器) : Supporting method decoration (支持方法装饰) ; Class Pitfall: Decorating Methods (类陷阱: 装饰方法)

- descriptors (描述符) : Using descriptors to decorate methods (Using 描述符到 decorate 方法)

- nested functions (嵌套函数) : Using nested functions to decorate methods (Using 嵌套函数到 decorate 方法)

- del (del (析构)) : Object Destruction: del (对象销毁: del: del) ; Destructor Usage Notes (Destructor 用法 Notes)
- dictionaries (字典) : More Dictionary Methods (More 字典方法)
- double underscores (双下划线) : Classes Can Intercept Python Operators (类可以拦截 Python 运算符)
- encapsulation (封装) : Coding Methods (编码方法)
- find, "Changing" (find, "Changing") (find, "更改" (find, "更改")) : Strings (字符串)
- get (get (描述符获取)) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getattr (getattr (属性缺失钩子)) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合) ; Attribute Reference (属性引用) ; Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getattribute (getattribute (属性访问钩子)) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- getitem (getitem (索引获取)) : Intercepting Slices (拦截切片)
- gt (gt (大于)) : Comparisons: lt, gt, and Others (比较运算: lt、gt 等: lt, gt, 与 Others)

- iadd (iadd (就地加)) : In-Place Addition (中的-In-Place Addition)
- index (index (转整数索引)) : But index Means As-Integer (But index Means As-整数)
- inherited (继承的) : Specializing Inherited Methods (特化继承方法)
- instance (实例) : Why the Special Methods? (Why Special 方法?)
- instance methods (实例方法) : Using Static and Class Methods (Using 静态方法与类方法)
- iter (iter (迭代器)) : Iterable Objects: iter and next (可迭代对象: iter 和 next: iter 与 next) ; User-Defined Iterables (用户定义的可迭代对象) ; Classes versus generators (类对比生成器)
- join, "Changing" (join, "Changing") (join, "更改" (join, "更改")) : Strings (字符串)
- lt (lt (小于)) : Comparisons: lt, gt, and Others (比较运算: lt、gt 等: lt, gt, 与 Others)
- metaclasses (元类)
- next (next (下一个)) : Iterable Objects: iter and next (可迭代对象: iter 和 next: iter 与 next)
- OOP (OOP (面向对象编程)) : Coding Class Trees (编写类树)
- operator overloading (运算符重载) : Common Operator-Overloading Methods (常用运算符重载方法)

- plain-function (plain-函数) : Plain-Function Methods (Plain-函数方法)
- radd (radd (右侧加)) : Right-Side Addition (右侧加法)
- replace, "Changing" (replace, "Changing") (replace, "更改" (replace, "更改")) : Strings (字符串)
- repr (repr (正式表示)) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str)
- scopes (作用域) : Scopes in Methods and Classes (作用域中的方法与类)
- set (set (描述符设置)) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- setattr (setattr (属性设置钩子)) : Attribute Assignment and Deletion (属性赋值与删除) ; Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- split (拆分) : More String Methods: Parsing Text (More 字符串方法: 解析文本)
- standard-library manual (标准库- manual) : All String Methods (Today) (全部字符串方法 (今日))
- static (静态) : Other Method-Call Possibilities (其他方法调用可能) ; Static and Class Methods (静态方法与类方法) ; Using Static and Class Methods (Using 静态方法与类方法)

- str (str (字符串表示)) : Why Two Display Methods? (Why Two Display 方法?) ; Display Usage Notes (Display 用法 Notes)
- string formatting (字符串格式化) : String-Formatting Options (字符串-格式化 Options)
- format method (format 方法) : The String-Formatting Method (字符串格式化方法) ; Advanced formatting method examples (高级格式化方法示例)
- string methods (字符串方法) : String Methods (字符串方法)
- syntax (语法) : Method Call Syntax (方法 Call 语法)
- strings (字符串) : Type-Specific Methods (类型-Specific 方法)
- subclasses (子类) : Augmenting Methods: The Bad Way (增强方法: 糟糕做法) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- superclass (超类) : Specializing Inherited Methods (特化继承方法)
- tuples (元组) : Conversions, methods, and immutability (转换、方法与不可变性)
- setitem (setitem (索引设置)) : Intercepting Item Assignments (拦截项赋值)
- methods (OOP) (方法 (OOP (面向对象编程))) : A First Example (getattr 第一个示例)

- MicroPython (MicroPython 微控制器版) : Python Implementation Alternatives (Python 实现替代方案)
- Microsoft Store (Microsoft Store 应用商店) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- minimalism of Python (minimalism 的 Python) : Software Quality (软件质量)
- mins.py (mins.py 示例) : Example: The min Wake up Call (示例: min 唤醒测试) ; The Punch Line (点睛之笔)
- mix-in classes (mix-中的类)
- attribute lists (属性列表) : Example: "Mix-in" Attribute Listers (示例: "混入"式属性列出器)
- class trees (类 trees) : Listing attributes per object in class trees (在类树中按对象列出属性)
- dict (dict (属性字典)) : Listing instance attributes with dict (用 dict 列出实例属性)
- dir (dir (属性列表)) : Listing inherited attributes with dir (用 dir 列出继承属性)
- mixed-type comparisons (mixed-类型比较运算: It、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值)
- module packages (see packages) (模块包 (see 包))
- module search path (模块 search path) : The Module Search Path (模块搜索路径)

- built-in modules (内置- 模块) : The Module Search Path (模块搜索路径)
- configuring (配置) : Configuring the Search Path (配置搜索路径)
- file selection (文件 selection) : Module File Selection (模块文件的选择) ; Selection priorities (选择优先级)
- file sources (文件 sources) : Module sources (模块的来源)
- home directory (主目录) : Search-Path Components (搜索路径的组成部分)
- packages (包) : Path Outliers: Standalones and Packages (路径特例: 独立可执行程序与包) ; Packages and the Module Search Path (包与模块搜索路径)
- .pth path-file directory (.pth 路径文件目录) : Search-Path Components (搜索路径的组成部分)
- PYTHONPATH directory (PYTHONPATH 目录) : Search-Path Components (搜索路径的组成部分) ; Configuring the Search Path (配置搜索路径)
- site-packages directory (site-packages 目录) : Search-Path Components (搜索路径的组成部分)
- standalones (独立程序) : Path Outliers: Standalones and Packages (路径特例: 独立可执行程序与包)
- standard-library directories (标准库- 目录) : Search-Path Components (搜索路径的组成部分)

- sys.path list (sys.path 列表) : The sys.path List (sys.path 列表) ; Changing the module search path (Changing 模块搜索路径)
- modules (模块) : Program Files (程序文件) ; Part I, Getting Started (第 I 部分: 入门)
- attributes (属性) : Module attributes: a first look (模块属性: first look)
- Creating Modules (创建模块)
- classes (类) : Classes Are Attributes in Modules (类是模块中的属性)
- creating (创建) : How Files Generate Namespaces (文件如何生成命名空间)
- dict (dict (属性字典)) : Namespace Dictionaries: dict (命名空间字典: dict)
- names, qualification (名字, qualification) : Attribute Name Qualification (属性名限定)
- classtools (类工具) : Name Considerations in Tool Classes (名字 Considerations 中的工具类)
- code reuse (code 复用) : Why Use Modules? (为什么使用模块?)
- cohesion (内聚) : Module Design Concepts (模块设计概念)
- collector (收集器) : Part V, Modules and Packages (第 V 部分, 模块与包)

- coupling (耦合) : Module Design Concepts (模块设计概念)
- creating (创建) : Creating Modules (创建模块) ; Other Kinds of Modules (其他类型的模块)
- defining (定义) : Creating Modules (创建模块)
- designing (设计) : Module Design Concepts (模块设计概念)
- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- extension modules (扩展模块) : Other Kinds of Modules (其他类型的模块)
- file names (文件名字) : Module attributes: a first look (模块属性: first look)
- filenames (文件名) : Module Filenames (模块文件名)
- files (文件) : Program Files (程序文件)
- from statement (from 语句) : The from Statement (from 语句)
- from statement (from 语句) : Module Essentials (模块要点) ; The from Statement (from 语句)
- potential problems (潜在问题) : Potential Pitfalls of the from Statement (from 语句的潜在陷阱) ; When import is required (何时必须用 import)
- global scope (全局 scope) : Why Use Modules? (

为什么使用模块？)

- import statement (import 语句) : Module Essentials (模块要点) ; The import Statement (import 语句)
- importing (导入中) : Importing modules (Importing 模块) ; Test Your Knowledge: Answers (测验答案) ; Imports and Attributes (导入与属性) ; Step 1: Find It (第一步: 查找文件)
- bytecode (字节码) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- bytecode execution (字节码执行) : Step 3: Run It (第三步: 运行)
- previous imports (previous 导入) : Step 2: Compile It (Maybe) (第二步: 编译 (或许))
- importlib.reload (importlib.reload 重新加载) : Module Essentials (模块要点)
- imports (导入)
- direct calls (direct 调用) : Direct Calls: Two Options (Direct 调用: Two Options)
- initialization code (初始化代码) : Initialization code (初始化代码)
- name strings (名字字符串) : Importing Modules by Name String (按名字字符串导入) ; Testing reload variants (测试 reload 变体)

- namespace nesting (命名空间嵌套) : Namespace Nesting (命名空间嵌套)
- one-time (一次性) : Imports Happen Only Once (导入只发生一次)
- runtime assignment (运行时赋值) : Imports Are Runtime Assignments (导入是运行时赋值) ; Cross-file name changes (Cross-文件名字修改)
- statement order (语句 order) : Statement Order Matters in Top-Level Code (语句 Order Matters 中的顶层- Code)
- variables (变量) : Imports Versus Scopes (导入与作用域)
- json (JSON 格式) : The pickle and json Serialization Modules (pickle 与 JSON 序列化模块)
- listing, dict attribute (listing, dict 属性) : Example: Listing Modules with dict (示例: 用 dict 列出模块)
- main (main (主模块)) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- math module (math 模块) : Numbers (数字)
- mutables, changing (可变对象, changing) : Changing mutables in modules (在模块中修改可变对象)
- name (name (名字)) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)

- namespaces (命名空间) : Why Use Modules? (为什么使用模块?)
- dictionaries (字典) : How Files Generate Namespaces (文件如何生成命名空间) ; Namespace Dictionaries: dict (命名空间字典: dict)
- generating (生成) : How Files Generate Namespaces (文件如何生成命名空间)
- initialization (初始化) : The Roles of init.py Files (init.py 文件的作用)
- nesting (嵌套) : Namespace Nesting (命名空间嵌套)
- partitioning (分区) : Why Use Modules? (为什么使用模块?)
- nested, attributes (嵌套, 属性) : Using the basic package (使用基础包)
- pickle (pickle (序列化)) : Pickles and Shelves (Pickles 与 Shelves) ; The pickle and json Serialization Modules (pickle 与 JSON 序列化模块)
- program architecture (程序架构) : Importing modules (Importing 模块) ; How to Structure a Program (如何组织一个程序)
- random (随机) : Numbers (数字)
- re (re (正则模块)) : The re Pattern-Matching Module (re Pattern-Matching 模块)

- redundancy (冗余) : Why Use Modules? (为什么使用模块?)
- reload function (reload 函数) : Reloading Modules (重载模块) ; reload Odds and Ends (reload Odds 与 Ends)
- reloads (多次重新加载) : Reloading modules (重载模块) ; Example: Transitive Module Reloads (示例 : 传递式模块重载)
- recursive reloaders (递归重新加载器) : A recursive reloader (递归重新加载器) ; Testing reload variants (测试 reload 变体)
- scopes (作用域)
- imports and (导入与) : How Files Generate Name spaces (文件如何生成命名空间)
- variables (变量) : Imports Versus Scopes (导入与作用域)
- shelve (shelve (持久字典)) : Pickles and Shelves (Pickles 与 Shelves) ; The shelve module (shelve 模块)
- standard-library modules (标准库模块) : Standard-Library Modules (标准库模块)
- statements (语句) : The Python Conceptual Hierarchy (Python 概念层级)
- import (import (导入)) : How Files Generate Namespaces (文件如何生成命名空间)

- strings (字符串) : String Object Basics (字符串对象基础)
- struct (struct (结构体模块)) : The struct Binary-Data Module (struct 二进制数据- 模块)
- using (使用) : Using Modules (使用模块) ; When import is required (何时必须用 import)
- versus classes (对比类) : Classes Versus Modules (类与模块)
- MRO (method resolution order) (MRO (方法 resolution order)) : Multiple Inheritance and the MRO (多重继承与 MRO) ; How the MRO Works (MRO 如何工作)
- attribute fetch (属性 fetch) : Attribute-fetch algorithm (属性获取算法)
- class inheritance (类继承) : Universal deployment (通用部署)
- class trees (类 trees) : Python Inheritance Algorithm: The Simple Version (Python 继承算法: 简单版) ; Python (Python 语言)
- Inheritance Algorithm: The Less Simple Version (继承 Algorithm: Less Simple Version)
- deployment (部署) : Universal deployment (通用部署)
- as flattened tree (作为展平树) : The Inheritance Bifurcation (继承 Bifurcation)

- inheritance (继承) : The Inheritance Bifurcation (继承 Bifurcation)
- super function (super 函数) : A “magic” proxy (“魔法” 代理)
- mro attribute (mro 属性) : How the MRO Works (MRO 如何工作) ; Example: Mapping Attributes to Inheritance Sources (示例: 将属性映射到继承来源) ; The Inheritance Bifurcation (继承 Bifurcation) ; Attribute-fetch algorithm (属性获取算法)
- Metaclass Versus Superclass (元类 vs 超类)
- multiline statements (multiline 语句) : Running Code Interactively (交互式运行代码)
- multiline strings (multiline 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- multiline text, triple-quoted strings (multiline text, triple-quoted 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- multiple inheritance (多重继承) : Multiple Inheritance and the MRO (多重继承与 MRO)
- attribute conflicts (属性 conflicts) : Attribute Conflict Resolution (属性冲突的解决)
- DFLR (depth first, left to right) (DFLR (深度优先、从左到右)) : Multiple Inheritance and the MRO (多重继承与 MRO)

- inheritance sources (继承 sources) : Example: Mapping Attributes to Inheritance Sources (示例: 将属性映射到继承来源)
- mix-in classes, attribute lists (mix-中的类, 属性列表) : Example: "Mix-in" Attribute Listers (示例: "混入"式属性列出器) ; Listing attributes per object in class trees (在类树中按对象列出属性)
- MRO (method resolution order) (MRO (方法 resolution order)) : Multiple Inheritance and the MRO (多重继承与 MRO) ; How the MRO Works (MRO 如何工作)
- order (顺序) : Multiple Inheritance: Order Matters (多重继承: Order Matters) ; Multiple Inheritance: (多重继承:)
- overview (A.5.1 总览: Python 应用三剑客) : How Multiple Inheritance Works (多重继承如何工作)
- slots (slots (插槽)) : Slot usage rules (slot 用法规则)
- multiple-target assignments (多重目标赋值) : Assignment Syntax Forms (赋值语法形式) ; Multiple-Target Assignments (多重目标赋值)
- shared references (共享引用) : Multiple-target assignment and shared references (Multiple-target assignment 与共享引用)
- multiplication (乘法) : Numbers (数字)

- multitasking (多任务)
- dictionaries (字典) : Dictionaries (字典) ; Changing Dictionaries in Place (Changing 字典中的 Place)
- in-place changes (中的-place 修改) : Shared References and In-Place Changes (共享引用与原地修改)
- lists (列表) : Lists (列表)
- mutable defaults (可变默认值) : Combining keywords and defaults (组合关键字与默认值)
- objects, changing (对象, changing)
- strings (字符串) : Immutability (不可变性)
- mutable arguments (可变参数) : Argument-Passing Basics (参数传递基础)
- changes (修改) : Avoiding Mutable Argument Changes (避免可变参数被修改)
- coupling (耦合) : Function Design Concepts (函数设计概念)
- mutable objects (可变对象)
- changing (更改) : Changing mutables in modules (在模块中修改可变对象)
- functions (函数) : Defaults and Mutable Objects (默认参数与可变对象)
- mutable types (可变类型) : Mutable Types Can Be Changed in Place (可变类型 Can Be Changed 中的 Place)

- mutables, assignments (可变对象, 赋值总论) : Common Coding Gotchas (常见编码陷阱)

N (字母 N)

- name annotations, basic assignment (名字注解, 基本赋值) : Basic Assignments (基本赋值)
- name assignments, nested scopes (名字赋值总论, 嵌套作用域) : Nested Scopes Overview (嵌套作用域概述)
- name attribute (name 属性) : Dual-Usage Modes: name and main (双用途模式: name 与 main) ; Example: Unit Tests with name (示例: 用 name 做单元测试)
- name clashes (名字 clashes) : Potential Pitfalls of the from Statement (from 语句的潜在陷阱) ; The as Extension for import and from (import 与 from 的 as 扩展) ; Module Name Clashes: Package and Package-Relative Imports (模块名字 Clashes: 包与包-相对导入)
- name collisions, tool classes (名字 collisions, 工具类) : Name Considerations in Tool Classes (名字 Considerations 中的工具类)
- name mangling (名字 mangling) : Pseudoprivate Class Attributes (伪私有类属性)
- name references, nested scopes (名字引用与嵌套作

- 用域) : Nested Scopes Overview (嵌套作用域概述)
- name strings, module imports (名字字符串, 模块导入) : Importing Modules by Name String (按名字字符串导入)
- code strings, running (code 字符串, 运行) : Running Code Strings (运行 Code 字符串)
- direct calls (direct 调用) : Direct Calls: Two Options (Direct 调用: Two Options)
- reloads (多次重新加载) : Example: Transitive Module Reloads (示例: 传递式模块重载) ; Testing reload variants (测试 reload 变体)
- named assignment expression (命名赋值 (海象) 表达式) : Assignment Syntax Forms (赋值语法形式) ; Named Assignment Expressions (命名赋值表达式) ; When to use named assignment (When 到用法命名赋值 (海象))
- named tuples (named 元组) : Records Revisited: Named Tuples (再看记录: 命名元组)
- names (名字)
- aliasing (别名 (共享引用)) : Arguments and Shared References (参数与共享引用)
- assignments (赋值总论) : Assignments (赋值总论) ; Scopes Overview (作用域概述) ; Name Resolution: The LEGB Rule (名字解析: LEGB 规则)

- def statement (def 语句) : Python Scopes Basics (Python 作用域基础)
- def statements (def 语句) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- global statements (全局语句) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- in-place changes (中的-place 修改) : Scopes Overview (作用域概述)
- lambda expressions (lambda 表达式) : Python Scopes Basics (Python 作用域基础)
- lambda statements (lambda 语句) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- LEGB rule (LEGB 规则) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则) ; Preview: Other Python scopes (预览: Python 的其他作用域)
- local (局部) : Scopes Overview (作用域概述)
- nonlocal statements (nonlocal 语句) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- references (引用) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- variables (变量) : Module Filenames (模块文件名)
- namespace packages (命名空间包) : Module Packages (模块包) ; Using the updated package (使用更新后的包) ; Namespace Packages (命名空间包)

- module searches (模块 searches) : The Module Search Algorithm (模块搜索算法)
- namespaces (命名空间) : Python Scopes Basics (Python 作用域基础) ; Why Use Modules? (为什么使用模块?) ; Testing as You Go (测试 as You Go)
- assignments (赋值总论) : The “Zen” of Namespaces: Assignments Classify Names (命名空间的“禅意” : 赋值给名字分类)
- classes (类)
- hierarchies (层次结构) : Why Use Classes? (为什么使用类?)
- inheritance (继承) : Example: Class Attributes (示例: 类属性)
- dictionaries (字典) : Classes: Under the Hood (类 : 揭开引擎盖) ; Namespace Dictionaries: Review (命名空间字典: 回顾)
- slots and (slots (插槽) 与) : Slot basics (slot 基础) ; Slots and namespace dictionaries (slots (插槽) 与命名空间字典)
- generating (生成) : How Files Generate Namespaces (文件如何生成命名空间)
- global (全局) : Simple Names: Global Unless Assigned (简单名字: 除非赋值否则全局)
- global statements (全局语句) : The global Statement (global 语句)

- links (链接) : Namespace Links: A Tree Climber (命名空间链接: 爬树工具)
- modules (模块)
- dictionaries (字典) : How Files Generate Namespaces (文件如何生成命名空间) ; Namespace Dictionaries: dict (命名空间字典: dict)
- init.py files (init.py 文件) : The Roles of init.py Files (init.py 文件的作用)
- nesting (嵌套) : Namespace Nesting (命名空间嵌套)
- partitioning (分区) : Why Use Modules? (为什么使用模块?)
- namespace trees (命名空间 trees) : Attribute Tree Construction (属性树构建)
- object namespaces (对象命名空间) : Attribute Names: Object Namespaces (属性名: 对象命名空间)
- OOP (OOP (面向对象编程)) : Classes and Instances (类与实例)
- naming variables (naming 变量) : Variable Name Rules (变量命名规则) ; Names have no type, but objects do (名字没有类型, 但对象有)
- nested calls (嵌套调用) : Numbers (数字)
- nested modules (嵌套模块) : Using the basic package (使用基础包)

- nested patterns (嵌套 patterns) : match statements (match 语句) ; Advanced match Usage (match 高级用法)
- nested scopes (嵌套作用域) : Nested Scopes Examples (嵌套作用域示例)
- arbitrary nesting (任意嵌套) : Arbitrary Scope Nesting (任意深度的作用域嵌套)
- name assignments (名字赋值总论) : Nested Scopes Overview (嵌套作用域概述)
- name references (名字引用) : Nested Scopes Overview (嵌套作用域概述)
- nested statement blocks (嵌套语句 blocks) : What Python Adds (Python 增加了什么)
- special cases (特殊情况) : Block rule special case (块规则特例)
- nesting (嵌套) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; Part II, Objects and Operations (第 II 部分, 对象与操作)
- assigning nested sequences (assigning 嵌套序列) : Advanced sequence-assignment patterns (高级序列赋值模式)
- classes, scope (类, scope) : Nested Classes: The LEGB Scopes Rule Revisited (嵌套类: 重温 LEGB 作用域规则)

- combined clauses (组合子句)
- composites (复合) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- decorators (装饰器) : Decorator Nesting (装饰器嵌套) ; Decorator nesting (装饰器嵌套)
- descriptors (描述符) : A First Example (getattr 第一个示例)
- dictionaries (字典) : Nesting Revisited (再谈嵌套) ; Nesting in dictionaries (嵌套中的字典)
- exception groups (异常 groups) : Example: Control-Flow Nesting (示例: 控制流嵌套)
- exception handlers (异常 handlers) : Nesting Exception Handlers (嵌套异常处理器)
- control-flow (控制流) : Example: Control-Flow Nesting (示例: 控制流嵌套)
- syntactic nesting (语法嵌套) : Example: Syntactic Nesting (示例: 语法嵌套)
- for loops (for 循环) : Nested for loops (嵌套 for 循环)
- functions, decorating methods (函数, decorating 方法) : Using nested functions to decorate methods (Using 嵌套函数到 decorate 方法)
- generator expressions (生成器表达式) : Generator expressions versus map (生成器表达式与 map 的对比)

- imports (导入) : Part V, Modules and Packages (第 V 部分, 模块与包)
- interactive loops (交互式循环) : Nesting Code Three Levels Deep (三层嵌套代码)
- iterators (迭代器) : Iterator nesting (迭代器嵌套)
- lambda expression (lambda 表达式) : Scopes: lambdas Can Be Nested Too (作用域: lambdas Can Be 嵌套 Too)
- loops, multiple (循环, multiple) : Breaking Out of Multiple Nested Loops: "go to" (跳出多层嵌套循环 : "goto")
- map function (map 函数) : Generator expressions versus map (生成器表达式与 map 的对比)
- match statements (match 语句) : Basic match Usage (match 基本用法)
- namespaces (命名空间)
- package folders (包文件夹) : Basic Package Structure (基本包结构)
- while loops (while 循环) : The nested-code alternative (嵌套代码替代方案)
- nesting objects (嵌套对象) : Nesting (嵌套)
- .NET Framework for Windows (Windows 上的 .NET Framework)
- inheritance (继承) : The Inheritance Bifurcation (

继承 Bifurcation)

- metaclasses (元类) : Overloading class creation calls with normal classes (用普通类重载类创建调用)
- None object (None 对象) : Booleans and None (布尔与 None) ; The None object (None 对象)
- nonempty data structures (非空数据结构) : The Meaning of True and False in Python (Python 中 True 与 False 的含义)
- nonlocal statement (nonlocal 语句) : Advanced Function Tools (高级函数工具) ; The nonlocal Statement (nonlocal 语句) ; nonlocal Boundary Cases (nonlocal 边界情形)
- state retention and (状态 retention 与) : Nonlocal s: Changeable, Per-Call, LEGB (nonlocal: 可修改、每调用独立、LEGB)
- normalization (规范化) : Unicode, Unicode Normalization: Whither Standard?-Unicode Normalization: Whither Standard? Notepad, BOM (byte order marker), Making BOMs in Text Editors-Making BOMs in Text Editors Nuitka, Python Implementation Alternatives NULL character, Escape Sequences Are Special Characters Numba, Numeric and Scientific Programming, Python Implementation Alternatives, Test Your Knowledge: Answers (Unicode, Unicode Normalization: Whither Standard?-Unicode Normalization: W

hither Standard? Notepad, BOM (字节 order marker), Making BOMs 中的文本编辑器 -Making BOMs 中的文本编辑器 Nuitka, Python 实现替代方案 NULL 字符, 转义序列是特殊字符 Numba, Numeric 与科学计算编程, Python 实现替代方案, 知识测验: Answers)

- Alternatives (替代方案) : Test Your Knowledge: Answers (测验答案)
- number types, files (数字类型, 文件) : Core Types Review and Summary (核心类型回顾与总结)
- numbers (数字) : Test Your Knowledge: Answers (测验答案)
- comparisons (比较运算: lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值)
- complex (复数/复杂) : Numbers (数字)
- decimals (十进制数) : Numbers (数字) ; Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- floating-point (浮点) : Numbers (数字) ; Test Your Knowledge: Answers (测验答案)
- floating-point numbers (浮点- 数字) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- fractions (分数) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)

- integers (整数) : Numbers (数字) ; Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- mathematical operations (mathematical 操作) : Numbers (数字)
- rationals (有理数) : Numbers (数字)
- sets (集合) : Numbers (数字)
- true and false (True 与 False) : The Meaning of True and False in Python (Python 中 True 与 False 的含义)
- types (类型) : Test Your Knowledge: Answers (测验答案)
- numeric programming (数值编程) : Numeric and Scientific Programming (Numeric 与科学计算编程)
- NumPy (NumPy 数值库) : Why Do People Use Python? (为什么人们使用 Python?) ; OK, but What' s the Downside? (好, 但缺点是什么?) ; Numeric and Scientific Programming (Numeric 与科学计算编程)

O (字母 O)

- obfuscation (混淆) : list comprehensions (列表推导式)
- as superclass (as 超类) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")

- inheritance type class (继承类型类) : And One "object" to Rule Them All (一个 "object" 统御万物)
- object factories (对象 factories) : Testing as You Go (测试 as You Go) ; Classes Are Objects: Generic Object Factories (类也是对象: 通用对象工厂) ; Why Factories? (为什么需要工厂?)
- object indexes, loops (对象 indexes, 循环) : Examples (示例)
- object memory (对象内存) : Nesting Revisited (再谈嵌套)
- object model (对象模型)
- classes, user-defined (类, 用户定义-) : Classes Are Types Are Classes (类 Are 类型 Are 类)
- instances, metaclasses (实例, 元类) : Some Instances Are More Equal Than Others (有些实例比其他实例更"平等")
- object namespaces (对象命名空间) : Attribute Names: Object Namespaces (属性名: 对象命名空间)
- object serialization (对象序列化) : Storing Objects with pickle (用 pickle 存储对象)
- object storage, conversions (对象 storage, 转换) : Storing Objects with Conversions (用转换来存储对象)
- object types (对象类型) : It' s Powerful (它很强大) ; Python' s Core Object Types (Python 的核心对

- 象类型) ; Test Your Knowledge: Answers (测验答案)
- built-ins (内置)
 - cyclic data structures (循环数据 structures) : Beware of Cyclic Data Structures (当心循环数据结构)
 - extending (扩展 (重复/连接)) : Extending Built-in Object Types (扩展内置对象类型) ; Extending Types by Subclassing (扩展 (重复/连接) 类型 by Subclassing)
 - immutable types (不可变类型) : Immutable Types Can't Be Changed in Place (不可变类型不能就地修改)
 - references (引用) : Assignment Creates References, Not Copies (赋值制造引用, 而非拷贝)
 - repetition (重复) : Repetition Adds One Level Deep (重复只加深一层)
 - hierarchy (层次) : Python's Type Hierarchies (Python 的类型体系)
 - object-based code (对象-based code) : OOP: The Big Picture (OOP: 大图景)
 - object-oriented language (对象-oriented language) : Is Python a "Scripting Language" ? (Python 是 "脚本语言"吗?) ; It's Object-Oriented and Functional (It's 对象-Oriented 与 Functional)

- object-oriented, definition (对象-oriented, 定义)
- User-Defined Objects (用户定义- 对象)
- object-relational mappers (对象-relational mappers) : Internet and Web Scripting (互联网与 Web 脚本)
- objects (对象) : Introducing Python (介绍 Python)
- aggregation (聚合) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- attributes (属性) : The dir Function (dir 函数)
- name strings (名字字符串) : The dir Function (dir 函数)
- type names (类型名字) : The dir Function (dir 函数)
- Booleans (布尔) : Booleans and None (布尔与 None)
- built-in (内置) : Why Use Built-in Objects? (为什么使用内置对象?)
- bytearray (bytearray (字节数组)) : The bytearray Object (bytearray 对象)
- bytes (字节串) : Unicode and Byte (Unicode 与字节) ; Strings (字符串) ; The bytes Object (bytes 对象)
- calling, core object types (calling, core 对象类型) : Python' s Core Object Types (Python 的核心对象)

类型)

- categories (类别) : Core Types Review and Summary (核心类型回顾与总结)
- class objects (类对象) : Attribute Inheritance Search (属性继承搜索) ; Classes Generate Multiple Instance Objects (类生成多个实例对象)
- default behavior (默认行为) : Class Objects Provide Default Behavior (类对象提供默认行为)
- classes, docstrings (类, 文档字符串) : User-defined docstrings (用户定义的文档字符串)
- comparisons (比较运算: lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值) ; Dictionary comparisons (字典比较)
- core object types (core 对象类型) : Comparisons, Equality, and Truth (比较、相等与真值)
- copies (拷贝) : References Versus Copies (引用与拷贝)
- copying (拷贝) : Shared References and In-Place Changes (共享引用与原地修改)
- databases, storage (数据库存储)
- pickle module (pickle 模块) : Pickles and Shelves (Pickles 与 Shelves)
- shelf object update (shelf 对象 update) : Updating Objects on a Shelf (Updating 对象 on Shelf)

- shelf database (shelf 数据库) : Storing Objects on a shelf Database (Storing 对象 on shelf 数据库) ; Exploring Shelves Interactively (交互式探索 shelf)
- shelf module (shelf 模块) : Pickles and Shelves (Pickles 与 Shelves) ; The shelf module (shelf 模块)
- dynamic typing (动态类型) : Variables, Objects, and References (变量、对象与引用)
- embedding (嵌入) : Other Ways to Combine Classes: Composites (组合类的其他方式: 复合)
- expressions (表达式) : The Python Conceptual Hierarchy (Python 概念层级)
- file objects (文件对象) : Files (文件)
- first-class object model (first-类对象模型) : The First-Class Object Model (一等对象模型)
- flexibility (灵活性) : Object Flexibility (对象灵活性)
- garbage collection and (垃圾回收与) : Objects Are Garbage-Collected (对象会被垃圾回收)
- generator objects (生成器对象) : Generator Expressions: Iterables Meet Comprehensions (生成器表达式: 可迭代对象遇见推导式)
- Generator Functions Versus Generator Expressions (生成器函数与生成器表达式对比) : Coding Alternati

- ve: iter Plus yield (编码替代: iter 加 yield) ; Multiple iterators with yield (用 yield 实现多个迭代器)
- implementation-related (实现-related) : Python's Core Object Types (Python 的核心对象类型)
- in-place changes (中的-place 修改) : Shared References and In-Place Changes (共享引用与原地修改)
- instance objects (实例对象) : Attribute Inheritance Search (属性继承搜索) ; Classes Generate Multiple Instance Objects (类生成多个实例对象) ; Instance Objects Are Concrete Items (实例对象是具体条目)
- attributes (属性) : Coding Constructors (编码构造器)
- iterable (see iterable objects) (可迭代对象 (see 可迭代对象: iter 和 next))
- method objects (方法对象) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Bound Methods in Action (绑定方法实战)
- bound methods (bound 方法) : Method Objects: Bound or Not (方法对象: 绑定与否)
- plain functions (plain 函数) : Method Objects: Bound or Not (方法对象: 绑定与否)
- mutable (可变)
- changing in place (changing 中的 place) : Core Types Review and Summary (核心类型回顾与总结)

- functions (函数) : Defaults and Mutable Objects (默认参数与可变对象)
- namespace objects (命名空间对象) : Testing as You Go (测试 as You Go)
- nesting (嵌套) : Nesting (嵌套)
- None (None (空值)) : Booleans and None (布尔与 None) ; The None object (None 对象)
- operation sharing (操作 sharing) : Core Types Review and Summary (核心类型回顾与总结)
- program units (程序 units) : Python' s Core Object Types (Python 的核心对象类型)
- properties (properties (属性))
- Pydoc (Pydoc 文档工具) : Getting Help (获取帮助)
- reference counts (引用 counts) : Nesting Revisited (再谈嵌套)
- references (引用) : References Versus Copies (引用与拷贝)
- assignments (赋值总论) : Assignments (赋值总论)
- dictionaries (字典) : Dictionaries (字典)
- sets (集合) : Sets (集合) ; Part VI, Classes and OOP (第 VI 部分, 类与 OOP (面向对象编程))
- shared objects (shared 对象) : Shared References

(共享引用)

- slice objects (slice 对象) : Intercepting Slices (拦截切片)
- state information (状态 information) : Function Attributes (函数属性)
- storage (存储)
- CSV module (CSV 模块) : Storing Objects with Other Tools (用其它工具存储对象)
- JSON (JSON 格式) : Storing Objects with JSON (用 JSON 存储对象)
- pickle module (pickle 模块) : Storing Objects with pickle (用 pickle 存储对象)
- struct module (struct 模块) : Storing Objects with Other Tools (用其它工具存储对象)
- str (str (字符串类型)) : The str Object (str 对象)
- type objects (类型对象) : Types (类型)
- types (类型) : Types Live with Objects, Not Variables (类型存在于对象中, 而非变量中)
- offsets (偏移量) : Offsets and Items: enumerate (Offsets 与项: enumerate)
- OO (object-oriented) code (OO (对象-oriented) code) : OOP: The Big Picture (OOP: 大图景)
- OO (object-oriented), definition (OO (对象-oriented), 定义) : Why Do People Use Python? (为什么人们

使用 Python?)

- OOP (object-oriented programming) (OOP (面向对象编程) (对象-oriented programming)) : Why Do People Use Python? (为什么人们使用 Python?) ; OOP: The Big Picture (OOP: 大图景)
- arguments, functions (参数, 函数) : Why Use Classes? (为什么使用类?)
- attribute fetches (属性 fetches) : Attribute Inheritance Search (属性继承搜索)
- class objects (类对象) : Classes Generate Multiple Instance Objects (类生成多个实例对象)
- default behavior (默认行为) : Class Objects Provide Default Behavior (类对象提供默认行为)
- class statement (类语句) : User-Defined Objects (用户定义- 对象)
- class trees (类 trees) : Coding Class Trees (编写类树) ; Part VI, Classes and OOP (第 VI 部分, 类与 OOP (面向对象编程))
- classes (类) : Attribute Inheritance Search (属性继承搜索) ; Step 1: Making Instances (第一步: 创建实例)
- constructors (构造器) : Coding Constructors (编码构造器)
- inheritance (继承) : Classes Are Customized by Inheritance (类通过继承进行定制) ; A Second Example

(第二个示例)

- intercepting operators (intercepting 运算符) : Classes Can Intercept Python Operators (类可以拦截 Python 运算符) ; Other operator-overloading methods (Other 运算符重载- 方法)
- methods (方法) : Step 2: Adding Behavior Methods (第二步: 添加行为方法) ; Coding Methods (编码方法)
- methods, augmenting (方法, augmenting) : Augmenting Methods: The Bad Way (增强方法: 糟糕做法) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- subclasses (子类) : Classes Are Customized by Inheritance (类通过继承进行定制)
- subclassing (子类化) : Coding Subclasses (编码子类) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- superclasses (超类) : Attribute Inheritance Search (属性继承搜索) ; Classes Are Customized by Inheritance (类通过继承进行定制)
- testing (测试) : Testing as You Go (测试 as You Go)
- code reuse (code 复用) : OOP Is About Code Reuse (OOP 的本质是代码复用) ; Programming by customization (通过定制来编程)

- Test Your Knowledge: Part VI Exercises (第六部分练习)
- composition (组合) : Why Use Classes? (为什么使用类?) ; OOP and Composition: "Has-a" Relationships (OOP 与组合: "有一个" 关系) ; Stream Processors Revisited (再访流处理器)
- consistency (一致性) : Test Your Knowledge: Part VI Exercises (第六部分练习)
- delegation (委托) : OOP and Delegation: "Like-a" Relationships (OOP 与委托: "像一个" 关系)
- encapsulation (封装) : Python and OOP (Python 与 OOP) ; Test Your Knowledge: Part VI Exercises (第六部分练习)
- frameworks (框架) : Programming by customization (通过定制来编程)
- inheritance (继承) : Why Use Classes? (为什么使用类?) ; Python and OOP (Python 与 OOP) ; OOP and Inheritance: "Is-a" Relationships (OOP (面向对象编程) 与继承: "Is- " Relationships)
- attributes (属性) : Attribute Inheritance Search (属性继承搜索)
- multiple inheritance (多重继承) : Coding Class Trees (编写类树)
- search attribute (search 属性) : Why Use Classes? (为什么使用类?)

- instance objects (实例对象) : Instance Objects Are Concrete Items (实例对象是具体条目)
- instances (实例) : Why Use Classes? (为什么使用类?) ; Attribute Inheritance Search (属性继承搜索) ; Classes Generate Multiple Instance Objects (类生成多个实例对象)
- maintenance (维护) : Test Your Knowledge: Part VI Exercises (第六部分练习)
- method calls (方法调用) : Method Calls (方法调用)
- methods (方法) : Coding Class Trees (编写类树) ; A First Example (getattr 第一个示例)
- namespaces (命名空间) : Classes and Instances (类与实例)
- operator overload (运算符 overload) : Why Use Classes? (为什么使用类?) ; Operator Overloading (运算符重载)
- operator overloading (运算符重载) : Step 3: Operator Overloading (第三步: 运算符重载)
- print displays (打印显示) : Providing Print Displays (提供打印显示)
- polymorphism (多态) : Polymorphism and classes (多态与类)
- Polymorphism in Action-Polymorphism in Action (多态中的 Action-多态中的 Action) : Python and OOP

- (Python 与 OOP) ; Test Your Knowledge: Part VI Exercises (第六部分练习)
- reloading (重新加载) : Classes Generate Multiple Instance Objects (类生成多个实例对象)
- search attribute (search 属性) : Why Use Classes? (为什么使用类?)
- structure (结构) : Test Your Knowledge: Part VI Exercises (第六部分练习)
- open function (open 函数) : Other File-Like Tools (Other 文件-Like 工具) ; Redefining built-in names : For better or worse (Redefining 内置- 名字: 用于 better 或 worse)
- customizing (定制) : Test Your Knowledge: Answers (测验答案)
- operations (操作)
- dictionaries (字典) : Dictionaries (字典)
- mapping (映射) : Mapping Operations (映射操作)
- strings (字符串) : Basic Operations (基本操作)
- operator overloading (运算符重载) : Core Types Review and Summary (核心类型回顾与总结) ; Classes Can Intercept Python Operators (类可以拦截 Python 运算符) ; Step 3: Operator Overloading (第三步: 运算符重载) ; The Basics (基础) ; Part VI, Classes and OOP (第 VI 部分, 类与 OOP (面向对象编程))

- attribute privacy (属性 privacy) : Emulating Privacy for Instance Attributes: Part 1 (模拟 Privacy 用于实例属性: Part 1)
- attributes (属性)
- assignment (赋值) : Attribute Assignment and Deletion (属性赋值与删除)
- deleting (删除) : Attribute Assignment and Deletion (属性赋值与删除)
- references (引用) : Attribute Reference (属性引用)
- Boolean tests (布尔测试: bool 和 len) : Boolean Tests: bool and len (布尔测试: bool 和 len: bool 与 len)
- call expressions (调用表达式: call) : Call Expressions: call (调用表达式: call: call) ; Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- class type propagation (类类型 propagation) : Propagating class type (Propagating 类类型)
- comparison operators (比较运算符) : Comparisons: lt, gt, and Others (比较运算: lt、gt 等: lt, gt, 与 Others)
- constructors (构造器) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)

- contains method (contains 方法) : Membership: contains, iter, and (成员测试: contains、iter 与 getitem: contains, iter, 与)
- getitem-Membership: contains, iter (getitem-成员测试: contains、iter 与 getitem: contains, iter)
- descriptors (描述符) : Workaround: Generating operator-overloading descriptors (Workaround: Generating 运算符重载- 描述符)
- destructors (析构器) : Object Destruction: del (对象销毁: del: del) ; Destructor Usage Notes (Destructor 用法 Notes)
- display formats (显示格式) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str) ; Display Usage Notes (Display 用法 Notes)
- expressions (表达式) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)
- fallback options (回退选项) : Membership: contains, iter, and getitem (成员测试: contains、iter 与 getitem: contains, iter, 与 getitem)
- getattr method, getattr (getattr 方法, getattr)
- getattribute method, getattr (getattribute 方法, getattr)
- iadd method (iadd 方法) : In-Place Addition (中的-Place Addition)

- in-place addition (中的-place addition) : In-Place Addition (中的-Place Addition)
- index iteration (索引迭代) : Index Iteration: getitem (索引迭代: getitem)
- instance indexing (实例索引) : Indexing and Slicing: getitem and setitem (索引与切片: getitem 和 setitem: getitem 与 setitem) ; But index Means As-Integer (But index Means As-整数)
- iterable objects (可迭代对象: iter 和 next) : Iterable Objects: iter and next (可迭代对象: iter 和 next: iter 与 next)
- multiple (多个) : Multiple Iterators on One Object (Multiple 迭代器 on One 对象) ; Classes versus slices (类对比 slices)
- user-defined iterables (用户定义的可迭代对象) : User-Defined Iterables (用户定义的可迭代对象) ; Classes versus generators (类对比生成器)
- metaclass methods (元类方法) : Operator Overloading in Metaclass Methods (元类方法中的运算符重载)
- methods (方法) : Common Operator-Overloading Methods (常用运算符重载方法) ; Common Operator - Overloading Methods (常用运算符重载方法)
- inline coding (inline 编码) : Workaround: Coding operator-overloading methods inline (Workaround:

编码运算符重载- 方法 inline)

- superclasses (超类) : Workaround: Coding operator-overloading methods in superclasses (Workaround: 编码运算符重载- 方法中的超类)
- object comparisons (对象比较运算: lt、gt 等) : Comparisons: lt, gt, and Others (比较运算: lt、gt 等: lt, gt, 与 Others)
- OOP (OOP (面向对象编程)) : Why Use Classes? (为什么使用类?) ; Operator Overloading (运算符重载)
- print displays (打印显示) : Providing Print Displays (提供打印显示)
- radd method (radd 方法) : Right-Side Addition (右侧加法)
- add reuse (add 复用) : Reusing add in radd (Reusing add 中的 radd)
- results (结果) : Returning results—or not (Returning results—或 not)
- right-side addition (右侧加法) : Right-Side Addition (右侧加法) ; Propagating class type (Propagating 类类型)
- strings (字符串) : Basic Operations (基本操作)
- super function (super 函数) : Noncalls and operator overloading (Noncalls 与运算符重载)

- operators (运算符)
- generic operations (通用操作) : Part II, Objects and Operations (第 II 部分, 对象与操作)
- lists (列表) : Basic List Operations (列表基本操作)
- optimization tools (优化工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- ordinals (序数) : String comparisons (字符串比较运算: lt、gt 等)
- ORMs (object-relational mappers) (ORMs (对象-relational mappers)) : Database Access (数据库访问)
- os module (os 模块) : Command-line launchers (命令行启动器)
- descriptor files (描述符文件) : Other File Tools (其它文件工具)
- OSError exception class (OSError 异常类) : Built-in Exception Classes (内置异常类)
- output buffering (output 缓冲) : Opening Files (打开文件)
- overloading operators (重载运算符) : Core Types Review and Summary (核心类型回顾与总结)

P (字母 P)

- package-relative imports (包相对导入) : Module P

ackages (模块包)

- absolute (绝对) : Relative and Absolute Imports (相对导入与绝对导入)

- name clashes (名字 clashes) : Module Name Clashes: Package and Package-Relative (模块名字 Clashes: 包与包-Relative)

- Imports (导入)

- relative (相对) : Relative and Absolute Imports (相对导入与绝对导入) ; The absolute-import solution (绝对导入的解法)

- Python Import Models (Python 的导入模型)

- packages (包)

- benefits (好处) : Why Packages? (为什么要用包?) ; A Tale of Two Systems (两个系统的故事)

- dotted-path syntax (点分路径- 语法) : Package Imports (包导入)

- module import (模块 import) : Using the basic package (使用基础包)

- relative imports (相对导入) : Relative-Import Rationales and Trade-Offs (相对导入的理由与权衡)

- encapsulation (封装) : Python and OOP (Python 与 OOP)

- folder bundles (文件夹捆绑包) : Module Packages (模块包)

- folders (文件夹)
- names (名字) : Basic Package Structure (基本包结构)
- nesting (嵌套) : Basic Package Structure (基本包结构)
- imports (导入) : Module Packages (模块包) ; Package Imports (包导入) ; Python Import Models (Python 的导入模型) ; Part V, Modules and Packages (第 V 部分, 模块与包)
- folder hierarchies (文件夹层次) : Package Imports (包导入)
- from statement (from 语句) : Using the basic package (使用基础包)
- modules (模块) : Using the basic package (使用基础包)
- init.py file (init.py 文件) : Package init.py Files (包的 init.py 文件) ; Using the updated package (使用更新后的包) ; The Roles of init.py Files (init.py 文件的作用)
- init.py files (init.py 文件)
- initialization (初始化) : The Roles of init.py Files (init.py 文件的作用)
- main.py file (main.py 文件) : Package main.py Files (包的 main.py 文件) ; Using the updated package (使用更新后的包)

- module search path and (模块 search path 与) : Packages and the Module Search Path (包与模块搜索路径)
- name clashes (名字 clashes) : Module Name Clashes: Package and Package-Relative (模块名字 Clashes: 包与包-Relative)
- Imports (导入)
- namespace packages (命名空间包) : Module Packages (模块包) ; Using the updated package (使用更新后的包)
- Namespace Packages-Namespace Packages in Action (命名空间包 -命名空间包实战)
- module searches (模块 searches) : The Module Search Algorithm (模块搜索算法)
- regular packages (regular 包) : Using the updated package (使用更新后的包)
- pandas (pandas 数据分析库) : Numeric and Scientific Programming (Numeric 与科学计算编程)
- parallel programming (并行编程)
- function calls (函数调用) : Common Coding Gotchas (常见编码陷阱)
- lambda expression (lambda 表达式) : lambda Basics (lambda 基础)
- statements (语句) : Parentheses are optional (括号可选)

- tuples (元组) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- parsing (解析)
- slicing (切片) : Indexing and Slicing (索引与切片 : getitem 和 setitem)
- text, string methods (text, 字符串方法) : More String Methods: Parsing Text (More 字符串方法: 解析文本)
- pass statements (pass 语句) : break, continue, pass, and the Loop else (break、continue、pass 与循环 else)
- pathnames (路径名) : Filenames in open and Other Filename Tools (文件名: open 与其它文件名工具)
- patterns (模式)
- factory functions (工厂函数) : Closures and Factory Functions (闭包与工厂函数)
- re module (re 模块) : The re Pattern-Matching Module (re Pattern-Matching 模块)
- sequence assignments (序列赋值) : Advanced sequence-assignment patterns (高级序列赋值模式)
- structural pattern matching (结构化模式匹配) : match Statements (match 语句)
- per-call scopes (per-call 作用域) : Scopes Overview (作用域概述)

- performance (性能) : Performance implications (性能 implications)
- list comprehensions (列表推导式)
- object storage (对象 storage) : Storing Objects with pickle (用 pickle 存储对象)
- pickle objects (pickle 对象) : Database Access (数据库访问) ; Stream Processors Revisited (再访流处理器)
- pipes (管道) : Other File Tools (其它文件工具)
- plain functions (plain 函数) : Method Objects: Bound or Not (方法对象: 绑定与否) ; Why the Special Methods? (Why Special 方法?)
- plain-function methods (plain-函数方法) : Plain-Function Methods (Plain-函数方法)
- pointers (指针) : Variables, Objects, and References (变量、对象与引用)
- polymorphism (多态) : Sequence Operations (序列操作) ; Types (类型) ; Test Your Knowledge: Answers (测验答案)
- Polymorphism in Python-Polymorphism in Python (Python 中的多态 -Python 中的多态) : Polymorphism and classes (多态与类) ; Polymorphism in Action (多态中的 Action)
- Python and OOP (Python 与 OOP) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)

- dynamic typing (动态类型) : Dynamic Typing Is Everywhere (动态类型无处不在)
- interfaces and (interfaces 与) : Polymorphism Means Interfaces, Not Call Signatures (多态意味着接口, 而非调用签名)
- intersect function (intersect 函数) : Polymorphism Revisited (再谈多态)
- strings and (字符串与) : Basic Operations (基本操作)
- popularity rise of Python (人气上升的 Python) : The Python Upside (Python 的优势)
- portability (可移植性) : Why Do People Use Python? (为什么人们使用 Python?) ; It's Portable (它可移植)
- Portable Operating System Interface (POSIX) (Portable Operating System 接口 (POSIX)) : Systems Programming (系统编程)
- positional arguments (位置式参数) : A Basic Range-Testing Decorator for Positional Arguments (基础版: 仅支持位置参数)
- positionals, argument matching (positionals, 参数匹配) : Argument Matching Overview (参数匹配概览)
- POSIX (Portable Operating System Interface) (POSIX (Portable Operating System 接口)) : Systems Pro

gramming (系统编程)

- precision in large numbers (precision 中的 large 数字) : Numbers (数字)
- print emulator, keyword-only arguments (print emulator, 仅限关键字- 参数) : Using Keyword-Only Arguments (使用仅限关键字参数)
- print function (print 函数) : The print function in action (print 函数中的 action)
- argument matching (参数匹配) : Example: Rolling Your Own Print (示例: 自己动手写 print) ; Using Keyword- Only Arguments (使用仅限关键字参数)
- call format (调用格式) : Call format (调用格式)
- end-of-line character (end-的-line character) : The print function in action (print 函数中的 action)
- f-strings (f-字符串) : The print function in action (print 函数中的 action)
- keyword arguments (keyword 参数) : The print function in action (print 函数中的 action)
- print operations (print 操作) : Print Operations (print 操作)
- file object methods (文件对象方法) : Print Operations (print 操作)
- print statement (print 语句) : Test Your Knowledge: Answers (测验答案)

- print stream redirection (打印流的重定向)
- automatic (自动) : Automatic stream redirection (自动流重定向)
- hello world (Hello World 示例) : The Python "hello world" program (Python "Hello World " 程序)
- manual (手册/手动) : Manual stream redirection (手动流重定向)
- standard output streams (标准输出流) : Print Operations (print 操作)
- stdout (标准输出) : Test Your Knowledge: Answers (测验答案)
- print statements (print 语句) : The Programmer's View (程序员视角)
- print stream (打印流)
- redirecting (重定向)
- automatic (自动) : Automatic stream redirection (自动流重定向)
- hello world (Hello World 示例) : The Python "hello world" program (Python "Hello World " 程序)
- manual (手册/手动) : Manual stream redirection (手动流重定向)
- print, viewing docstrings (print, viewing 文档字符串)

-) : User-defined docstrings (用户定义的文档字符串)
- printing (打印)
- exceptions, built-in (异常, 内置-) : Default Printing and State (默认打印与状态)
- custom printing (自定义打印) : Custom Print Displays (自定义打印显示)
- operator overloading and (运算符重载与) : Providing Print Displays (提供打印显示)
- procedures (过程) : Why Use Functions? (为什么要用函数?)
- productivity, developers (生产力, developers) : Why Do People Use Python? (为什么人们使用 Python?) ; Developer Productivity (开发者生产力)
- profilers (性能分析器) : Development Tools for Larger Projects (更大型项目的开发工具)
- program architecture (程序架构)
- attributes (属性) : Imports and Attributes (导入与属性)
- files (文件)
- text files (text 文件) : How to Structure a Program (如何组织一个程序)
- top-level (顶层) : How to Structure a Program (如何组织一个程序)

- modules (模块)
- files (文件) : How to Structure a Program (如何组织一个程序)
- importing (导入中) : Imports and Attributes (导入与属性) ; Standard-Library Modules (标准库模块)
- standard-library (标准库-) : Standard-Library Modules (标准库模块)
- program design (see design) (程序设计 (see 设计))
- program execution (程序执行) : The Programmer's View (程序员视角)
- program files (程序文件)
- command line (命令行) : Program Files (程序文件) ; Running Files with Command Lines (用命令行运行文件)
- file icons (文件 icons) : Clicking and Tapping File Icons (Clicking 与 Tapping 文件 Icons)
- modules (模块) : Program Files (程序文件)
- naming convention (命名约定) : The Programmer's View (程序员视角)
- scripts (脚本) : Program Files (程序文件)
- program units (程序 units) : Python's Core Object Types (Python 的核心对象类型)
- programming-in-the-large (programming-中的--large) : It's Powerful (它很强大)

- programs (程序) : Part I, Getting Started (第 I 部分: 入门)
- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)
- module files (模块文件) : Program Files (程序文件)
- modules (模块) : The Python Conceptual Hierarchy (Python 概念层级)
- prompts, interactive coding (提示符, 交互式编程)
: What Not to Type: Prompts and Comments (不要键入的内容: 提示符与注释) ; Why the Interactive Prompt? (为什么用交互式提示符?)
- attribute fetches (属性 fetches) : The Basics (基础)
- attributes (属性) : Property basics (property (属性) 基础)
- computing value (computing 值) : Computed Attributes (getattr 计算属性)
- creating (创建) : The Basics (基础)
- decorators (装饰器) : Coding Properties with Decorators (用装饰器编写 property)
- deleters (删除器) : Setter and deleter decorators (setter 与 deleter 装饰器)
- setters (设置器) : Setter and deleter decorators (setter 与 deleter 装饰器)

- descriptors and (描述符与) : How Properties and Descriptors Relate (property 与描述符如何关联) ; Descriptors (描述符)
- inheritance (继承) : A First Example (getattr 第一个示例)
- objects (对象) : class statement, Property basics prototyping, Rapid Prototyping pseudoprivate attributes, Pseudoprivate Class Attributes-Why Use Pseudoprivate Attributes? pseudoprivate naming, Listing instance attributes with dict PVM (Python Virtual Machine), The Python Virtual Machine (PVM), Test Your Knowledge: Answers .py suffix, Module Filenames py2wasm, Internet and Web Scripting .pyc files, Bytecode compilation (类语句, property (属性) 基础 prototyping, Rapid Prototyping pseudoprivate 属性, 伪私有类属性 -Why 用法 Pseudoprivate 属性? pseudoprivate naming, 用 dict 列出实例属性 PVM (Python 虚拟机), Python 虚拟机 (PVM), 知识测验: Answers.py suffix, 模块文件名 py2wasm, 互联网与 Web 脚本.pyc 文件, 字节码编译)
- .pyc files (.pyc 文件) : Bytecode compilation (字节码编译)
- PyCharm (PyCharm 集成开发环境) : Other IDEs for Python (其他 Python IDE)
- PyDev (PyDev (Eclipse 插件)) : Other IDEs for Python (其他 Python IDE)

- Pydoc (Pydoc 文档工具) : Getting Help (获取帮助) ; Python Documentation Sources (Python 文档资源)
- browser mode (浏览器模式) : Using Pydoc' s browser interface (Using Pydoc' s browser 接口)
- built-in tools (内置- 工具) : Running help on built-in tools (运行 help on 内置- 工具)
- imported modules (imported 模块) : More Pydoc tips (更多 Pydoc 技巧)
- module-index page (模块-索引 page) : Using Pydoc' s browser interface (Using Pydoc' s browser 接口)
- modules (模块) : Running help on your own code (运行 help on your own code)
- Pygments (Pygments 语法高亮) : Beyond Docstrings: Sphinx (超越 Docstring: Sphinx)
- Pyjamas (Pyjamas (Python 到 JS)) : Internet and Web Scripting (互联网与 Web 脚本)
- pyjnius (Pyjnius (Python 调 Java)) : Component Integration (组件集成)
- Pyodide (Pyodide (浏览器 CPython)) : Internet and Web Scripting (互联网与 Web 脚本)
- PyPy (PyPy 实现) : OK, but What' s the Downside? (好, 但缺点是什么?) ; Numeric and Scientific Programming (Numeric 与科学计算编程)

- Python Implementation Alternatives (Python 实现替代方案) : Python Implementation Alternatives (Python 实现替代方案) ; Test Your Knowledge: Answers (测验答案)
- PyQt (PyQt GUI 绑定) : GUIs and UIs (GUI 与 UI)
- PyScript (PyScript 浏览器 Python) : Internet and Web Scripting (互联网与 Web 脚本)
- PyScripter (PyScripter 集成开发环境) : Other IDEs for Python (其他 Python IDE)
- Pyston (Pyston 实现) : Python Implementation Alternatives (Python 实现替代方案)
- Python (Python 语言)
- on Android (在 Android 上) : Using Python on Android (A.5 在 Android 上使用 Python)
- applications (应用) : What Can I Do with Python? (我能用 Python 做什么?) ; And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- bytecode and (字节码与) : OK, but What' s the Downside? (好, 但缺点是什么?)
- code size (代码体积) : Why Do People Use Python? (为什么人们使用 Python?)
- current users (当前用户) : Who Uses Python Today? (今天谁在用 Python?)

- developer productivity (开发者生产力) : Why Do People Use Python? (为什么人们使用 Python?) ; Developer Productivity (开发者生产力)
- embedding (嵌入) : Other Launch Options (其它启动选项)
- execution speed (执行速度) : OK, but What' s the Downside? (好, 但缺点是什么?)
- on iOS (在 iOS 上) : Using Python on iOS (A.6 在 iOS 上使用 Python)
- on Linux (在 Linux 上) : Using Python on Linux (A.4 在 Linux 上使用 Python)
- on macOS (在 macOS 上) : Using Python on macOS (A.3 在 macOS 上使用 Python)
- popularity rise (人气上升) : The Python Upside (Python 的优势)
- portability (可移植性) : Why Do People Use Python? (为什么人们使用 Python?)
- as scripting language (作为脚本语言) : Is Python a "Scripting Language" ? (Python 是"脚本语言"吗?)
- software quality (软件质量) : Why Do People Use Python? (为什么人们使用 Python?) ; Software Quality (软件质量)
- on Windows (在 Windows 上) : Using Python on Windows (A.2 在 Windows 上使用 Python)

- Command Prompt (命令提示符) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- IDLE (A.2.6 IDLE: Python 自带 IDE) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- WSL (WSL (Windows 的 Linux 子系统)) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- Python Virtual Machine (Python 虚拟机) : The Python Virtual Machine (PVM) (Python 虚拟机 (PVM)) ; Test Your Knowledge: Answers (测验答案)
- Python/Java bridges (Python/Java 桥接) : Component Integration (组件集成)
- PYTHONPATH directory (PYTHONPATH 目录) : Search-Path Components (搜索路径的组成部分)
- configuration search path (配置搜索路径) : Configuring the Search Path (配置搜索路径)
- PyThran (PyThran 科学计算加速) : Numeric and Scientific Programming (Numeric 与科学计算编程) ; Python (Python 语言)

Q (字母 Q)

- qualification, attribute names (qualification, 属性名字) : Attribute Name Qualification (属性名限定)
- quality assurance applications (质量 assurance applications) : And More: AI, Games, Images, QA, Excel

, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)

- queues (队列)

- FIFO (first-in-first-out) (FIFO (先进先出)) : Recursion versus queues and stacks (Recursion 对比 queues 与 stacks)

- recursion (递归) : Recursion versus queues and stacks (Recursion 对比 queues 与 stacks)

- quote characters in strings (引号字符中的字符串)
: Other Ways to Code Strings (Other Ways 到 Code 字符串)

R (字母 R)

- radd method (radd 方法) : Right-Side Addition (右侧加法)

- add reuse (add 复用) : Reusing add in radd (Reusing add 中的 radd)

- raise statement (raise 语句) : The raise Statement (raise 语句) ; Breaking Out of Multiple Nested Loops: "go to" (跳出多层嵌套循环: "goto")

- except block, scopes (except block, 作用域) : Scopes and except as (作用域与 except as)

- except clause (except 子句) : The except as hook (except as 钩子)

- exception propagation (异常 propagation) : Propagating Exceptions with raise (用 raise 传播异常)
- from clause (from 子句) : Exception Chaining: raise from (异常链: raise from)
- random module (random 模块) : Numbers (数字)
- range object (range 对象)
- counter loops (counter 循环) : Counter Loops: range (Counter 循环: range)
- iteration protocol (迭代协议) : Reprise: Dictionaries, range, enumerate, and zip (重温: 字典、range、enumerate 与 zip)
- lists (列表) : Changing Lists: range and Comprehensions (Changing 列表: range 与推导式)
- sequence reordering (序列 reordering) : Sequence Shufflers: range and len (序列 Shufflers: range 与 len)
- sequence scans (序列 scans) : Sequence Scans: while, range, and for (序列扫描: while、range 与 for)
- skipping items (skipping 项) : Skipping Items: range and Slices (Skipping 项: range 与 Slices)
- range-testing decorator (range-测试 decorator)
: A Basic Range-Testing Decorator for Positional Arguments (基础版: 仅支持位置参数) ; Decorator Arg

uments Versus Function Annotations (装饰器参数对比函数注解)

- rational numbers (rational 数字) : Numbers (数字)
- raw bytes (raw 字节串) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- raw strings (raw 字符串) : Example: Rolling Your Own Print (示例: 自己动手写 print)
- docstrings (文档字符串) : User-defined docstrings (用户定义的文档字符串)
- escape sequences and (escape 序列与) : Raw Strings Suppress Escapes (Raw 字符串抑制转义) ; Raw Strings (Raw 字符串)
- Suppress Escapes (抑制转义)
- re module, pattern-matching (re 模块, pattern-matching) : The re Pattern-Matching Module (re Pattern-Matching 模块)
- read-only descriptors (只读描述符) : Read-only descriptors (只读描述符)
- readability (可读性) : Why Do People Use Python? (为什么人们使用 Python?)
- record factories (记录工厂) : Step 2: Adding Behavior Methods (第二步: 添加行为方法)
- recursion (递归) : Comparisons, Equality, and Truth (比较、相等与真值) ; Part V, Modules and Packa

ges (第 V 部分, 模块与包)

- arbitrary structures (任意 structures) : Handling Arbitrary Structures (Handling 任意 Structures)

- testing (测试) : Testing with a separate script (测试带 separate 脚本)

- cycles (循环) : Cycles, paths, and stack limits (Cycles, paths, 与栈限制)

- direct (直接) : Coding Alternatives (编码替代方案)

- from imports (from 导入) : Recursive from Imports May Not Work (Recursive from 导入 May Not Work)

- functions (函数) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)

- if else expression (if else 表达式) : Coding Alternatives (编码替代方案)

- indirect (间接) : Coding Alternatives (编码替代方案)

- local variables (局部变量) : Scopes Overview (作用域概述)

- versus loop statements (对比 loop 语句) : Loop Statements Versus Recursion (Loop 语句对比 Recursion)

- paths (路径) : Cycles, paths, and stack limits (Cycles, paths, 与栈限制)

- queues (队列) : Recursion versus queues and stacks (Recursion 对比 queues 与 stacks)
- stack limits (栈限制) : Cycles, paths, and stack limits (Cycles, paths, 与栈限制)
- stacks (栈) : Recursion versus queues and stacks (Recursion 对比 queues 与 stacks)
- sum function (sum 函数) : Summation with Recursion (Summation 带 Recursion)
- reduce function (reduce 函数) : Combining Items in Iterables: reduce (Combining 项中的可迭代对象: reduce)
- redundancy (冗余)
- classes (类) : OOP: The Big Picture (OOP: 大图景)
- increase in (increase 中的) : The Python Tsunami (Python 海啸)
- modules (模块) : Why Use Modules? (为什么使用模块?)
- removing (移除) : Coding Methods (编码方法)
- reference counts (引用 counts) : Nesting Revisited (再谈嵌套)
- references (引用) : References Versus Copies (引用与拷贝)
- built-in types (内置- 类型) : Assignment Creates R

- references, Not Copies (赋值制造引用, 而非拷贝)
- lists (列表) : Lists (列表)
- names (名字) : Name Resolution: The LEGB Rule (名字解析: LEGB 规则)
- nested scopes (嵌套作用域) : Nested Scopes Overview (嵌套作用域概述)
- shared (共享) : Shared References (共享引用)
- argument passing (参数传递) : Arguments and Shared References (参数与共享引用)
- equality (相等) : Shared References and Equality (共享引用与相等性) ; Shared References (共享引用)
- in-place changes (中的-place 修改) : Shared References and In-Place Changes (共享引用与原地修改)
- multiple-target assignments (多重目标赋值) : Multiple-target assignment (多目标赋值)
- variables (变量) : Variables, Objects, and References (变量、对象与引用)
- relative imports (相对导入) : Relative and Absolute Imports (相对导入与绝对导入) ; Python Import Models (Python 的导入模型)
- relative magnitude comparisons (相对大小比较运算 : lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值)

- reload function (reload 函数) : Reloading Module s (重载模块) ; reload Odds and Ends (reload Odds 与 Ends)
- from imports, reload (from 导入, reload)
- interactive testing and, reload (交互式测试与, reload)
- OOP (OOP (面向对象编程)) : Classes Generate Multiple Instance Objects (类生成多个实例对象)
- recursive coding (recursive 编码) : Alternative codings (替代写法)
- recursive reloaders (递归重新加载器) : A recursive reloader (递归重新加载器)
- testing (测试) : Testing recursive reloads (测试 recursive reloads)
- reload variants (reload 变体) : Testing reload variants (测试 reload 变体)
- repetition (重复) : The Python Conceptual Hierarchy (Python 概念层级)
- replace method, "Changing" (replace 方法, "Changing") : Strings (字符串)
- code folders (代码目录) : Where to Run: Code Folders (Where 到 Run: 代码目录)
- f-string literal (f-字符串 literal) : F-string formatting basics (F-字符串格式化基础)

- IPython (IPython 增强交互环境) : Other Python REPLs (其他 Python REPL)
- preloading tools (preloading 工具) : Example: Listing Modules with dict (示例: 用 dict 列出模块)
- starting (启动) : Starting an Interactive REPL (启动交互式 REPL)
- repr method (repr 方法) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str)
- reserved words in variable naming (reserved words 中的 variable naming) : Variable Name Rules (变量命名规则)
- REST (REST 风格) : Component Integration (组件集成)
- reStructuredText markup language (reStructuredText 标记语言) : Beyond Docstrings: Sphinx (超越 Docstring: Sphinx)
- return statement (return 语句) : Basic Function Tools (基本函数工具) ; return Statements (return 语句)
- returns, none (返回 None) : Functions Without returns (没有 return 的函数)
- runtime declarations (运行时声明)
- function decorators (函数装饰器)
- imports (导入) : How Imports Work (import 是如何工作的)

· module imports (模块导入) : Imports Are Runtime Assignments (导入是运行时赋值)

S (字母 S)

· sandbox (沙箱) : The Python Sandbox (Python 沙盒)

· scientific programming (科学计算编程) : Numeric and Scientific Programming (Numeric 与科学计算编程)

· SciPy (SciPy 科学计算库) : Numeric and Scientific Programming (Numeric 与科学计算编程)

· scopes (作用域) : Python Scopes Basics (Python 作用域基础) ; Scopes in Methods and Classes (作用域中的方法与类)

· argument defaults and (argument 默认值与) : Scopes and Argument Defaults (作用域与参数默认值) ; Loops Require Defaults, Not Scopes (循环需要默认值, 而不是作用域)

· built-ins (内置) : The Built-in Scope (内置作用域)

· LEGB rule (LEGB 规则) : Redefining built-in names: For better or worse (Redefining 内置- 名字: 用于 better 或 worse)

· comprehension variables (comprehension 变量) : Scopes and comprehension variables (作用域与推导式变量)

- comprehensions, loop variables (推导式, loop 变量) : Preview: Other Python scopes (预览: Python 的其他作用域)
- cross-file changes (cross-文件修改) : Program Design: Minimize Cross-File Changes (程序设计: 尽量减少跨文件修改)
- enclosing (外层封闭) : Enclosing scopes and loop variables (外层作用域与循环变量)
- functions (函数) : Enclosing scopes and loop variables (外层作用域与循环变量)
- state retention and (状态 retention 与) : State retention and enclosing scopes (状态 retention 与 enclosing 作用域)
- examples (示例) : Scopes Examples (作用域示例)
- exception variables (异常变量) : Preview: Other Python scopes (预览: Python 的其他作用域)
- exceptions (异常) : Scopes and except as (作用域与 except as)
- global scopes (全局作用域) : Scopes Overview (作用域概述)
- global statement emulation (全局语句 emulation) : Other Ways to Access Globals (访问全局的其他方式)
- global variables, program design (全局变量, 程序设计) : Program Design: Minimize Global Variables (

程序设计：尽量减少全局变量)

- global, state retention (全局, 状态 retention)
- lexical scoping (词法作用域) : Python Scopes Basics (Python 作用域基础) ; Imports Versus Scopes (导入与作用域)
- local scopes (局部作用域) : Scopes Overview (作用域概述)
- modules (模块)
- imports and (导入与) : How Files Generate Name spaces (文件如何生成命名空间)
- variables (变量) : Imports Versus Scopes (导入与作用域)
- named assignments (named 赋值总论) : Preview: Other Python scopes (预览: Python 的其他作用域)
- nested class (嵌套类) : Nested Classes: The LEGB Scopes Rule Revisited (嵌套类: 重温 LEGB 作用域规则)
- nested scopes (嵌套作用域) : Nested Scopes Examples (嵌套作用域示例)
- arbitrary nesting (任意嵌套) : Arbitrary Scope Nesting (任意深度的作用域嵌套)
- name assignments (名字赋值总论) : Nested Scopes Overview (嵌套作用域概述)
- name references (名字引用) : Nested Scopes Over

view (嵌套作用域概述)

- per-call scopes (per-call 作用域) : Scopes Overview (作用域概述)

- script1.py (script1.py 示例) : A First Script (第一个脚本)

- scripting (脚本编写)

- server-side scripting (服务器端脚本) : Internet and Web Scripting (互联网与 Web 脚本)

- web scripting (Web 脚本) : Internet and Web Scripting (互联网与 Web 脚本)

- scripting language (脚本语言) : Is Python a “Scripting Language” ? (Python 是“脚本语言”吗?)

- control language and (控制语言与) : Is Python a “Scripting Language” ? (Python 是“脚本语言”吗?)

- ease of use and (ease 的用法与) : Is Python a “Scripting Language” ? (Python 是“脚本语言”吗?)

- shell tools and (shell 工具与) : Is Python a “Scripting Language” ? (Python 是“脚本语言”吗?)

- scripts (脚本) : Program Files (程序文件) ; Part I, Getting Started (第 I 部分: 入门)

- exercise (练习) : Test Your Knowledge: Part I Exercises (练习: 测验你的知识)

- web servers (Web 服务器) : Other Launch Option

s (其它启动选项)

- search tables (查找表) : Why Use Built-in Objects ? (为什么使用内置对象?)

- searches (搜索)

- init.py files (init.py 文件) : The Roles of init.py Files (init.py 文件的作用)

- lists (列表) : Test Your Knowledge: Answers (测验答案)

- packages (包) : Packages and the Module Search Path (包与模块搜索路径)

- selection (选择) : The Python Conceptual Hierarchy (Python 概念层级)

- self attribute (self 属性) : Why Use Pseudoprivate Attributes? (为何使用伪私有属性?)

- semicolons in statements (semicolons 中的语句) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; End-of-line is end of statement (End-of-line is end 的语句)

- send method (send 方法) : Extended generator function protocol: send versus next (扩展的生成器函数协议: send 与 next)

- sequence (序列) : The Python Conceptual Hierarchy (Python 概念层级)

- sequence assignments (序列赋值) : Assignment Syntax Forms (赋值语法形式) ; Sequence Assignment

- ts (序列赋值) ; Advanced sequence-assignment patterns (高级序列赋值模式)
- sequence operations, “Changing” (序列操作, “Changing”) : Strings (字符串)
- arbitrary expressions (任意表达式) : Sequence Operations (序列操作)
- byte strings (字节字符串) : Sequence Operations (序列操作) ; Formatting (格式化)
- indexing expressions (索引表达式) : Sequence Operations (序列操作)
- list comprehensions (列表推导式) : Conversions, methods, and immutability (转换、方法与不可变性)
- lists (列表) : Sequence Operations (序列操作)
- sequence patterns (序列 patterns) : Advanced match Usage (match 高级用法)
- sequence scans (序列 scans)
- for loop (用于 loop) : Sequence Scans: while, range, and for (序列扫描: while、range 与 for)
- range object (range 对象) : Sequence Scans: while, range, and for (序列扫描: while、range 与 for)
- while loop (while 循环) : Sequence Scans: while, range, and for (序列扫描: while、range 与 for)
- sequences (序列) : Test Your Knowledge: Answers (测验答案)

- dictionaries (字典) : Dictionary Usage Tips (字典用法 Tips)
- for loops (for 循环) : Tuple (sequence) assignment in for loops (元组 (序列) assignment 中的 for 循环)
- lists (列表) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- permutation (排列) : Permutating Sequences (Permutating 序列) ; Why generators here: Space, time, and more (Why 生成器 here: Space, time, 与 more)
- reordering (重排) : Sequence Shufflers: range and len (序列 Shufflers: range 与 len)
- slicing (切片) : Scrambling Sequences (Scrambling 序列)
- functions (函数) : Simple functions (Simple 函数)
- generator expressions (生成器表达式) : Generator expressions (生成器表达式)
- generator functions (生成器函数) : Generator functions (生成器函数)
- tester (测试器) : Tester client (测试客户端)
- strings (字符串) : String Object Basics (字符串对象基础) ; Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)

- tuples (元组) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories) ; Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- virtual sequences (虚拟序列) : Iterations (迭代)
- zip object (zip 对象) : Parallel Traversals: zip (并行遍历: zip) ; More zip roles: dictionaries (zip 的更多角色: 字典)
- serialization (序列化)
- json module (JSON 模块) : The pickle and json Serialization Modules (pickle 与 JSON 序列化模块)
- objects (对象) : Storing Objects with pickle (用 pickle 存储对象)
- pickle module (pickle 模块) : The pickle and json Serialization Modules (pickle 与 JSON 序列化模块)
- server-side scripting (服务器端脚本) : Internet and Web Scripting (互联网与 Web 脚本)
- set comprehensions (集合推导式) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- syntax (语法) : Formal Comprehension Syntax (推导式的正式语法)
- set function (集合函数) : Sets (集合)
- set method (set 方法) : Inserting Code to Run on Attribute Access (在属性访问时插入代码)

- setattr method (setattr 方法) : Attribute Assignment and Deletion (属性赋值与删除) ; Inserting Code to Run on Attribute Access (在属性访问时插入代码)
- setitem method (setitem 方法) : Indexing and Slicing: getitem and setitem (索引与切片: getitem 和 setitem: getitem 与 setitem)
- Intercepting Item Assignments (拦截项赋值)
- sets (集合) : Numbers (数字) ; Sets (集合) ; Test Your Knowledge: Answers (测验答案) ; Core Types Review and Summary (核心类型回顾与总结)
- comparisons (比较运算: lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值)
- dictionaries (字典) : Dictionary views and sets (字典 views 与集合)
- shared objects (shared 对象) : Shared References (共享引用)
- shared references (共享引用) : Shared References (共享引用)
- argument passing (参数传递) : Arguments and Shared References (参数与共享引用)
- equality (相等) : Shared References and Equality (共享引用与相等性)
- in-place changes (中的-place 修改) : Shared References and In-Place Changes (共享引用与原地修改)

- multiple-target assignments (多重目标赋值) : Multiple-target assignment (多目标赋值)
- command line (命令行) : Command-Line Usage Variations (命令行用法变体)
- stream redirection (流重定向) : Command-Line Usage Variations (命令行用法变体)
- shell tools (shell 工具) : Is Python a "Scripting Language" ? (Python 是"脚本语言"吗?) ; Systems Programming (系统编程)
- shell-command streams (shell 命令流) : Other File Tools (其它文件工具)
- shelve module (shelve 模块) : Pickles and Shelves (Pickles 与 Shelves) ; The shelve module (shelve 模块)
- object storage (对象 storage) : Storing Objects on a shelve Database (Storing 对象 on shelve 数据库) ; Exploring Shelves Interactively (交互式探索 shelf)
- updates (更新) : Updating Objects on a Shelf (Updating 对象 on Shelf)
- shelves (shelve 存储) : Other File Tools (其它文件工具) ; Stream Processors Revisited (再访流处理器)
- shipping (发布交付) : Development Tools for Larger Projects (更大型项目的开发工具)

- single-scan iteration (single-scan 迭代) : Single v
ersus multiple scans (Single 对比 multiple scans)
- site-packages directory (site-packages 目录) : Se
arch-Path Components (搜索路径的组成部分)
- slice assignments (slice 赋值总论) : Test Your Kno
wledge: Answers (测验答案) ; Index (索引)
- cleanup and (cleanup 与) : Extended slicing: The
third limit and slice objects (扩展切片: third limit 与
slice 对象)
- skipping items (skipping 项) : Skipping Items: ran
ge and Slices (Skipping 项: range 与 Slices)
- slicing (切片) : Sequence Operations (序列操作)
; String Object Basics (字符串对象基础) ; Test Your
Knowledge: Part II Exercises (自测: 第二部分练习)
; Part II, Objects and Operations (第 II 部分, 对象与
操作)
- lists (列表) : Indexing and Slicing (索引与切片: g
etitem 和 setitem)
- out of bounds (out 的边界) : Test Your Knowledg
e: Part II Exercises (自测: 第二部分练习)
- parsing (解析) : Indexing and Slicing (索引与切片
: getitem 和 setitem)
- sequences (序列) : Scrambling Sequences (Scra
mbling 序列)

- functions (函数) : Simple functions (Simple 函数)
- generator expressions (生成器表达式) : Generator expressions (生成器表达式)
- generator functions (生成器函数) : Generator functions (生成器函数)
- tester (测试器) : Tester client (测试客户端)
- strings (字符串) : Indexing and Slicing (索引与切片: getitem 和 setitem) ; Extended slicing: (扩展切片:)
- extended slicing (扩展切片) : Extended slicing: The third limit and slice objects (扩展切片: third limit 与 slice 对象)
- slots (slots (插槽)) : Classes: Under the Hood (类: 揭开引擎盖) ; Namespace Dictionaries: Review (命名空间字典: 回顾) ; Slots: Attribute Declarations (slots (插槽) : 属性 Declarations)
- class-level defaults (类-level 默认值) : Slot usage rules (slot 使用规则)
- declaring (声明) : Slot basics (slot 基础)
- descriptors (描述符) : Descriptors and slots and more (描述符、slots 及更多)
- generic programs (通用程序) : Handling slots and other "virtual" attributes generically (Handling slots (插槽) 与 other "virtual" 属性 generically)

- ListTree class (ListTree 类) : Example impacts of slots: ListTree and mapattrs (示例 impacts 的 slots (插槽) : ListTree 与 mapattrs)
- multiple inheritance (多重继承) : Slot usage rules (slot 使用规则)
- multiple inheritance classes (多重继承类) : Example impacts of slots: ListTree and mapattrs (示例 impacts 的 slots (插槽) : ListTree 与 mapattrs)
- namespaces dictionaries and (命名空间字典与) : Slot basics (slot 基础) ; Slots and namespace dictionaries (slots (插槽) 与命名空间字典)
- single-inheritance trees (single-继承 trees) : Example impacts of slots: ListTree and mapattrs (示例 impacts 的 slots (插槽) : ListTree 与 mapattrs)
- speed (速度) : What about slots speed? (What about slots (插槽) 速度?)
- subclasses (子类) : Slot usage rules (slot 使用规则)
- superclasses (超类) : Slot usage rules (slot 使用规则)
- usage rules (使用规则) : You shouldn't normally use slots (You shouldn't normally 使用 slots (插槽))
- slots attribute (slots 属性) : Multiple slot lists in superclasses (Multiple slot 列表中的超类)

- smartphone apps (智能手机应用) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- Smartphone Apps (智能手机应用)
- SOAP (SOAP 协议) : Component Integration (组件集成)
- sockets (套接字) : Other File Tools (其它文件工具)
- software quality of Python (软件质量的 Python) : Why Do People Use Python? (为什么人们使用 Python?) ; Software Quality (软件质量)
- software reuse (software 复用) : Why Do People Use Python? (为什么人们使用 Python?)
- spaces, indentation (spaces, 缩进) : and, Why Indentation Syntax? special characters, escape sequences, Escape Sequences Are Special Characters- Escape Sequences Are Special Characters special methods , Static and Class Methods special-case syntax, Advanced match Usage Sphinx, Beyond Docstrings: Sphinx split method, More String Methods: Parsing Text Spyder, Other IDEs for Python (与, Why 缩进语法? special characters, escape 序列, 转义序列是特殊字符 - 转义序列是特殊字符 special 方法, 静态方法与类方法特殊情况- 语法, match 高级用法 Sphinx, Beyond 文档字符串: Sphinx split 方法, More 字符串方法: 解析文本 Spyder

er, 其他 Python IDE)

- Spyder (Spyder 科学计算 IDE) : Other IDEs for Python (其他 Python IDE)

- SQLAlchemy (SQLAlchemy 对象关系映射) : Database Access (数据库访问)

- SQLite (SQLite 数据库) : Database Access (数据库访问)

- SQLAlchemy (SQLAlchemy 对象关系映射) : Database Access (数据库访问)

- Stackless (Stackless 无栈 Python) : Python Implementation Alternatives (Python 实现替代方案)

- standalone executables (独立可执行文件) : Standalone Executables (独立可执行文件) ; Other Launch Options (其它启动选项)

- Standalone Apps and Executables-Etcetera (A.7 独立应用与可执行程序 -A.8 其他)

- standard library (标准库) : Why Do People Use Python? (为什么人们使用 Python?) ; It' s Portable (它可移植)

- directories, module search path (目录, 模块 search path) : Search-Path Components (搜索路径的组成部分)

- methods (方法) : All String Methods (Today) (全部字符串方法 (今日))

- modules (模块) : Standard-Library Modules (标准库模块)
- standard manuals (标准手册) : The Standard Manuals (标准手册) ; Web Resources (网络资源)
- standard output streams (标准输出流) : print operations (print 操作) ; Print Operations (print 操作)
- standard streams (标准流) : Other File Tools (其它文件工具)
- startup-speed optimization, bytecode (startup-速度优化, 字节码)
- descriptors (描述符) : Using State Information in Descriptors (在描述符中使用状态信息)
- event handlers (事件处理器) : Function Interfaces and Callback-Based Code (函数 Interfaces 与回调-Based Code)
- exception classes (异常类) : Exception Objects (异常对象)
- exceptions, built-in (异常, 内置-) : Default Printing and State (默认打印与状态)
- custom (自定义) : Custom State and Behavior (自定义状态与行为) ; Providing Exception Methods (提供异常方法)
- instances (实例) : Using State Information in Descriptors (在描述符中使用状态信息)

- retaining (保留) : State with class-instance attributes (状态带类-实例属性) ; State with function attributes (状态带函数属性)
- classes (类) : Classes: Changeable, Per-Call, OOP (类: 可修改、每调用独立、OOP)
- function attributes (函数属性) : Function Attributes: Changeable, Per-Call, Explicit (函数属性: 可修改、每调用独立、显式)
- -Function Attributes: Changeable (函数属性: 可变)
- global scope (全局 scope) : Globals: Changeable but Shared (全局变量: 可修改但共享)
- nonlocal statement (nonlocal 语句) : Nonlocals: Changeable, Per-Call, LEGB (nonlocal: 可修改、每调用独立、LEGB)
- scope mutables (scope 可变对象) : Function Attributes: Changeable, Per-Call, Explicit (函数属性: 可修改、每调用独立、显式)
- suspension (挂起) : State suspension (状态 suspension)
- state information, function objects (状态 information, 函数对象) : Function Attributes (函数属性)
- statements (语句) : Introducing Python (介绍 Python)

- as expression-based equivalents (as 表达式-based equivalents) : How (Not) to Obfuscate Your Python Code (How (Not) 到 Obfuscate Your Python Code)
- assignment statements (assignment 语句) : Python's Statements (Python 的语句)
- assignments (赋值总论) : Assignment Syntax Forms (赋值语法形式)
- async (异步) : Advanced Function Tools (高级函数工具)
- await (await (等待)) : Advanced Function Tools (高级函数工具)
- class (类) : User-Defined Objects (用户定义- 对象) ; OOP: The Big Picture (OOP: 大图景) ; The class Statement (class 语句)
- attributes (属性) : Example: Class Attributes (示例: 类属性)
- syntax (语法) : General Syntax and Usage (一般语法与用法)
- compound statements (compound 语句) : Missing Keys: if Tests (Missing 键: if Tests) ; What Python Adds (Python 增加了什么)
- concept hierarchy (A.7.1 概念: 把 Python 打包成普通程序 hierarchy) : The Python Conceptual Hierarchy Revisited (重访 Python 概念层级)

- def (def (函数定义)) : Basic Function Tools (基本函数工具) ; def Statements (def 语句) ; def Executes at Runtime (def 在运行时执行) ; Calls (调用)
- executable (可执行文件)
- from (from (导入自)) : Imports Are Runtime Assignments (导入是运行时赋值)
- import (import (导入)) : Imports Are Runtime Assignments (导入是运行时赋值)
- expression statements (表达式语句) : Expression Statements (表达式语句)
- expressions in (表达式中的) : The Python Conceptual Hierarchy (Python 概念层级)
- for (用于) : Basic Operations (基本操作)
- from (from (导入自)) : The from Statement (from 语句)
- potential problems (潜在问题) : Potential Pitfalls of the from Statement (from 语句的潜在陷阱) ; When import is required (何时必须用 import)
- from (from (导入自))
- function-related (函数-related) : Function Basics (函数基础)
- global (全局) : Advanced Function Tools (高级函数工具)
- namespaces (命名空间) : The global Statement (

global 语句)

- if statements (if 语句) : A Tale of Two ifs (两种 if 的故事) ; if Statements (if 语句)
- multiple-choice selections (多选逻辑) : Multiple-Choice Selections (多选逻辑) ; Handling larger actions (处理更大型动作)
- if/then/else (if/then/else 分支) : The if/else Ternary Expression (if/else 三元表达式)
- import (import (导入)) : The import Statement (import 语句)
- indentation (缩进) : Why Indentation Syntax? (为什么采用缩进语法?)
- interactive loops (交互式循环) : A Simple Interactive Loop (简单交互循环) ; Nesting (嵌套)
- match statements (match 语句) : Missing Keys: if Tests (Missing 键: if Tests) ; match Statements (match 语句)
- attribute patterns (属性 patterns) : Advanced match Usage (match 高级用法)
- if statements (if 语句) : Match versus if live (Match 对比 if live)
- instance patterns (实例 patterns) : Advanced match Usage (match 高级用法)
- literal patterns (字面量模式) : Advanced match Usage (match 高级用法)

- mapping patterns (映射 patterns) : Advanced match Usage (match 高级用法)
- nested patterns (嵌套 patterns) : Advanced match Usage (match 高级用法)
- nesting (嵌套) : Basic match Usage (match 基本用法)
- parentheses (圆括号) : Advanced match Usage (match 高级用法)
- sequence patterns (序列 patterns) : Advanced match Usage (match 高级用法)
- module imports (模块导入) : How Files Generate Namespaces (文件如何生成命名空间)
- nested statements (嵌套语句)
- blocks (代码块) : What Python Adds (Python 增加了什么)
- special cases (特殊情况) : Block rule special case (块规则特例)
- nonlocal (nonlocal (非局部)) : Advanced Function Tools (高级函数工具) ; The nonlocal Statement (nonlocal 语句) ; nonlocal Boundary Cases (nonlocal 边界情形)
- order (顺序) : Statement Order Matters in Top-Level Code (语句 Order Matters 中的顶层- Code)
- parentheses (圆括号) : Parentheses are optional (括号可选)

- raise (raise (引发)) : Breaking Out of Multiple Nested Loops: “go to” (跳出多层嵌套循环: “goto”)
- return (return (返回)) : Basic Function Tools (基本函数工具) ; return Statements (return 语句)
- semicolon (分号) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- semicolons (分号) : End-of-line is end of statement (End-的-line is end 的语句)
- special cases rules (特殊情况规则) : Statement rule special cases (语句规则特殊情况)
- syntax (语法) : Python' s Statements (Python 的语句) ; Python Syntax Revisited (Python 语法回顾)
- blocks (代码块) : Block Delimiters: Indentation Rules (块定界: 缩进规则)
- delimiters (分隔符) : Statement Delimiters: Lines and Continuations (语句 Delimiters: Lines 与 Continuations)
- indentation (缩进) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- special syntax (special 语法) : Special Syntax Cases in Action (Special 语法 Cases 中的 Action)
- truth values (truth 值) : Truth Values Revisited (真值回顾)

- try (try (尝试)) : Handling Errors with try Statements (Handling Errors 带 try 语句)
- yield (yield (产出)) : Advanced Function Tools (高级函数工具) ; Iteration protocol integration (与迭代协议的集成)
- static methods (static 方法) : Other Method-Call Possibilities (其他方法调用可能) ; Static and Class Methods (静态方法与类方法) ; Why the Special Methods? (Why Special 方法?) ; Using Static and Class Methods (Using 静态方法与类方法)
- alternatives (替代方案) : Static Method Alternatives (Static 方法替代方案)
- instance counting (实例 counting) : Counting Instances with Static Methods (Counting 实例带 Static 方法)
- staticmethod function (staticmethod 函数) : Using Static and Class Methods (Using 静态方法与类方法)
- statistically nested scopes (statistically 嵌套作用域) : Nested Functions and Scopes (嵌套函数与作用域)
- stdout (标准输出) : Test Your Knowledge: Answers (测验答案)
- STEM (science, technology, engineering, and math) (STEM (science, technology, engineering, 与 math))

-) : Why Do People Use Python? (为什么人们使用 Python?)
- storage (存储)
 - class (类) : Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
 - objects (对象)
 - CSV module (CSV 模块) : Storing Objects with Other Tools (用其它工具存储对象)
 - JSON (JSON 格式) : Storing Objects with JSON (用 JSON 存储对象)
 - pickle module (pickle 模块) : Storing Objects with pickle (用 pickle 存储对象)
 - struct module (struct 模块) : Storing Objects with Other Tools (用其它工具存储对象)
 - per-instance (per-实例) : Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
 - str function (str 函数) : Numbers (数字)
 - str method (str 方法) : Why Two Display Methods? (Why Two Display 方法?) ; Display Usage Notes (Display 用法 Notes)
 - str object (str 对象) : The str Object (str 对象)
 - stream processors, composition (流处理器, 组合) : Stream Processors Revisited (再访流处理器)
 - streams (流)

- redirection, shells (重定向与 shell) : Command-Line Usage Variations (命令行用法变体)
- shell command (shell 命令) : Other File Tools (其它文件工具)
- string formatting (字符串格式化) : String Formatting: The Triathlon (字符串格式化: 铁人三项) ; Formatting (格式化)
- expressions (表达式) : String-Formatting Options (字符串-格式化 Options)
- advanced (高级) : Advanced formatting expression examples (高级格式化表达式示例)
- custom formats (自定义格式) : Formatting expression custom formats (格式化表达式自定义格式) ; Formatting (格式化)
- expression custom formats (表达式自定义格式)
- dictionary-based (字典-based) : Dictionary-based formatting expressions (字典-based 格式化表达式) ; Dictionary based formatting expressions (字典 based 格式化表达式)
- format method (format 方法) : String-Formatting Options (字符串-格式化 Options) ; The String-Formatting Method (字符串格式化方法)
- advanced (高级) : Advanced formatting method examples (高级格式化方法示例)

- attributes (属性) : Adding keys, attributes, and offsets (添加键、属性与偏移)
- binary formats (二进制格式) : Advanced formatting method examples (高级格式化方法示例)
- custom formats (自定义格式) : Formatting method custom formats (格式化方法自定义格式) ; Formatting (格式化)
- method custom formats (方法自定义格式)
- f-string literal (f-字符串 literal) : The F-String Formatting Literal (F-String 格式化字面量) ; Advanced f-string examples (高级 f-string 示例)
- floating-point numbers (浮点- 数字) : Advanced formatting method examples (高级格式化方法示例)
- format spec component (format spec 组件) : Formatting method custom formats (格式化方法自定义格式)
- hex formats (十六进制格式) : Advanced formatting method examples (高级格式化方法示例)
- keys (键) : Adding keys, attributes, and offsets (添加键、属性与偏移)
- octal formats (八进制格式) : Advanced formatting method examples (高级格式化方法示例)
- offsets (偏移量) : Adding keys, attributes, and offsets (添加键、属性与偏移)

- substitution targets (替换目标) : Formatting method custom formats (格式化方法自定义格式)
- literals (字面量) : String-Formatting Options (字符串-格式化 Options)
- manual (手册/手动) : String-Formatting Options (字符串-格式化 Options)
- repr method (repr 方法) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str) ; Display Usage Notes (Display 用法 Notes)
- str method (str 方法) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str) ; Display Usage Notes (Display 用法 Notes)
- string indexing (字符串索引) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习) ; Part II, Objects and Operations (第 II 部分, 对象与操作)
- string interpolation (字符串 interpolation) : The F-String Formatting Literal (F-String 格式化字面量)
- string literals (字符串字面量) : Literals and Basic Properties (字面量与基本属性)
- backslash escape sequences and (反斜杠转义序列与) : Other Ways to Code Strings (Other Ways 到 Code 字符串)
- docstrings (文档字符串) : Docstrings and doc (文档字符串与 doc)

- format (格式) : String Literals (字符串字面量)
- operations (操作) : String Object Basics (字符串对象基础)
- quotes (引号) : Single and Double Quotes Are the Same (Single 与 Double Quotes Are Same)
- UTF-8 encoding (UTF-8 编码) : Source-File Encoding Declarations (源文件编码声明)
- string methods (字符串方法) : String Methods (字符串方法) ; “Changing” Strings, Part 2: String Methods (“Changing” 字符串, Part 2: 字符串方法)
- find, “Changing” (find, “Changing”) (find, “更改” (find, “更改”)) : Strings (字符串)
- join, “Changing” (join, “Changing”) (join, “更改” (join, “更改”)) : Strings (字符串)
- method calls, syntax (方法调用, 语法) : Method Call Syntax (方法 Call 语法)
- parsing text (解析文本) : More String Methods: Parsing Text (More 字符串方法: 解析文本)
- replace, “Changing” (replace, “Changing”) (replace, “更改” (replace, “更改”)) : Strings (字符串)
- split (拆分) : More String Methods: Parsing Text (More 字符串方法: 解析文本)
- substring test (子串测试) : Other Common String Methods (其他常用字符串方法)

- whitespace (空白) : Other Common String Methods (其他常用字符串方法)
- string objects (字符串对象) : The raise Statement (raise 语句)
- string representation methods (字符串表示: repr 和 str 方法) : String Representation: repr and str (字符串表示: repr 和 str: repr 与 str) ; Display Usage Notes (Display 用法 Notes)
- strings (字符串) : Test Your Knowledge: Answers (测验答案) ; String Fundamentals (字符串 Fundamentals) ; Types (类型)
- as immutable sequences (as 不可变序列) : String Object Basics (字符串对象基础)
- backslash characters (反斜杠字符) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- backslash escape (反斜杠转义) : Other Ways to Code Strings (Other Ways 到 Code 字符串)
- block strings (block 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- character arrays and (character arrays 与) : String Object Basics (字符串对象基础)
- code points (码点) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- combining (组合) : String Object Basics (字符串对象基础)

- comparisons (比较运算: lt、gt 等) : String comparisons (字符串比较运算: lt、gt 等) ; Comparisons, Equality, and Truth (比较、相等与真值)
- concatenation (拼接) : String Object Basics (字符串对象基础) ; Single and Double Quotes Are the Same (Single 与 Double Quotes Are Same) ; Basic Operations (基本操作)
- conversion (转换) : String Conversion Tools (字符串 Conversion 工具)
- character-code (字符码) : Character-code conversions (字符码转换)
- floating-point numbers (浮点- 数字) : String Conversion Tools (字符串 Conversion 工具)
- strings to numbers (字符串到数字) : String Conversion Tools (字符串 Conversion 工具)
- delimiter, "Changing" (分隔符, "Changing") : Strings (字符串)
- docstrings (文档字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- empty (空) : Files in Action (文件实战)
- escape characters (转义字符) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- expressions (表达式) : String Object Basics (字符串对象基础)

- fetching by offset (按偏移量读取) : String Object Basics (字符串对象基础)
- help resources (帮助资源) : Getting Help (获取帮助)
- immutability (不可变性) : Immutability (不可变性)
- indexing (索引) : String Object Basics (字符串对象基础) ; Indexing and Slicing (索引与切片: getitem 和 setitem) ; Extended slicing: (扩展切片:)
- iteration (迭代) : Basic Operations (基本操作)
- loops and (循环与) : Basic Operations (基本操作)
- methods (方法) : Type-Specific Methods (类型-Specific 方法) ; String Object Basics (字符串对象基础)
- modules (模块) : String Object Basics (字符串对象基础)
- multiline (多行) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- newline characters (换行符) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- NULL character (NULL 字符) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- operations (操作) : Basic Operations (基本操作)

- operator overload and (运算符 overload 与) : Basic Operations (基本操作)
- polymorphism and (多态与) : Basic Operations (基本操作)
- quote characters (引号字符) : Other Ways to Code Strings (Other Ways 到 Code 字符串)
- raw (原始) : Raw Strings Suppress Escapes (Raw 字符串抑制转义)
- section extraction (章节提取) : String Object Basics (字符串对象基础)
- sequence operations, "Changing" (序列操作, "Changing") : Strings (字符串)
- arbitrary expressions (任意表达式) : Sequence Operations (序列操作)
- indexing expressions (索引表达式) : Sequence Operations (序列操作)
- sequences (序列) : String Object Basics (字符串对象基础)
- slicing (切片) : String Object Basics (字符串对象基础) ; Indexing and Slicing (索引与切片: getitem 和 setitem) ; Extended slicing: (扩展切片:)
- extended slicing (扩展切片) : Extended slicing: The third limit and slice objects (扩展切片: third limit 与 slice 对象)

- split method (split 方法) : Storing Objects with Conversions (用转换来存储对象)
- strip method (strip 方法) : Storing Objects with Conversions (用转换来存储对象)
- substrings, testing (substrings, 测试)
- text strings (text 字符串) : Unicode Strings (Unicode 字符串)
- triple-quotes (三引号) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- Unicode (Unicode 统一码) : Unicode Strings (Unicode 字符串) ; String Fundamentals (字符串 Fundamentals)
- struct module (struct 模块) : Test Your Knowledge: Answers (测验答案)
- binary data (二进制数据) : The struct Binary-Data Module (struct 二进制数据- 模块)
- object storage (对象 storage) : Storing Objects with Other Tools (用其它工具存储对象)
- structural pattern matching (结构化模式匹配) : match Statements (match 语句)
- sub method, operator overloading (sub 方法, 运算符重载) : Constructors and Expressions: init and sub (构造器与表达式: init 和 sub: init 与 sub)
- subclasses (子类) : Classes Are Customized by Inheritance (类通过继承进行定制)

- metaclasses (元类) : Metaclasses Are Subclasses of type (元类是 type 的子类)
- slots (slots (插槽)) : Slot usage rules (slot 使用规则)
- subclassing (子类化) : Coding Subclasses (编码子类) ; Part VI, Classes and OOP (第 VI 部分, 类与 OOP (面向对象编程))
- built-in types extension (内置- 类型扩展) : Extending Types by Subclassing (扩展 (重复/连接) 类型 by Subclassing)
- extending code (扩展 (重复/连接) code) : Inherit, Customize, and Extend (Inherit, Customize, 与 Extend)
- hierarchies (层次结构) : OOP: The Big Idea (OOP : 大思想)
- inheritance (继承) : Inherit, Customize, and Extend (Inherit, Customize, 与 Extend) ; The Inheritance Bifurcation (继承 Bifurcation)
- instances, namespace dictionaries (实例, 命名空间字典) : Namespace Dictionaries: Review (命名空间字典: 回顾)
- methods (方法) : Augmenting Methods: The Bad Way (增强方法: 糟糕做法) ; Augmenting Methods: The Good Way (增强方法: 正确做法)
- polymorphism (多态) : Polymorphism in Action (

多态中的 Action)

- subprime code (次优代码) : Loop else (循环 else)
- subroutines (子程序) : Why Use Functions? (为什么要用函数?)
- substrings, testing (substrings, 测试)
- argument lists (argument 列表) : Same argument lists (相同的参数列表)
- attribute-fetch algorithm (属性获取算法) : Attribute-fetch algorithm (属性获取算法)
- call-chain anchors (调用链锚点) : Call-chain anchors (调用链锚点)
- cooperator method dispatch (cooperator 方法 dispatch) : Call-chain anchors (调用链锚点)
- deployment (部署) : Universal deployment (通用部署)
- MRO algorithm (MRO 算法) : A “magic” proxy (“魔法” 代理)
- noncalls (非调用) : Noncalls and operator overloading (Noncalls 与运算符重载)
- operator overloading (运算符重载) : Noncalls and operator overloading (Noncalls 与运算符重载)
- superclasses (超类) : Attribute Inheritance Search (属性继承搜索) ; Classes Are Customized by Inheri

tance (类通过继承进行定制)

- abstract (抽象) : Abstract Superclasses (抽象超类) ; Preview: Abstract superclasses with library tools (预览: 用库工具实现抽象超类) ; Stream Processors Revisited (再访流处理器)
- library tools (库工具) : Preview: Abstract superclasses with library tools (预览: 用库工具实现抽象超类)
- constructors (构造器) : Step 5: Customizing Constructors, Too (第五步: 同样定制构造函数) ; Miscellaneous Class Gotchas (Miscellaneous 类的陷阱)
- exception superclass (异常超类) : Coding Exceptions Classes (编写异常类)
- exceptions (异常) : Exception Classes (异常类)
- interfacing with (interfacing 带) : Class Interface Techniques (类接口技术)
- metaclasses comparison (元类 comparison) : Metaclass Versus Superclass (元类 vs 超类)
- method calls (方法调用) : Specializing Inherited Methods (特化继承方法)
- operator overloading method coding (运算符重载方法编码) : Workaround: Coding operator- overloading methods in superclasses (Workaround: 编码运算符重载- 方法中的超类)
- slots (slots (插槽)) : Slot usage rules (slot 用法规则)

- slots attribute (slots 属性) : Multiple slot lists in superclasses (Multiple slot 列表中的超类)
- support (支持) : Why Do People Use Python? (为什么人们使用 Python?)
- SWIG (SWIG 接口生成器) : Component Integration (组件集成)
- syntactic nesting, exception handlers (语法嵌套, 异常 handlers) : Example: Syntactic Nesting (示例: 语法嵌套)
- syntax (语法)
- argument matching (参数匹配) : Argument Matching Syntax (参数匹配语法)
- assignments (赋值总论) : Application to for loops (应用于 for 循环)
- class statement (类语句) : General Syntax and Usage (一般语法与用法)
- combined clauses (组合子句)
- compound statements (compound 语句) : What Python Adds (Python 增加了什么)
- comprehension syntax (comprehension 语法) : Comprehensions and (推导式与)
- decorators (装饰器) : Why Decorators? (为什么使用装饰器?)
- dictionary comprehensions (字典推导式) : Formal

Comprehension Syntax (推导式的正式语法)

- dotted-path (点分路径) : Package Imports (包导入) ; Using the basic package (使用基础包)
- relative imports (相对导入) : Relative-Import Rationales and Trade-Offs (相对导入的理由与权衡)
- function decorator (函数 decorator) : Property basics (property (属性) 基础)
- function decorators (函数装饰器)
- instance generation (实例 generation) : Classes Are Types Are Classes (类 Are 类型 Are 类)
- lambda expression (lambda 表达式) : lambda Basics (lambda 基础)
- list comprehensions, formal (列表推导式, formal) : Formal Comprehension Syntax (推导式的正式语法)
- method calls (方法调用) : Method Call Syntax (方法 Call 语法)
- set comprehensions (集合推导式) : Formal Comprehension Syntax (推导式的正式语法)
- special-case (特殊情况) : Advanced match Usage (match 高级用法)
- statements (语句) : Python' s Statements (Python 的语句) ; Python Syntax Revisited (Python 语法回顾)

- block rule special cases (block 规则特殊情况) : Block rule special case (块规则特例)
- blocks (代码块) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- delimiters (分隔符) : Statement Delimiters: Lines and Continuations (语句 Delimiters: Lines 与 Continuations)
- indentation (缩进) : End of indentation is end of block (缩进结束即代码块结束) ; Why Indentation Syntax? (为什么采用缩进语法?) ; Block Delimiters: Indentation Rules (块定界: 缩进规则)
- Syntax? (语法?) : Block Delimiters: Indentation Rules (块定界: 缩进规则) ; Avoid mixing tabs and spaces (避免混用 Tab 与空格)
- nested statement blocks (嵌套语句 blocks) : What Python Adds (Python 增加了什么)
- parentheses (圆括号) : Parentheses are optional (括号可选)
- semicolons (分号) : End-of-line is end of statement (End-的-line is end 的语句)
- special cases rules (特殊情况规则) : Statement rule special cases (语句规则特殊情况)
- special syntax (special 语法) : Special Syntax Cases in Action (Special 语法 Cases 中的 Action)

- tuples (元组) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- variable names (variable 名字) : Variable Name Rules (变量命名规则)
- sys.excinfo (sys.excinfo 异常信息) : More on sys.excinfo (深入 sys.excinfo) ; The sys.exception alternative—and diss (sys.exception 替代方案——以及吐槽)
- sys.platform (sys.platform 平台标识) : A First Script (第一个脚本)
- systems programming (系统编程) : Systems Programming (系统编程)

T (字母 T)

- tabs, indentation (tabs, 缩进)
- replacements, “Changing” (替换, “Changing”) : Strings (字符串)
- try/finally and (try/finally 与) : Example: Coding termination actions with try/finally (示例: 用 try/finally 编写终止动作)
- with statement (带语句) : The with Statement and Context Managers (with 语句与上下文管理器)
- termination handlers (终止处理器) : The Termination-Handlers Shoot-Out (终止处理器巅峰对决) ; The Termination- Handlers Shoot-Out (终止处理器巅峰对

决)

- testdriver function (testdriver 函数) : Running In-Process Tests (运行中的-Process Tests)
- tester, slicing sequences (tester, slicing 序列) : Tester client (测试客户端)
- testing (测试) : And More: AI, Games, Images, QA, Excel, Apps... (还有更多: AI、游戏、图像、QA、Excel、应用...)
- class creation (类 creation) : Testing as You Go (测试 as You Go)
- comparisons (比较运算: lt、gt 等) : Comparisons, Equality, and Truth (比较、相等与真值) ; Dictionary comparisons (字典比较)
- equality (相等) : Comparisons, Equality, and Truth (比较、相等与真值)
- in-process tests, exceptions (中的-process tests, 异常) : Running In-Process Tests (运行中的-Process Tests)
- interactive tests (交互式 tests) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- user input, error handling (用户输入与错误处理) : Handling Errors by Testing Inputs (Handling Errors by 测试 Inputs)
- testing tools (测试工具) : Development Tools for Larger Projects (更大型项目的开发工具)

- text editors (文本编辑器) : Other Launch Options (其它启动选项)
- BOM (byte order marker) (BOM (字节顺序标记)) : Making BOMs in Text Editors (Making BOMs 中的文本编辑器)
- indentation (缩进) : Why Indentation Syntax? (为什么采用缩进语法?)
- text files (text 文件) : Text and Binary Files: The Short Story (文本与二进制文件: 速览) ; Text and Binary Files: (文本文件与二进制文件:)
- binary mode (二进制模式) : Text and Binary Modes (Text 与 Binary Modes)
- bytes object (字节串对象) : Text and Binary Modes (Text 与 Binary Modes)
- conversions (转换) : Storing Objects with Conversions (用转换来存储对象)
- iterators (迭代器) : Using Files (使用文件)
- lines comparison (行比较) : The Termination-Handlers Shoot-Out (终止处理器巅峰对决)
- pathname (路径名) : Text-File Basics (Text-文件基础)
- text mode (文本模式) : Text and Binary Modes (Text 与 Binary Modes)
- newline character (换行符) : Text and Binary Modes (Text 与 Binary Modes)

- Unicode text files (Unicode 文本文件) : Unicode-Text Files (Unicode 文本文件)
- text output file (文本输出文件) : Files (文件)
- text strings (text 字符串)
- literals (字面量) : Literals and Basic Properties (字面量与基本属性)
- non-ASCII characters (非 ASCII 字符) : Unicode Strings (Unicode 字符串)
- type conversions (类型转换) : String Type Conversions (字符串类型转换)
- Unicode code-points (Unicode 码点) : Unicode Strings (Unicode 字符串)
- Unicode text (Unicode 文本) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- text-mode files (文本模式- 文件)
- BOM headers (BOM 头) : Using Text and Binary Files (使用文本与二进制文件)
- newline character (换行符) : Text and Binary Modes (Text 与 Binary Modes)
- textual data (文本数据) : Introducing Python String Tools (介绍 Python 字符串工具)
- third-party domain (第三方- domain) : Why Do People Use Python? (为什么人们使用 Python?)
- third-party utilities (第三方- 工具) : It's Powerful

(它很强大)

- timeit module (timeit 模块)
- API-calls mode (API 调用模式) : API-calls mode (API 调用模式)
- benchmarking automation (基准测试自动化) : Automating timeit Benchmarking (自动化 timeit 基准测试)
- benchmark module (基准测试模块) : Benchmark module (基准测试模块) ; Benchmark script (基准测试脚本)
- benchmark script (基准测试脚本) : Benchmark script (基准测试脚本) ; Timing individual Pythons (对单个 Python 计时)
- dictionaries (字典) : Timing set and dictionary iterations (测试集合与字典迭代) ; Conclusion: Comparing tools (结论: 比较工具)
- individual Pythons (多个独立 Python) : Timing individual Pythons (对单个 Python 计时)
- multiple Pythons (多个 Python) : Timing multiple Pythons (对多个 Python 计时)
- sets (集合) : Timing set and dictionary iterations (测试集合与字典迭代) ; Conclusion: Comparing tools (结论: 比较工具)
- callable objects (callable 对象) : Basic timeit Usage (timeit 基本用法)

- command-line mode (命令行模式) : Command-line mode (命令行模式)
- multiline statements (multiline 语句) : Handling multiline statements (处理多行语句)
- setup code (启动/设置代码) : Other timeit usage modes (timeit 的其他用法模式)
- sort speed (sort 速度) : Timing sort speed (测试排序速度)
- statement strings (语句字符串) : Basic timeit Usage (timeit 基本用法)
- timer decorator (计时装饰器) : Adding Decorator Arguments (添加装饰器参数)
- timer utility functions (timer utility 函数) : Timer Module: Take 2 (计时模块: 第二版)
- timestamp-based bytecode files (timestamp-based 字节码文件) : Step 2: Compile It (Maybe) (第二步 : 编译 (或许))
- timing tools (timing 工具) : Part IV, Functions and Generators (第 IV 部分: 函数与生成器)
- Toga (Toga 原生 UI 工具包) : GUIs and UIs (GUI 与 UI) ; Standalone Apps and Executables (A.7 独立应用与可执行程序)
- tools (工具)
- built-in (内置) : It' s Powerful (它很强大)

- increase in (increase 中的) : The Python Tsunami (Python 海啸)
- toolsets (工具集)
- built-in tools (内置- 工具) : The Python Toolset (Python 工具组)
- development tools (development 工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- debuggers (调试器) : Development Tools for Larger Projects (更大型项目的开发工具)
- documentation tools (文档工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- error-checking tools (error-checking 工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- IDEs (集成开发环境) (IDE (集成开发环境) (IDE)) : Development Tools for Larger Projects (更大型项目的开发工具)
- installation management (A.3.1 安装: 预装迷局与 python.org 安装器 management) : Development Tools for Larger Projects (更大型项目的开发工具)
- optimization (优化) : Development Tools for Larger Projects (更大型项目的开发工具)
- profilers (性能分析器) : Development Tools for Larger Projects (更大型项目的开发工具)

- shipping (发布交付) : Development Tools for Larger Projects (更大型项目的开发工具)
- testing tools (测试工具) : Development Tools for Larger Projects (更大型项目的开发工具)
- extensions (扩展) : The Python Toolset (Python 工具组)
- traceback object (traceback 对象) : Displaying Errors and Tracebacks (显示错误与回溯信息)
- triple-quoted strings (triple-quoted 字符串) : Triple Quotes and Multiline Strings (三引号与 Multiline 字符串)
- truth values (truth 值) : The Meaning of True and False in Python (Python 中 True 与 False 的含义) ; The bool type (bool 类型) ; Truth Values Revisited (真值回顾)
- try statement (try 语句)
- built-in exceptions (内置- 异常) : Example: Catching built-in exceptions (示例: 捕捉内置异常)
- combined clauses (组合子句) : Combined try Clauses (组合 try 子句)
- nesting (嵌套) : Combining finally and except by nesting (通过嵌套组合 finally 与 except)
- syntax (语法) : Combined-clause syntax rules (组合子句的语法规则)

- debugging (调试) : Debugging with Outer try Statements (调试带 Outer try 语句)
- default exception behavior (default 异常 behavior) : Example: Default behavior (示例: 默认行为)
- else clause (else 子句) : The except and else Clauses (except 与 else 子句) ; Example: Catching built-in exceptions (示例: 捕捉内置异常)
- error handling (错误处理) : Handling Errors with try Statements (Handling Errors 带 try 语句)
- except clause (except 子句) : The except and else Clauses (except 与 else 子句) ; Example: Catching built-in exceptions (示例: 捕捉内置异常)
- except clause (except 子句) : Exception Groups: Yet Another Star! (异常组: 又一颗新星!)
- finally block (finally 块) : Combined try Clauses (组合 try 子句)
- finally clause (finally 子句) : The finally Clause (finally 子句) ; Example: Coding termination actions with try/finally (示例: 用 try/finally 编写终止动作)
- forms (形式) : try Statement Clauses (try 语句的子句)
- handlers (处理器) : The try Statement (try 语句)
- nesting (嵌套) : Nesting Exception Handlers (嵌套异常处理器)

- tuples (元组) : Catching many exceptions with a tuple (用元组捕捉多个异常)
- try/finally (try/finally 清理结构) : Combined try clauses (组合 try 子句)
- nesting (嵌套) : Combining finally and except by nesting (通过嵌套组合 finally 与 except)
- syntax (语法) : Combined-clause syntax rules (组合子句的语法规则)
- termination actions and (终止动作与) : Example: Coding termination actions with try/finally (示例: 用 try/finally 编写终止动作)
- tuples (元组) : Tuples (元组) ; Why Tuples? (Why 元组?) ; Test Your Knowledge: Answers (测验答案) ; Types (类型)
- assignments (赋值总论) : Assignment Syntax Forms (赋值语法形式)
- commas (逗号) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- commas in expressions (逗号中的表达式) : Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- compared to lists (compared 到列表) : Why Lists and Tuples? (为什么需要列表和元组?)
- conversions (转换) : Conversions, methods, and immutability (转换、方法与不可变性)

- converting from dictionaries (converting from 字典) : Records Revisited: Named Tuples (再看记录: 命名元组)
- for loops (for 循环) : Tuple (sequence) assignment in for loops (元组 (序列) assignment 中的 for 循环)
- generating (生成) : Test Your Knowledge: Answers (测验答案)
- immutability (不可变性) : Conversions, methods, and immutability (转换、方法与不可变性)
- methods (方法) : Conversions, methods, and immutability (转换、方法与不可变性)
- named (命名的) : Records Revisited: Named Tuples (再看记录: 命名元组)
- parentheses (圆括号) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- sequence assignment (序列 assignment) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- sorting (排序) : Conversions, methods, and immutability (转换、方法与不可变性)
- syntax (语法) : Tuple syntax peculiarities: Commas and parentheses (元组语法特性: 逗号与括号)
- try statement (try 语句) : Catching many exceptions with a tuple (用元组捕捉多个异常)

- TurboGears (TurboGears Web 框架) : Internet and Web Scripting (互联网与 Web 脚本)
- type calls, comprehensions (类型调用, 推导式) : Comprehensions versus type calls and generators (推导式对比类型调用与生成器)
- type conversions, text strings (类型转换, text 字符串) : String Type Conversions (字符串类型转换)
- type function (类型函数) : Types (类型)
- type hinting (类型 hinting) : Type Hinting (类型 Hinting) ; Type Hinting: Optional, Unused, and Why? (类型标注: 可选、不使用、为什么?)
- type objects (类型对象) : Types (类型) ; Type Objects (类型对象)
- built-ins (内置) : Other Types in Python (Python 中的其它类型)
- type-specific operations, lists (类型-specific 操作, 列表) : Type-Specific Operations (类型-Specific 操作)
- types (类型) : It's Powerful (它很强大)
- built-ins, extending (内置-, 扩展 (重复/连接)) : Extending Types by Embedding (扩展 (重复/连接) 类型 by 嵌入)
- classes and (类与) : The Python Object Model (Python 对象模型)

- immutable (不可变) : Mutable Types Can Be Changed in Place (可变类型 Can Be Changed 中的 Place) ; Test Your Knowledge: Part II Exercises (自测: 第二部分练习)
- mappings (映射) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- metaclass/class dichotomy (元类/类 dichotomy) : The Metaclass/Class Dichotomy (元类/类 Dichotomy)
- metaclasses (元类)
- class statements (类语句) : Class Statements Call a type (class 语句会调用 type) ; Class Statements Can Choose a type (class 语句可选择 type)
- instances (实例) : Classes Are Instances of type (类是 type 的实例)
- subclasses (子类) : Metaclasses Are Subclasses of type (元类是 type 的子类)
- mutable (可变) : Mutable Types Can Be Changed in Place (可变类型 Can Be Changed 中的 Place)
- numbers (数字) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- object types (对象类型) : Types Live with Objects, Not Variables (类型存在于对象中, 而非变量中)
- hierarchy (层次) : Python' s Type Hierarchies (Python 的类型体系)

- sequences (序列) : Types Share Operation Sets by Categories (类型 Share 操作集合 by Categories)
- types library module (类型库模块) : Type Objects (类型对象)
- TypeScript (TypeScript 语言) : Type Hinting (类型 Hinting)
- typing, dynamic (动态类型) : It' s Powerful (它很强大)

U (字母 U)

- UIs (user interfaces) (UI (用户界面)) : GUIs and UIs (GUI 与 UI)
- Unicode (Unicode 统一码) : Unicode and Byte (Unicode 与字节) ; Strings (字符串)
- character encodings (字符编码) : Character Encodings (字符编码)
- character representations (字符表示) : Character Representations (字符表示)
- code points (码点) : Escape Sequences Are Special Characters (转义序列是特殊字符) ; Character Representations (字符表示)
- value (值) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- code-points (码点) : Unicode Strings (Unicode 字符串)

- default encoding (默认编码) : Filenames in open and Other Filename Tools (文件名: open 与其它文件名工具)
- escapes (转义) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- files (文件) : Unicode and Byte Files (Unicode 与字节文件)
- normalization (规范化) : Unicode Normalization: Whither Standard? (Unicode 规范化: 标准在何方?)
- raw bytes (raw 字节串) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- strings (字符串) : Unicode Strings (Unicode 字符串) ; String Fundamentals (字符串 Fundamentals)
- code points (码点) : Unicode Strings (Unicode 字符串)
- strings, coding (字符串, 编码) : Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- text files (text 文件) : Unicode-Text Files (Unicode 文本文件)
- UTF-16 (UTF-16 编码) : Escape Sequences Are Special Characters (转义序列是特殊字符)
- union operation, dictionaries (union 操作, 字典) : Dictionary "Union" Operator (字典 "Union" 运算符)

- unit testing, name attribute (unit 测试, name 属性) : Example: Unit Tests with name (示例: 用 name 做单元测试)
- Unix (Unix 系统) : Python (Python 语言)
- boundary cases (边界情况) : Boundary cases (边界情况)
- for loops (for 循环) : Application to for loops (应用于 for 循环)
- update method (update 方法) : Missing Keys: if Tests (Missing 键: if Tests)
- usage mode flags (用法 mode flags) : Dual-Usage Modes: name and main (双用途模式: name 与 main)
- user inputs (用户输入)
- error handling (错误处理) : Handling Errors by Testing Inputs (Handling Errors by 测试 Inputs)
- math (数学) : Doing Math on User Inputs (对用户输入做数学运算)
- user interfaces (UIs) (用户界面 (UI)) : GUIs and UIs (GUI 与 UI)
- user-defined classes (用户定义- 类) : Classes Are Types Are Classes (类 Are 类型 Are 类)
- user-defined docstrings (用户定义的文档字符串) : User-defined docstrings (用户定义的文档字符串)

- user-defined exceptions, nonerror conditions (用户定义异常与非错误条件) : Functions Can Signal Conditions with raise (函数可用 raise 发出条件信号)
- user-defined function decorators (用户定义的函数装饰器) : A First Look at User-Defined Function Decorators (用户定义函数装饰器初探)
- user-defined iterables (用户定义的可迭代对象) : User-Defined Iterables (用户定义的可迭代对象) ; Multiple iterators with yield (用 yield 实现多个迭代器)
- UTF-16 encoding (UTF-16 编码) : Escape Sequences Are Special Characters (转义序列是特殊字符) ; Character Encodings (字符编码) ; Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- UTF-32 encoding (UTF-32 编码) : Character Encodings (字符编码) ; Coding Unicode Strings in Python (在 Python 中编写 Unicode 字符串)
- UTF-8 encoding (UTF-8 编码) : Character Encodings (字符编码) ; Filenames in open and Other Filename Tools (文件名: open 与其它文件名工具)
- BOM (byte order marker) (BOM (字节顺序标记)) : Making BOMs in Python (在 Python 中制作 BOM)
- string literals (字符串字面量) : Source-File Encoding Declarations (源文件编码声明)
- utilities (工具)

- library utilities (库工具) : It' s Powerful (它很强大)
- third-party (第三方) : It' s Powerful (它很强大)

V (字母 V)

- values method (values 方法) : Item Iteration: for Loops (按项迭代: for 循环)
- variables (变量) : Running Code Interactively (交互式运行代码)
- all (all (导出列表)) : Minimizing from Damage: X and all (最小化 from 的破坏: X 与 all)
- comprehensions (推导式) : Preview: Other Python scopes (预览: Python 的其他作用域)
- scopes and (作用域与) : Scopes and comprehension variables (作用域与推导式变量)
- creating (创建) : Variables, Objects, and References (变量、对象与引用)
- from statement (from 语句) : from Can Obscure the Meaning of Variables (from 可能模糊变量含义)
- global (全局)
- coupling (耦合) : Function Design Concepts (函数设计概念)
- program design (程序设计) : Program Design: Minimize Global Variables (程序设计: 尽量减少全局变量)

-)
- state, decorators (状态与装饰器)
- imports (导入) : Imports Versus Scopes (导入与作用域)
- local (局部) : Segue: Local Variables (过渡: 局部变量)
- static locals (静态局部量) : Function Attributes (函数属性)
- loops (循环) : Enclosing scopes and loop variables (外层作用域与循环变量)
- names (名字) : Module Filenames (模块文件名)
- names (名字)
- naming rules (命名规则) : Variable Name Rules (变量命名规则) ; Names have no type, but objects do (名字没有类型, 但对象有)
- references (引用) : Variables, Objects, and References (变量、对象与引用)
- scope (作用域) : Python Scopes Basics (Python 作用域基础)
- scopes (作用域) : Imports Versus Scopes (导入与作用域)
- types (类型) : Variables, Objects, and References (变量、对象与引用)
- use (用法) : Variables, Objects, and References (

变量、对象与引用)

- view objects (视图对象) : Dictionary key/value/item view objects (字典 key/value/item 视图对象)
- virtual machines (VMs) (see VMs (virtual machines) (虚拟机 (见 VMs))
- virtual sequences (虚拟序列) : Iterations (迭代)
- visibility (可见性)
- PVM (Python Virtual Machine) (PVM (Python 虚拟机)) : The Python Virtual Machine (PVM) (Python 虚拟机 (PVM))
- VSCode (VS Code 编辑器) (VS Code 编辑器 (对比 Code)) : Other IDEs for Python (其他 Python IDE)

W (字母 W)

- Wasm (WebAssembly) (WebAssembly 字节码) : WebAssembly for Browsers (浏览器中的 WebAssembly)
- web scripting (Web 脚本) : Internet and Web Scripting (互联网与 Web 脚本)
- web server scripts (Web 服务器脚本) : Other Launch Options (其它启动选项)
- WebAssembly (WebAssembly 字节码) : GUIs and UIs (GUI 与 UI) ; Internet and Web Scripting (互联网与 Web 脚本)

- while loops (while 循环) : while Loops (while 循环) ; Examples (示例)
- break statement, break-The named-assignment alternative (break 语句——海象赋值替代)
- named assignment (命名赋值 (海象)) : The named-assignment alternative (海象赋值替代方案)
- break statements (break 语句) : break, continue, pass, and the Loop else (break、continue、pass 与循环 else)
- continue statement, continue-The nested-code alternative (continue 语句——嵌套代码替代)
- nested code (嵌套代码) : The nested-code alternative (嵌套代码替代方案)
- do until (do-until 风格) : Examples (示例)
- else (else (否则)) : General Format (一般格式)
- else clause (else 子句) : Loop else (循环 else) ; Why the loop else? (为什么要有循环 else?)
- interactive loops (交互式循环) : A Simple Interactive Loop (简单交互循环)
- pass statement, pass-The ellipsis-literal alternative (pass 语句——省略号字面量替代)
- ellipsis (省略号) : The ellipsis-literal alternative (省略号字面量替代方案)
- sequence scans (序列 scans) : Sequence Scans: w

hile, range, and for (序列扫描: while、range 与 for)

- whitespace (空白) : Other Common String Methods (其他常用字符串方法) ; End of indentation is end of block (缩进结束即代码块结束)

- wide-character strings (宽字符字符串) : Character Representations (字符表示)

- Windows (Windows 系统)

- directory paths (目录路径) : Files in Action (文件实战)

- interactive coding and (与交互式编程) : Starting an Interactive REPL (启动交互式 REPL)

- Python installation (Python 安装) : Installing Python (安装 Python) ; Using Python on Windows (A.2 在 Windows 上使用 Python)

- Python on (上的 Python) : Using Python on Windows (A.2 在 Windows 上使用 Python)

- Command Prompt (命令提示符) : Using Python on Windows (A.2 在 Windows 上使用 Python)

- IDLE (A.2.6 IDLE: Python 自带 IDE) : Using Python on Windows (A.2 在 Windows 上使用 Python)

- WSL (WSL (Windows 的 Linux 子系统)) : Using Python on Windows (A.2 在 Windows 上使用 Python)

- Wing (Wing 集成开发环境) : Other IDEs for Python (其他 Python IDE)
- with statement (带语句) : The with Statement and Context Managers (with 语句与上下文管理器) ; Basic with Usage (with 基本用法)
- context managers, multiple (多个上下文管理器) : Multiple Context Managers (多个上下文管理器)
- context-management protocol (上下文管理协议) : The Context-Management Protocol (上下文管理协议)
- termination actions (终止动作) : The with Statement and Context Managers (with 语句与上下文管理器)
- termination handlers (终止处理器) : The Termination-Handlers Shoot-Out (终止处理器巅峰对决)
- working directory, commands (工作目录与命令) : Running Files with Command Lines (用命令行运行文件)
- wrappers (包装器) : OOP and Delegation: "Like-a" Relationships (OOP 与委托: "像一个" 关系) ; What Should Be Wrapped (应该包裹什么)
- call proxies (调用代理) : Managing Calls and Instances (管理调用与实例)
- class decorators (类装饰器) : Managing Calls and Instances (管理调用与实例)

- layers (层) : Decorator Nesting (装饰器嵌套)
- stacking (堆叠) : Decorator Nesting (装饰器嵌套)
- wrapping code, “Overwrapping-itis” (包装代码, “过度包装症”)
- WSL (Windows Subsystem for Linux) (WSL (Windows 的 Linux 子系统)) : Using Python on Windows (A.2 在 Windows 上使用 Python)
- wxPython (wxPython GUI 库) : GUIs and UIs (GUI 与 UI)

X (字母 X)

- X prefix (X 前缀) : Minimizing from Damage: X and all (最小化 from 的破坏: X 与 all)
- X pseudoprivate name mangling (X 伪私有名称改写) : Pseudoprivate Class Attributes (伪私有类属性) ; Using “X” pseudoprivate names (使用 “X” 伪私有名)
- XML-RPC (XML-RPC 协议) : Component Integration (组件集成)

Y (字母 Y)

- yield function, generator objects (yield 与生成器对象) : Coding Alternative: iter Plus yield (编码替代: iter 加 yield) ; Multiple iterators with yield (用 yield

d 实现多个迭代器)

- yield statement (yield 语句) : Advanced Function Tools (高级函数工具) ; Iteration protocol integration (与迭代协议的集成)
- generator functions (生成器函数) : The yield from extension (yield from 扩展)

Z (字母 Z)

- zip function (zip 函数)
- emulating (模拟) : Example: Emulating zip and map (示例: 模拟 zip 和 map) ; Coding Your Own zip and 2.X map (自己实现 zip 与 2.X 的 map)
- iterables (可迭代对象) : Coding Your Own zip and 2.X map (自己实现 zip 与 2.X 的 map)
- zip object (zip 对象) : Parallel Traversals: zip (并行遍历: zip) ; More zip roles: dictionaries (zip 的更多角色: 字典)
- iteration protocol (迭代协议) : Reprise: Dictionaries, range, enumerate, and zip (重温: 字典、range、enumerate 与 zip)
- ZODB (ZODB 面向对象数据库) : Database Access (数据库访问)
- Zope (Zope 应用服务器) : Internet and Web Scripting (互联网与 Web 脚本)

共收录 3119 个索引词条，按 Symbols + A-Z 排列。